JSC-61777-models/dynamics/derived_state

JSC Engineering Orbital Dynamics Derived State Model

Simulation and Graphics Branch (ER7)
Software, Robotics, and Simulation Division
Engineering Directorate

Package Release JEOD v5.4

Document Revision 1.1
January 2026



National Aeronautics and Space Administration
Lyndon B. Johnson Space Center
Houston, Texas

JSC-61777-models/dynamics/derived_state

JSC Engineering Orbital Dynamics
Derived State Model

Document Revision 1.1
January 2026

Gary Turner

Simulation and Graphics Branch (ER7)
Software, Robotics, and Simulation Division
Engineering Directorate

National Aeronautics and Space Administration
Lyndon B. Johnson Space Center
Houston, Texas

Abstract

When describing the state of a vehicle, we are describing the translational and rotational position and velocity with respect to some reference frame. For simplicity, the integration of a vehicular state is performed in an inertial reference frame, but the input and output of that state may be expressed in any of a boundless set of representations, such as LVLH or Orbital Elements. The Derived State model provides the capability to express a vehicular state in one or more of the most frequently used representations, and the extension capability to define other representations not released with JEOD v5.4.

Contents

I Overview **1**

1 Introduction **2**

1.1 Purpose and Objectives of the Derived State Model 2

1.2 Context within JEOD 2

1.3 Documentation History 3

1.4 Documentation Organization 3

2 Product Requirements **4**

2.1 Requirements Traceability 4

3 Product Specification **6**

3.1 Conceptual Design 6

3.2 Mathematical Formulations 7

3.3 Detailed Design 8

3.3.1 Functional Design 8

3.4 Inventory 10

4 User's Guide **19**

5 Verification and Validation **20**

5.1 Verification 20

5.2 Validation 21

5.3 Metrics 22

5.3.1 Code Metrics 22

II Euler Angles State Representation **27**

6 Introduction **28**

6.1 Purpose and Objectives of the Euler Angles State Representation 28

7 Product Requirements **29**

7.1 Requirements Traceability 29

8 Product Specification **30**

8.1 Conceptual Design 30

8.2 Mathematical Formulations 30

8.3 Detailed Design 30

8.3.1 Process Architecture 30

8.3.2 Functional Design 31

9 User's Guide **32**

9.1 Analysis 32

9.1.1 Identifying the Euler Angles State Representation 32

9.1.2 Editing the Euler Angles State Representation 33

9.1.3 Output Data 33

9.2 Integration 34

9.3 Extension 34

10 Verification and Validation **35**

10.1 Verification 35

10.2 Validation 36

III Local-Vertical-Local-Horizontal (LVLH) Reference Frame **42**

11 Introduction **43**

11.1 Purpose and Objectives of the Local-Vertical-Local-Horizontal (LVLH) Reference Frame 43

12 Product Requirements **44**

12.1 Requirements Traceability 44

13 Product Specification **45**

13.1 Conceptual Design . . . 45

13.2 Mathematical Formulations . . . 45

13.3 Detailed Design . . . 47

13.3.1 Process Architecture . . . 47

13.3.2 Functional Design . . . 47

14 User's Guide **49**

14.1 Analysis . . . 49

14.1.1 Identifying the Local-Vertical-Local-Horizontal (LVLH) Reference Frame . . . 50

14.1.2 Editing the Local-Vertical-Local-Horizontal (LVLH) Reference Frame . . . 50

14.1.3 Output Data . . . 50

14.2 Integration . . . 51

14.2.1 Generating the S_define . . . 51

14.2.2 Generating the Input File . . . 52

14.2.3 Logging the Data . . . 52

14.3 Extension . . . 52

15 Verification and Validation **53**

15.1 Verification . . . 53

15.2 Validation . . . 54

IV Local-Vertical-Local-Horizontal (LVLH) Relative Derived State **55**

16 Introduction **56**

16.1 Purpose and Objectives of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State . . . 56

17 Product Requirements **57**

17.1 Requirements Traceability . . . 57

18 Product Specification **58**

18.1 Conceptual Design . . . 58

18.2 Mathematical Formulations . . . 58

18.2.1 Curvilinear Coordinates . . . 59
18.2.2 Translational State . . . 59
18.2.3 Rotational State . . . 61
18.2.4 *convert_circ_to_rect* . . . 62
18.3 Detailed Design . . . 63
18.3.1 Process Architecture . . . 63
18.3.2 Functional Design . . . 63

19 User's Guide **65**
19.1 Using the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State in a Simulation . . . 65
19.2 Other Options . . . 66
19.3 Output Data . . . 66

20 Verification and Validation **67**
20.1 Verification . . . 67
20.2 Validation . . . 68

V North-East-Down Reference Frame **69**

21 Introduction **70**
21.1 Purpose and Objectives of the North-East-Down Reference Frame . . . 70

22 Product Requirements **72**
22.1 Requirements Traceability . . . 72

23 Product Specification **74**
23.1 Conceptual Design . . . 74
23.2 Mathematical Formulations . . . 74
23.3 Detailed Design . . . 74
23.3.1 Process Architecture . . . 74
23.3.2 Functional Design . . . 75

24 User's Guide **77**
24.1 Analysis . . . 77

24.1.1 Identifying the North-East-Down Reference Frame . . . 78
24.1.2 Editing the North-East-Down Reference Frame . . . 78
24.1.3 Output Data . . . 78
24.2 Integration . . . 79
24.2.1 Generating the S_define . . . 79
24.2.2 Generating the Input File . . . 80
24.2.3 Logging the Data . . . 81
24.3 Extension . . . 81

25 Verification and Validation **82**
25.1 Verification . . . 82
25.1.1 Inspection of Modeling Requirements . . . 82
25.1.2 Verification of Model . . . 82
25.2 Validation . . . 90

VI Orbital Elements State Representation **91**

26 Introduction **92**
26.1 Purpose and Objectives of the Orbital Elements State Representation . . . 92

27 Product Requirements **93**
27.1 Requirements Traceability . . . 93

28 Product Specification **94**
28.1 Conceptual Design . . . 94
28.2 Mathematical Formulations . . . 94
28.3 Detailed Design . . . 94
28.3.1 Process Architecture . . . 94
28.3.2 Functional Design . . . 94

29 User's Guide **96**
29.1 Analysis . . . 96
29.1.1 Identifying the Orbital Elements State Representation . . . 96
29.1.2 Editing the Orbital Elements State Representation . . . 97

29.1.3 Output Data . . . 97
29.2 Integration . . . 97
29.2.1 Generating the S_define . . . 98
29.2.2 Generating the Input File . . . 98
29.2.3 Logging the Data . . . 98
29.3 Extension . . . 98

30 Verification and Validation **99**
30.1 Verification . . . 99
30.2 Validation . . . 99

VII Planetary Reference Frame State Representation **100**

31 Introduction **101**
31.1 Purpose and Objectives of the Planetary Reference Frame State Representation . . . 101

32 Product Requirements **103**
32.1 Requirements Traceability . . . 103

33 Product Specification **104**
33.1 Conceptual Design . . . 104
33.2 Mathematical Formulations . . . 104
33.3 Detailed Design . . . 104
33.3.1 Process Architecture . . . 104
33.3.2 Functional Design . . . 104

34 User's Guide **106**
34.1 Analysis . . . 106
34.1.1 Identifying the Planetary Reference Frame State Representation . . . 106
34.1.2 Editing the Planetary Reference Frame State Representation . . . 107
34.1.3 Output Data . . . 107
34.2 Integration . . . 107
34.2.1 Generating the S_define . . . 108
34.2.2 Generating the Input File . . . 108

34.2.3 Logging the Data . . . 108
34.3 Extension . . . 108

35 Verification and Validation **109**
35.1 Verification . . . 109
35.1.1 Inspection of Modeling Requirements . . . 109
35.1.2 Verification of Surface Representations . . . 109
35.2 Validation . . . 122

VIII Relative Reference Frame State Representation **123**

36 Introduction **124**
36.1 Purpose and Objectives of the Relative Reference Frame State Representation . . . 124

37 Product Requirements **125**
37.1 Requirements Traceability . . . 125

38 Product Specification **126**
38.1 Conceptual Design . . . 126
38.2 Mathematical Formulations . . . 128
38.3 Detailed Design . . . 128
38.3.1 Process Architecture . . . 128
38.3.2 Functional Design . . . 128

39 User's Guide **130**
39.1 Analysis . . . 130
39.1.1 Identifying the Relative Reference Frame State Representation . . . 130
39.1.2 Editing the Relative Reference Frame State Representation . . . 131
39.1.3 Output Data . . . 132
39.2 Integration . . . 132
39.2.1 Generating the S_define . . . 132
39.2.2 Generating the Input File . . . 132
39.2.3 Logging the Data . . . 133
39.3 Extension . . . 133

40 Verification and Validation **135**
40.1 Verification . . . 135
40.1.1 Inspection of Behavior in Integrated Testing . . . 135
40.1.2 Analytical Verification of Comprehensive Behavior in Unit Testing . . . 135
40.2 Validation . . . 141

IX Solar Beta Angle Representation **147**

41 Introduction **148**
41.1 Purpose and Objectives of the Solar Beta Angle Representation . . . 148

42 Product Requirements **149**
42.1 Requirements Traceability . . . 149

43 Product Specification **150**
43.1 Conceptual Design . . . 150
43.2 Mathematical Formulations . . . 151
43.3 Detailed Design . . . 152
43.3.1 Process Architecture . . . 152
43.3.2 Functional Design . . . 152

44 User's Guide **154**
44.1 Analysis . . . 154
44.1.1 Identifying the Solar Beta Angle Representation . . . 154
44.1.2 Editing the Solar Beta Angle Representation . . . 155
44.1.3 Output Data . . . 155
44.2 Integration . . . 155
44.2.1 Generating the S_define . . . 155
44.2.2 Generating the Input File . . . 156
44.2.3 Logging the Data . . . 156
44.3 Extension . . . 156

45 Verification and Validation **157**
45.1 Verification . . . 157

45.2 Validation 158

Part I

Overview

Chapter 1

Introduction

1.1 Purpose and Objectives of the Derived State Model

The Derived State model provides the capability to express a vehicular state in one or more of the most frequently used reference frame concepts (such as LVLH), with respect to another object included in the simulation. While effort has been made to identify and include those reference frame representations most frequently utilized in orbital dynamics simulations, we recognize that we have not provided complete coverage of what is a boundless number of possible representations, and so have provided the flexibility that allows for the further definition of additional reference frame representations for inclusion into the model as needed.

The following reference frame representations are included in the Derived State Model:

1. Euler Angles representation
2. Local-Vertical-Local-Horizontal (LVLH) Reference Frame
3. North-East-Down Reference Frame
4. Orbital Elements state representation
5. Planetary Reference Frame state representation
6. Relative Reference Frame state representation
7. Solar Beta Angle representation

1.2 Context within JEOD

The following document is parent to this document:

- *JSC Engineering Orbital Dynamics* [5]

The Derived State Model forms a component of the dynamics suite of models within JEOD v5.4. It is located at models/dynamics/derived_state.

1.3 Documentation History

Author	Date	Revision	Description
Gary Turner	November, 2009	1.0	JEOD2.0.0 release
Robert Shelton	April, 2016	1.1	Added Documentation for LVLH Relative Derived State

1.4 Documentation Organization

This document is formatted in accordance with the NASA Software Engineering Requirements Standard [8]. The document comprises several parts. The first part is an overview of the model; each subsequent part describes a sub-model of the Derived State Model, with each sub-model representing an independent reference frame or state representation. Each part is organized into a similar structure, comprising the following chapters in order:

Introduction - This introduction describes the objective and purpose of the object of the respective part; the introduction in Part I - Overview also contains the document history.

Product Requirements - Describes requirements for the object of the respective part.

Product Specification - Describes the underlying theory, architecture, and design of the object of the respective part in detail. It will be organized into the following sections:

- Conceptual Design.
- Mathematical Formulations.
- Detailed Design.
- Version Inventory (Part I - Overview only).

User's Guide - Describes how to use the object of the respective part in a Trick simulation. The User's Guide for each sub-model is divided into three sections to represent the JEOD defined user types:

- Analysts or users of simulations (Analysis).
- Integrators or developers of simulations (Integration).
- Model Extenders (Extension).

Verification and Validation - Contains verification and validation procedures and results for the object of the resepctive part.

Chapter 2

Product Requirements

Requirement DerivedState_1: Top-level requirement

Requirement:
This model shall meet the JEOD project requirements specified in the JEOD v5.4 top-level document.

Rationale:
This model shall, at a minimum, meet all external and internal requirements applied to the JEOD v5.4 release.

Verification:
Inspection

Requirement DerivedState_2: Minimal Functionality

Requirement:
The JEOD v5.4 release of the Derived State Model shall include, at a minimum, the same functionality as was available in version JEODv1.5.2.

Rationale:
The new framework should not detract from the functionality of the package.

Verification:
Inspection.

2.1 Requirements Traceability

Table 2.1: Requirements Traceability

Requirement	**Inspection and Testing**
DerivedState_1 - Top-level Requirements	Insp. DerivedState_1
DerivedState_2 - Minimum Functionality	Insp. DerivedState_2

Chapter 3

Product Specification

3.1 Conceptual Design

The Derived State Model shall provide the interface by which the state of any given object can be expressed in any given reference frame, or a new reference frame created with respect to any other reference frame if the desired representation is not already available. Two of the sub-classes (Local-Vertical-Local-Horizontal (LVLH) Reference Frame and North-East-Down Reference Frame) are specialized to the point of providing only a reference frame; two (Planetary Reference Frame state representation and Solar Beta Angle representation) are closely tied to planetary bodies and rely on the pre-definition of those reference frames; Orbital Elements state representation provides the state of a vehicle in classical orbital elements, and tacitly assumes that there is a planetary body involved for the vehicle to be orbiting; the other two (Euler Angles representation and Relative Reference Frame state representation) express the state of one reference frame relative to any other.

The generic Derived State provides a name for the state, based on entries it receives from the sub-classes for the names of the *subject frame* (the reference frame associated with the vehicle whose state is to be enumerated) and the *reference frame* (the frame in which that state is to be evaluated). Each of the specific examples of derived states will inherit from the generic derived state, specifying those frames (although the reference frame need not be specified for all instances), thereby providing the data to the generic DerivedState for generating the state name. Excepting the Local-Vertical-Local-Horizontal (LVLH) Reference Frame and the North-East-Down Reference Frame each example of a DerivedState will then output a set of values that identify those parts of the subject's state for which it is responsible. The Local-Vertical-Local-Horizontal (LVLH) Reference Frame and North-East-Down Reference Frame define their respective reference frames, for access by a state-generating Derived State, such as the Relative Reference Frame state representation.

While the sub-classes of Derived State (i.e., the reference frames and state representations) are largely independent of one another, there are places where there is some interdependency, and the methods and definitions from one sub-class are similar to, or dependent upon, methods and definitions from another class. In those situations, links to the documentation appropriate to the situation are included as necessary.

3.2 Mathematical Formulations

The top-level DerivedState class contains no mathematical manipulations. All of the mathematics in this model is handled within the appropriate sub-class. See the individual Mathematical Formulations sections in the respective parts of this document for details.

3.3 Detailed Design

3.3.1 Functional Design

See the Reference Manual[1] for a summary of member data and member methods for all classes. This section describes the functional operation of the methods identified in the base-class, Derived-State.

The Derived State provides common functionality to its sub-classes.

- **find_planet**

  Uses the Dynamics Manager method *find_planet* to find a planet with name *planet_name*

- **initialize**

  The *initialize* method receives a reference to the subject body and to the dynamics manager. Its primary responsibility is to set the identifying name of the state, *state_identifier*. This comprises the following elements:

  1. The type name
     The type name would be something like *LvlhDerivedState*, or *RelativeDerivedState*.
  2. The name of the subject body
     This is set by the user, often in the input file, or a Modified data file. Both are included in the verification simulations: for example, the Solar Beta verification simulation uses the input file (textitverif/SIM_SolarBeta/SET_test/RUN_comp_ISS/input)

     ```
     veh.dyn_body.set_name( "solar_beta_test_vehicle" );
     ```

     whereas the LVLH verification simulation uses a Modified Data file entry (*verif/SIM_LVLH/Modified_data/veh.d*)

     ```
     veh.dyn_body.set_name( "vehicle1" );
     ```

  3. The reference name *(optional)*
     The reference name is used in identifying a reference frame by name, and in naming the state identifier. By coincidence, for those sub-classes of DerivedState included in JEOD v5.4 when the reference name is being used to identify a reference frame, it is always used to identify a planet. However, this is a coincidence, not a rule; it is not set in stone and any extensions to JEOD v5.4 need not adhere to this statement.

     Not all subclasses of DerivedState require the *reference_name* in all situations, some require it only when the user requires something other than the default reference frame, and some never require it. For those subclasses and situations that do not require a reference name, the inclusion of one will only be used in the state identifier. Those that do not have a defined reference name will have only a 2-member identifier, whereas those with a reference name will have a 3-member identifier. This value is also set by the user, again usually in the input file or a Modified data file. For example, from *verif/SIM_SolarBeta/SET_test/RUN_comp_ISS/input*:

     ```
     veh.solar_beta.reference_name = "Earth";
     ```

Continuing with the example from *verif/SIM_SolarBeta/SET_test/RUN_comp_ISS/input*, the simulation has a SolarBetaDerivedState, for which this initialize method is called. The resulting *state_identifier* value that is output is

```
SolarBetaDerivedState.solar_beta_test_vehicle.Earth
```

- **update**

  This method is included for consistency; it has no other purpose.

- **validate_name**

  Used by *find_planet* to ensure that names have been specified.

3.4 Inventory

All Derived State Model files are located in the directory `${JEOD_HOME}/models/dynamics/derived_state`. Relative to this directory,

- Header and source files are located in the model `include` and `src` subdirectories. Table 3.1 lists the configuration-managed files in these directories.
- Documentation files are located in the model `docs` subdirectory. See table 3.2 for a listing of the configuration-managed files in this directory.
- Verification files are located in the model `verif` subdirectory. See table 3.3 for a listing of the configuration-managed files in this directory.

Table 3.1: Source Files

File Name
include/class_declarations.hh
include/derived_state.hh
include/derived_state_messages.hh
include/euler_derived_state.hh
include/lvlh_derived_state.hh
include/lvlh_relative_derived_state.hh
include/ned_derived_state.hh
include/orb_elem_derived_state.hh
include/planetary_derived_state.hh
include/relative_derived_state.hh
include/solar_beta_derived_state.hh
src/cmake_file_list.cmake
src/derived_state.cc
src/derived_state_messages.cc
src/euler_derived_state.cc
src/lvlh_derived_state.cc
src/lvlh_relative_derived_state.cc
src/ned_derived_state.cc
src/orb_elem_derived_state.cc
src/planetary_derived_state.cc
src/relative_derived_state.cc
src/solar_beta_derived_state.cc

Table 3.2: Documentation Files

File Name
docs/derived_state.pdf
docs/refman.pdf
docs/tex/DerivedState.bib
docs/tex/DerivedStateAbstract.tex
docs/tex/DerivedStateIntro_change_history.tex
docs/tex/DerivedStateIntroduction.tex
docs/tex/DerivedStateReqt.tex
docs/tex/DerivedStateReqt_trace.tex
docs/tex/DerivedStateSpec_Concept.tex
docs/tex/DerivedStateSpec_Detail.tex
docs/tex/DerivedStateSpec_Inventory.tex
docs/tex/DerivedStateSpec_Math.tex
docs/tex/DerivedStateVV_Metrics.tex
docs/tex/DerivedStateVV_Valid.tex
docs/tex/DerivedStateVV_Verif.tex
docs/tex/Euler.tex
docs/tex/EulerIntroduction.tex
docs/tex/EulerReqt.tex
docs/tex/EulerReqt_trace.tex
docs/tex/EulerSpec_Concept.tex
docs/tex/EulerSpec_Detail.tex
docs/tex/EulerSpec_Math.tex
docs/tex/EulerUser_Analysis.tex
docs/tex/EulerUser_Extension.tex
docs/tex/EulerUser_Integration.tex
docs/tex/EulerVV_Valid.tex
docs/tex/EulerVV_Verif.tex
docs/tex/LRDS.tex
docs/tex/LRDSIntroduction.tex
docs/tex/LRDSReqt.tex
docs/tex/LRDSReqt_trace.tex
docs/tex/LRDSSpec_Concept.tex
docs/tex/LRDSSpec_Detail.tex
docs/tex/LRDSSpec_Math.tex
docs/tex/LRDSUser.tex

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/LRDSVV_Valid.tex
docs/tex/LRDSVV_Verif.tex
docs/tex/LVLH.tex
docs/tex/LVLHIntroduction.tex
docs/tex/LVLHReqt.tex
docs/tex/LVLHReqt_trace.tex
docs/tex/LVLHSpec_Concept.tex
docs/tex/LVLHSpec_Detail.tex
docs/tex/LVLHSpec_Math.tex
docs/tex/LVLHUser_Analysis.tex
docs/tex/LVLHUser_Extension.tex
docs/tex/LVLHUser_Integration.tex
docs/tex/LVLHVV_Valid.tex
docs/tex/LVLHVV_Verif.tex
docs/tex/NED.tex
docs/tex/NEDIntroduction.tex
docs/tex/NEDReqt.tex
docs/tex/NEDReqt_trace.tex
docs/tex/NEDSpec_Concept.tex
docs/tex/NEDSpec_Detail.tex
docs/tex/NEDSpec_Math.tex
docs/tex/NEDUser_Analysis.tex
docs/tex/NEDUser_Extension.tex
docs/tex/NEDUser_Integration.tex
docs/tex/NEDVV_Valid.tex
docs/tex/NEDVV_Verif.tex
docs/tex/OrbElem.tex
docs/tex/OrbElemIntroduction.tex
docs/tex/OrbElemReqt.tex
docs/tex/OrbElemReqt_trace.tex
docs/tex/OrbElemSpec_Concept.tex
docs/tex/OrbElemSpec_Detail.tex
docs/tex/OrbElemSpec_Math.tex
docs/tex/OrbElemUser_Analysis.tex

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/OrbElemUser_Extension.tex
docs/tex/OrbElemUser_Integration.tex
docs/tex/OrbElemVV_Valid.tex
docs/tex/OrbElemVV_Verif.tex
docs/tex/Planetary.tex
docs/tex/PlanetaryIntroduction.tex
docs/tex/PlanetaryReqt.tex
docs/tex/PlanetaryReqt_trace.tex
docs/tex/PlanetarySpec_Concept.tex
docs/tex/PlanetarySpec_Detail.tex
docs/tex/PlanetarySpec_Math.tex
docs/tex/PlanetaryUser_Analysis.tex
docs/tex/PlanetaryUser_Extension.tex
docs/tex/PlanetaryUser_Integration.tex
docs/tex/PlanetaryVV_Valid.tex
docs/tex/PlanetaryVV_Verif.tex
docs/tex/Relative.tex
docs/tex/RelativeIntroduction.tex
docs/tex/RelativeReqt.tex
docs/tex/RelativeReqt_trace.tex
docs/tex/RelativeSpec_Concept.tex
docs/tex/RelativeSpec_Detail.tex
docs/tex/RelativeSpec_Math.tex
docs/tex/RelativeUser_Analysis.tex
docs/tex/RelativeUser_Extension.tex
docs/tex/RelativeUser_Integration.tex
docs/tex/RelativeVV_Valid.tex
docs/tex/RelativeVV_Verif.tex
docs/tex/SolarBeta.tex
docs/tex/SolarBetaIntroduction.tex
docs/tex/SolarBetaReqt.tex
docs/tex/SolarBetaReqt_trace.tex
docs/tex/SolarBetaSpec_Concept.tex
docs/tex/SolarBetaSpec_Detail.tex

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/SolarBetaSpec_Math.tex
docs/tex/SolarBetaUser_Analysis.tex
docs/tex/SolarBetaUser_Extension.tex
docs/tex/SolarBetaUser_Integration.tex
docs/tex/SolarBetaVV_Valid.tex
docs/tex/SolarBetaVV_Verif.tex
docs/tex/derived_state.sty
docs/tex/derived_state.tex
docs/tex/makefile
docs/tex/model_name.mk
docs/tex/figures/SOLAR_BETA_fig5.pdf
docs/tex/figures/SOLAR_BETA_fig6.pdf
docs/tex/figures/SOLAR_BETA_fig7.pdf
docs/tex/figures/euler_aligned_pyr.jpg
docs/tex/figures/euler_aligned_rpy.jpg
docs/tex/figures/euler_nonaligned_pyr.jpg
docs/tex/figures/euler_nonaligned_rpy.jpg
docs/tex/figures/euler_nonaligned_yrp.jpg
docs/tex/figures/ned_elliptical_rectilinear.jpg
docs/tex/figures/ned_equ_sph.jpg
docs/tex/figures/ned_inc_sph.jpg
docs/tex/figures/ned_polar_ell.jpg
docs/tex/figures/ned_polar_ell_zoom.jpg
docs/tex/figures/ned_polar_sph.jpg
docs/tex/figures/planetary_geo_cart.jpg
docs/tex/figures/planetary_geo_inrtl.jpg
docs/tex/figures/planetary_geo_lal.jpg
docs/tex/figures/planetary_leo_ecc_cart.jpg
docs/tex/figures/planetary_leo_ecc_inrtl.jpg
docs/tex/figures/planetary_leo_ecc_lal.jpg
docs/tex/figures/planetary_leo_equ_cart.jpg
docs/tex/figures/planetary_leo_equ_inrtl.jpg
docs/tex/figures/planetary_leo_equ_lal.jpg
docs/tex/figures/planetary_leo_inc_cart.jpg

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/figures/planetary_leo_inc_inrtl.jpg
docs/tex/figures/planetary_leo_inc_lal.jpg
docs/tex/figures/planetary_leo_polar_cart.jpg
docs/tex/figures/planetary_leo_polar_inrtl.jpg
docs/tex/figures/planetary_leo_polar_lal.jpg
docs/tex/figures/rectilinear.jpg
docs/tex/figures/rectilinear.odg
docs/tex/figures/ref_frame.odg
docs/tex/figures/rel_state_ref_orient.jpg
docs/tex/figures/relative6_vab.png
docs/tex/figures/relative6_vba.png
docs/tex/figures/relative6_xab.png
docs/tex/figures/relative6_xba.png
docs/tex/figures/relative7_vab.png
docs/tex/figures/relative7_vba.png
docs/tex/figures/relative7_wab.png
docs/tex/figures/relative7_wba.png
docs/tex/figures/relative7_xab.png
docs/tex/figures/relative7_xba.png
docs/tex/figures/relative_config.png
docs/tex/figures/solar_beta.jpg
docs/tex/figures/solar_beta.odg
docs/tex/figures/solar_beta_iss.jpg
docs/tex/figures/solbeta123.jpg
docs/tex/figures/sphericalelliptical.jpg
docs/tex/figures/sphericalelliptical.odg

Table 3.3: Verification Files

File Name
verif/Modified_data/date_and_time.py
verif/Modified_data/veh_config.py
verif/Modified_data/veh_mass.py
verif/Modified_data/veh_state.py

Continued on next page

Table 3.3: Verification Files (continued from previous page)

File Name
verif/SIM_Euler/Modified_data
verif/SIM_Euler/S_define
verif/SIM_Euler/S_overrides.mk
verif/SIM_Euler/input_common.py
verif/SIM_Euler/DP_Product/DP_validation.xml
verif/SIM_Euler/Log_data/log_orbital_state_rec.py
verif/SIM_Euler/SET_test/RUN_ecc/input.py
verif/SIM_Euler/SET_test/RUN_equ/input.py
verif/SIM_Euler/SET_test/RUN_inc/input.py
verif/SIM_Euler/SET_test/RUN_offset_equ/input.py
verif/SIM_LVLH/Modified_data
verif/SIM_LVLH/S_define
verif/SIM_LVLH/S_overrides.mk
verif/SIM_LVLH/input_common.py
verif/SIM_LVLH/DP_Product/DP_validation.xml
verif/SIM_LVLH/Log_data/log_orbital_state_rec.py
verif/SIM_LVLH/SET_test/RUN_ecc/input.py
verif/SIM_LVLH/SET_test/RUN_equ/input.py
verif/SIM_LVLH/SET_test/RUN_inc/input.py
verif/SIM_LvlhRelative/Modified_data
verif/SIM_LvlhRelative/S_define
verif/SIM_LvlhRelative/S_overrides.mk
verif/SIM_LvlhRelative/Log_data/log_data.py
verif/SIM_LvlhRelative/SET_test/RUN_test0/input.py
verif/SIM_LvlhRelative/SET_test/RUN_test1/input.py
verif/SIM_LvlhRelative/test_code/main.cc
verif/SIM_LvlhRelative/test_code/makefile
verif/SIM_NED/Modified_data
verif/SIM_NED/S_define
verif/SIM_NED/S_overrides.mk
verif/SIM_NED/input_common.py
verif/SIM_NED/DP_Product/DP_SIM_NED.xml
verif/SIM_NED/DP_Product/DP_validation.xml
verif/SIM_NED/Log_data/log_orbital_state_rec.py
verif/SIM_NED/SET_test/RUN_ell_equ/input.py

Continued on next page

Table 3.3: Verification Files (continued from previous page)

File Name
verif/SIM_NED/SET_test/RUN_ell_inc/input.py
verif/SIM_NED/SET_test/RUN_ell_polar/input.py
verif/SIM_NED/SET_test/RUN_sph_equ/input.py
verif/SIM_NED/SET_test/RUN_sph_inc/input.py
verif/SIM_NED/SET_test/RUN_sph_polar/input.py
verif/SIM_OrbElem/Modified_data
verif/SIM_OrbElem/S_define
verif/SIM_OrbElem/S_overrides.mk
verif/SIM_OrbElem/DP_Product/DP_validation.xml
verif/SIM_OrbElem/Log_data/log_orbital_state_rec.py
verif/SIM_OrbElem/SET_test/RUN_ecc/input.py
verif/SIM_Planetary/Modified_data
verif/SIM_Planetary/S_define
verif/SIM_Planetary/S_overrides.mk
verif/SIM_Planetary/input_common.py
verif/SIM_Planetary/DP_Product/DP_validation.xml
verif/SIM_Planetary/Log_data/log_orbital_state_rec.py
verif/SIM_Planetary/SET_test/RUN_GEO/input.py
verif/SIM_Planetary/SET_test/RUN_LEO_ecc/input.py
verif/SIM_Planetary/SET_test/RUN_LEO_equ/input.py
verif/SIM_Planetary/SET_test/RUN_LEO_inc/input.py
verif/SIM_Planetary/SET_test/RUN_LEO_polar/input.py
verif/SIM_Relative/Modified_data
verif/SIM_Relative/S_define
verif/SIM_Relative/S_overrides.mk
verif/SIM_Relative/input_common.py
verif/SIM_Relative/DP_Product/DP_validation.xml
verif/SIM_Relative/Log_data/log_data.py
verif/SIM_Relative/SET_test/RUN_AB_rot_AB_trans/input.py
verif/SIM_Relative/SET_test/RUN_AB_rot_no_trans/input.py
verif/SIM_Relative/SET_test/RUN_A_rot_no_trans/input.py
verif/SIM_Relative/SET_test/RUN_B_rot_no_trans/input.py
verif/SIM_Relative/SET_test/RUN_no_rot_AB_trans/input.py
verif/SIM_Relative/SET_test/RUN_no_rot_A_trans/input.py
verif/SIM_Relative/SET_test/RUN_no_rot_B_trans/input.py

Continued on next page

Table 3.3: Verification Files (continued from previous page)

File Name
verif/SIM_SolarBeta/Modified_data
verif/SIM_SolarBeta/S_define
verif/SIM_SolarBeta/S_overrides.mk
verif/SIM_SolarBeta/input_common.py
verif/SIM_SolarBeta/DP_Product/DP_validation.xml
verif/SIM_SolarBeta/Log_data/log_orbital_state_rec.py
verif/SIM_SolarBeta/SET_test/RUN_comp_ISS/input.py
verif/SIM_SolarBeta/SET_test/RUN_comp_STS121a/input.py
verif/SIM_SolarBeta/SET_test/RUN_comp_STS121b/input.py
verif/SIM_SolarBeta/SET_test/RUN_incl_0/input.py
verif/SIM_SolarBeta/SET_test/RUN_incl_23_4/input.py
verif/SIM_SolarBeta/SET_test/RUN_incl_23_4_spher/input.py
verif/SIM_SolarBeta/SET_test/RUN_incl_23_4_spher_moon/input.py
verif/SIM_SolarBeta/SET_test/RUN_incl_23_4_spher_sunmoon/input.py
verif/SIM_SolarBeta/SET_test/RUN_incl_51_6/input.py
verif/SIM_SolarBeta/SET_test/RUN_incl_51_6_sphericgrav/input.py
verif/unit_tests/CMakeLists.txt
verif/unit_tests/derived_state_ut.cc
verif/unit_tests/euler_derived_state_ut.cc
verif/unit_tests/lvlh_derived_state_ut.cc
verif/unit_tests/lvlh_relative_derived_state_ut.cc
verif/unit_tests/makefile
verif/unit_tests/ned_derived_state_ut.cc
verif/unit_tests/orb_elem_derived_state_ut.cc
verif/unit_tests/planetary_derived_state_ut.cc
verif/unit_tests/relative_derived_state_ut.cc
verif/unit_tests/solar_beta_derived_state_ut.cc

Chapter 4

User's Guide

The top level of the Derived State Model should not be used directly in a simulation. Specific instances of the Derived State Model should be used instead, although the implementation varies from instance to instance. See the User's Guides in the respective Part of this document for details of specific implementation.

Chapter 5

Verification and Validation

5.1 Verification

Inspection DerivedState_1: Top-level Requirement

This document structure, the code, and associated files have been inspected, and together satisfy requirement DerivedState_1.

Inspection DerivedState_2: Minimum Functionality

This model provides:

1. The following reference frames:
   (a) Local-Vertical-Local-Horizontal (LVLH)
   (b) North-East-Down(NED)
2. The translational states in the following formats, using reference frames defined elsewhere in JEOD v5.4:
   (a) Classical Orbital Elements
   (b) Planetary altitude-latitude-longitude
3. The rotational states utilizing a sequence of 3 rotation angles about coordinate axes,
4. The Solar Beta angle associated with any given state,
5. The ability to express the state of one reference frame in terms of another.

Together, these capabilities satisfy requirement DerivedState_2.

5.2 Validation

Validation, where applicable, is performed for the specific implementations of the model. See the Validation sections in each respective Part of this document for validation data appropriate to each sub-class.

5.3 Metrics

5.3.1 Code Metrics

Table 5.1 presents coarse metrics on the source files that comprise the model.

Table 5.1: Coarse Metrics

	Number of Lines			
File Name	**Blank**	**Comment**	**Code**	**Total**
include/class_declarations.hh	11	57	14	82
include/derived_state.hh	25	92	36	153
include/derived_state_messages.hh	25	95	22	142
include/euler_derived_state.hh	28	97	34	159
include/lvlh_derived_state.hh	25	94	29	148
include/lvlh_relative_derived_state.hh	25	81	34	140
include/ned_derived_state.hh	30	102	40	172
include/orb_elem_derived_state.hh	28	99	38	165
include/planetary_derived_state.hh	26	94	32	152
include/relative_derived_state.hh	38	129	58	225
include/solar_beta_derived_state.hh	25	123	34	182
src/cmake_file_list.cmake	2	0	16	18
src/derived_state.cc	22	70	50	142
src/derived_state_messages.cc	15	27	14	56
src/euler_derived_state.cc	22	68	41	131
src/lvlh_derived_state.cc	18	50	38	106
src/lvlh_relative_derived_state.cc	46	102	152	300
src/ned_derived_state.cc	26	66	59	151
src/orb_elem_derived_state.cc	22	62	52	136
src/planetary_derived_state.cc	20	55	39	114
src/relative_derived_state.cc	35	84	110	229
src/solar_beta_derived_state.cc	26	65	71	162
Total	540	1712	1013	3265

Table 5.2 presents the extended cyclomatic complexity (ECC) of the methods defined in the model.

Table 5.2: Cyclomatic Complexity

Method	File	Line	ECC
jeod::RelativeDerivedState::set_name(std::set_name (std::string name_in)	include/relative_derived_state.hh	171	1
jeod::RelativeDerivedState::set_target_frame (RefFrame & tf)	include/relative_derived_state.hh	179	1
jeod::RelativeDerivedState::set_subject_frame (BodyRef Frame & sf)	include/relative_derived_state.hh	189	1
jeod::DerivedState::set_reference_name (const std:: string & new_name)	src/derived_state.cc	52	1
jeod::DerivedState::initialize (DynBody & subject_body, DynManager & dyn_manager JEOD_UNUSED)	src/derived_state.cc	61	2
jeod::DerivedState::update ()	src/derived_state.cc	85	1
jeod::DerivedState::find_planet (const DynManager & dyn_manager, const std:: string & planet_name, const std::string & variable_name)	src/derived_state.cc	98	2
jeod::EulerDerivedState::~EulerDerivedState ()	src/euler_derived_state.cc	44	2
jeod::EulerDerivedState::initialize (DynBody & subject_body, DynManager & dyn_manager)	src/euler_derived_state.cc	56	1
jeod::EulerDerivedState::initialize (RefFrame & ref_frame, DynBody & subject_body, DynManager & dyn_manager)	src/euler_derived_state.cc	70	1
jeod::EulerDerivedState::update ()	src/euler_derived_state.cc	91	3
jeod::LvlhDerivedState::~Lvlh DerivedState ()	src/lvlh_derived_state.cc	47	2

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::LvlhDerivedState:: initialize (DynBody & subject_body, DynManager & dyn_manager)	src/lvlh_derived_state.cc	62	2
jeod::LvlhDerivedState:: update ()	src/lvlh_derived_state.cc	90	1
jeod::LvlhRelativeDerived State::LvlhRelativeDerived State ()	src/lvlh_relative_derived_ state.cc	50	1
jeod::LvlhRelativeDerived State::initialize (DynBody & subject_body, Dyn Manager & dyn_manager)	src/lvlh_relative_derived_ state.cc	59	1
jeod::LvlhRelativeDerived State::update ()	src/lvlh_relative_derived_ state.cc	72	4
jeod::LvlhRelativeDerived State::convert_rect_to_circ (const RefFrameState & rect_rel_state)	src/lvlh_relative_derived_ state.cc	111	2
jeod::LvlhRelativeDerived State::convert_circ_to_rect (const RefFrameState & curvi_rel_state)	src/lvlh_relative_derived_ state.cc	186	2
jeod::LvlhRelativeDerived State::do_theta_dot_ correction (double omega[3], const RefFrame State & s, const double r, bool c2r)	src/lvlh_relative_derived_ state.cc	263	3
jeod::NedDerivedState::~Ned DerivedState ()	src/ned_derived_state.cc	48	3
jeod::NedDerivedState::set_ use_alt_pfix (const bool use_ alt_pfix_in)	src/ned_derived_state.cc	68	1
jeod::NedDerivedState:: initialize (DynBody & subject_body, DynManager & dyn_manager)	src/ned_derived_state.cc	76	3
jeod::NedDerivedState:: update ()	src/ned_derived_state.cc	122	1

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::NedDerivedState:: compute_ned_frame (const RefFrameTrans & rel_trans)	src/ned_derived_state.cc	135	1
jeod::OrbElemDerivedState::~ OrbElemDerivedState ()	src/orb_elem_derived_state.cc	45	2
jeod::OrbElemDerivedState:: set_use_alt_inertial (const bool use_alt_inertial_in)	src/orb_elem_derived_state.cc	56	1
jeod::OrbElemDerivedState:: initialize (DynBody & subject_body, DynManager & dyn_manager)	src/orb_elem_derived_state.cc	64	2
jeod::OrbElemDerivedState:: update ()	src/orb_elem_derived_state.cc	99	2
jeod::OrbElemDerivedState:: compute_orbital_elements (const RefFrameTrans & rel_trans)	src/orb_elem_derived_state.cc	120	1
jeod::PlanetaryDerivedState:: ~PlanetaryDerivedState ()	src/planetary_derived_state.cc	44	2
jeod::PlanetaryDerivedState:: set_use_alt_pfix (const bool use_alt_pfix_in)	src/planetary_derived_state.cc	55	1
jeod::PlanetaryDerivedState:: initialize (DynBody & subject_body, DynManager & dyn_manager)	src/planetary_derived_state.cc	63	2
jeod::PlanetaryDerivedState:: update ()	src/planetary_derived_state.cc	95	1
jeod::RelativeDerivedState::~ RelativeDerivedState ()	src/relative_derived_state.cc	48	5
jeod::RelativeDerivedState:: initialize (DynBody & subject_body, DynManager & dyn_manager)	src/relative_derived_state.cc	64	1
jeod::RelativeDerivedState:: initialize (DynManager & dyn_manager)	src/relative_derived_state.cc	81	6

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::RelativeDerivedState:: update ()	src/relative_derived_state.cc	172	3
jeod::RelativeDerivedState:: set_activation_flag (bool raf)	src/relative_derived_state.cc	196	2
jeod::SolarBetaDerivedState:: initialize (DynBody & subject_body, DynManager & dyn_manager)	src/solar_beta_derived_ state.cc	55	2
jeod::SolarBetaDerivedState:: update ()	src/solar_beta_derived_ state.cc	85	5
jeod::SolarBetaDerivedState:: ~SolarBetaDerivedState ()	src/solar_beta_derived_ state.cc	143	2

Part II

Euler Angles State Representation

Chapter 6

Introduction

6.1 Purpose and Objectives of the Euler Angles State Representation

The Euler Derived State represents the relative attitude of a particular reference frame with respect to some other frame as a series of Euler angle rotations. The output will be a 3-vector of angles; the user may specify in which order those angles are to be expressed.

Chapter 7

Product Requirements

Requirement DerivedState_3: Calculate relative attitude between two reference frames

Requirement:
Given a reference frame associated with a body, the Euler Angles representation will calculate an Euler-type angle sequence from body frame to the reference frame parent to it, and from the parent frame to the body frame using the 3 basic axes, represented as roll(x-axis rotation), pitch (y-axis rotation), and yaw (z-axis rotation).

Rationale:
To provide a means of expressing relative attitude between reference frames.

Verification:
Test

7.1 Requirements Traceability

Table 7.1: Requirements Traceability

Requirement	Inspection and Testing
DerivedState_3 - Calculate relative attitude between two reference frames	Test DerivedState_1

Chapter 8

Product Specification

8.1 Conceptual Design

A reference frame attitude can be described as a set of three rotations about independent axes. In the Euler Angles representation the Roll, Pitch, and Yaw axes are used, and can be input in any order **without repeats** (i.e. all three axes must be used). Numerically, three values are specified in a single vector, corresponding to the angle of rotation about each specified axis in turn.

The angles represent a process for conducting a transformation from one reference frame to another, thereby providing data on the relative attitude of the two frames. Because the relative attitude can be expressed in either direction, two sets of angles, *ref_body_angles* and *body_ref_angles* are included in the available output. *ref_body_angles* provides the transformation from the external reference frame to the subject-body reference frame, and *body_ref_angles* provides the reverse transformation.

When calculating the Euler angles from two reference frames, both sets of angles are calculated.

8.2 Mathematical Formulations

The Euler Angles representation relies on the Trick method *euler_matrix* for generating the euler angles. No further mathematical framework is implemented in the Euler Angles representation.

8.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

8.3.1 Process Architecture

The process architecture for the Euler Angles representation is trivial, comprising entirely independent methods.

8.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The Euler Angles representation contains only one class:

- **Class: EulerDerivedState** **Inheritance:** *DerivedState* (on page 8)

  This contains the methods *initialize*, and *update*:

  1. **initialize**

     There are two processes for initialization, depending on the relationship in the Reference Frame Tree between the subject-body reference frame, and the exterior reference frame. If the former is not a child of the latter, then it becomes necessary to specify which frame is to be used as the external reference frame. In this case, a separate initialization process allows for the specification of that frame. Otherwise, the initialization process simply calls the generic DerivedState initialization routine to establish the naming convention of the reference object (i.e., the planet about which the vehicle is orbiting) and state identifier.

  2. **update**

     If the Euler angles are to be computed relative to a frame other than the parent frame, that relative state is first calculated. Then the Trick method *euler_matrix* is called for both reference-to-subject and subject-to-reference frame transformations.

Chapter 9

User's Guide

The Analysis section of the User's Guide is intended primarily for users of pre-existing simulations. It contains:

- A description of how to modify Euler Angles representation variables after the simulation has compiled, including an in-depth discussion of the input file,
- An overview of how to interpret (but not edit) the S_define file,
- A sample of some of the typical variables that may be logged.

The Integration section of the User's Guide is intended for simulation developers. It describes the necessary configuration of the Euler Angles representation within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section of the user guide. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 8).

The Extension section of the User's Guide is intended primarily for developers needing to extend the capability of the Euler Angles representation. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 8).

9.1 Analysis

9.1.1 Identifying the Euler Angles State Representation

If the Euler angles have been included in the simulation, there will be an instance of *EulerDerived-State* located in the S_define file. This would typically be found in the vehicle object. There should be an accompanying call to an initialization routine, which takes a reference to the *subject_body* as one if its inputs, and may also take a reference to a reference-frame, although this is not necessary. There will also be an accompanying call to an update function. The essential variable *sequence*, the order in which the angles should be assigned to each axis, is defined elsewhere, often in the input file.

Example:

```
sim_object{
dynamics/derived_state:    EulerDerivedState example_of_euler_angles;

(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_euler_angles.initialize (
    Inout DynBody &      subject_body = vehicle_1.dyn_body,
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);

{environment} dynamics/derived_state:
example_of_rel_state_object.example_of_euler_angles.update ( )

} example_of_rel_state_object;
```

The input file may contain an entry comparable to:

```
example_of_rel_state_object.example_of_euler_angles.sequence = Roll_Pitch_Yaw;
```

This variable can take one of the following values:

- Roll_ Pitch_ Yaw
- Roll_ Yaw_ Pitch
- Pitch_ Yaw_ Roll
- Pitch_ Roll_ Yaw
- Yaw_ Roll_ Pitch
- Yaw_ Pitch_ Roll

9.1.2 Editing the Euler Angles State Representation

The sequence is open to edit by the analyst.

9.1.3 Output Data

The direct output from the Euler Angles representation comprises only the two vectors of angles, *ref_body_angles* (the Euler angles from the reference frame to the subject frame), and *body_ref_angles* (the Euler angles to the reference frame from the subject frame).

```
example_of_rel_state_object.example_of_euler_angles.ref_body_angles[0-2]
example_of_rel_state_object.example_of_euler_angles.body_ref_angles[0-2]
```

9.2 Integration

Simply including the Euler Angles representation is a trivial undertaking, all of the contents that can be included into the model are identified in the previous section (9.1).

9.3 Extension

The Euler Angles representation is not written with the intention of being extended.

Chapter 10

Verification and Validation

10.1 Verification

For testing the Euler Angles representation a single vehicle was initialized in one of 3 orbits: an equatorial circular orbit, an inclined orbit, and an eccentric equatorial orbit. An LVLH frame was also generated, and the Euler angles between the vehicle body frame (the subject frame), and the LVLH frame (the reference frame), were computed.

Test DerivedState_1: Verification of Euler Angles representation Output Data

Purpose:
To demonstrate that the output from the Euler Angles representation provides meaningful data.

Requirements:
Satisfactory conclusion of this test satisfies requirement DerivedState_3

Procedure:
Two cases were considered, one in which the vehicle frame was initially aligned with the LVLH frame, and one in which it was given an initial relative attitude. In both cases, the vehicle body frame was initialized with no rotation (with respect to the inertial frame). Since the LVLH frame maintains an orientation dependent upon the position of the origin, the two frames rotate with respect to one another. The angles generated by this orbital motion are considered.

Predictions: In the LVLH frame, the frame-to-frame relative rotation will appear as a pitch (rotation about the LVLH y-axis). In the tests in which the two frames are initially aligned, that is equivalent to a pitch in the body frame. Both transformations should exhibit slowly oscillating pitch values, with constant zero values for yaw and roll.

When the two frames are not initially aligned, in general, the variation of angles resulting from the varying relative attitude will be across all three axes. Exceptions to this rule include transformations from body to LVLH that end with a pitch, or those from LVLH to body that

start with a pitch. This is due to the order of calculation, consider the transformation from body to LVLH:

The transformation from body to initial LVLH frame is constant, and the LVLH rotates with respect to the initial LVLH frame on its y-axis; a transformation from body to LVLH can be seen as a combination of transformations from body to the initial LVLH frame followed by one from the initial LVLH frame to LVLH, which comprises only a pitch component. Therefore, if pitch is last in the order, the two pitch adjustments simply add; the roll and yaw are due to the constant difference between body and initial LVLH, and so should remain constant, whereas the pitch should start at some pre-assigned value, and exhibit the same periodic variation thereafter that was seen in the case of the initially aligned frames. The same argument can be traced in reverse for the reverse transformation.

For other sets in which the pitch is calculated in an inconvenient sequence (second or third, or first from body frame to reference frame, or last in reference frame to body frame), the effect in the yaw and roll angles of the addition of the pitch due to the variational relative frame rotation will manifest as a time dependency on all 3 axes, due to the cross-coupling effects of rotational sequences.

The variation with time in the rate at which the angles change will be constant for the circular orbits, but variable for the elliptic orbits.

For further confirmation of the data, the euler angles of the vehicle with respect to the inertially-oriented planetary axes we investigated, these should show no variation.

Results: All output data confirmed expectations, within numerical precision. Some expected constant values were not quite constant, but exhibited some very small oscillatory or noisy behavior.

For the aligned frames, the YRP, RYP, PYR, and PRY sequences showed the pitch slowly changing from $\pm\pi$ to $\mp\pi$ before immediately resetting. The YPR and RPY sequences had the pitch varying only between $\pm\frac{\pi}{2}$, and doing so with constant ramps up and down. For these two sequences, the roll and pitch also switched to $\pm\pi$ during the time that the pitch was decreasing, and returned to zero while the pitch was increasing. These solutions are completely valid, and illustrate that there is more than one way to represent a particular orientation. These results were valid for both the LVLH-body and body-LVLH orientations. See figures 10.1 and 10.2 for Pitch-Yaw-Roll and Roll-Pitch-Yaw sequences.

For the tests in which the frames were not initially aligned, those with Pitch in the correct sequencing showed the similar pattern to the aligned frames, as expected. Other sequences had periodic behavior. See Figures 10.3, 10.5, and 10.4 for Pitch-Yaw-Roll, Roll-Pitch-Yaw, and Yaw-Roll-Pitch sequences.

10.2 Validation

There is no independent validation of this sub-model.

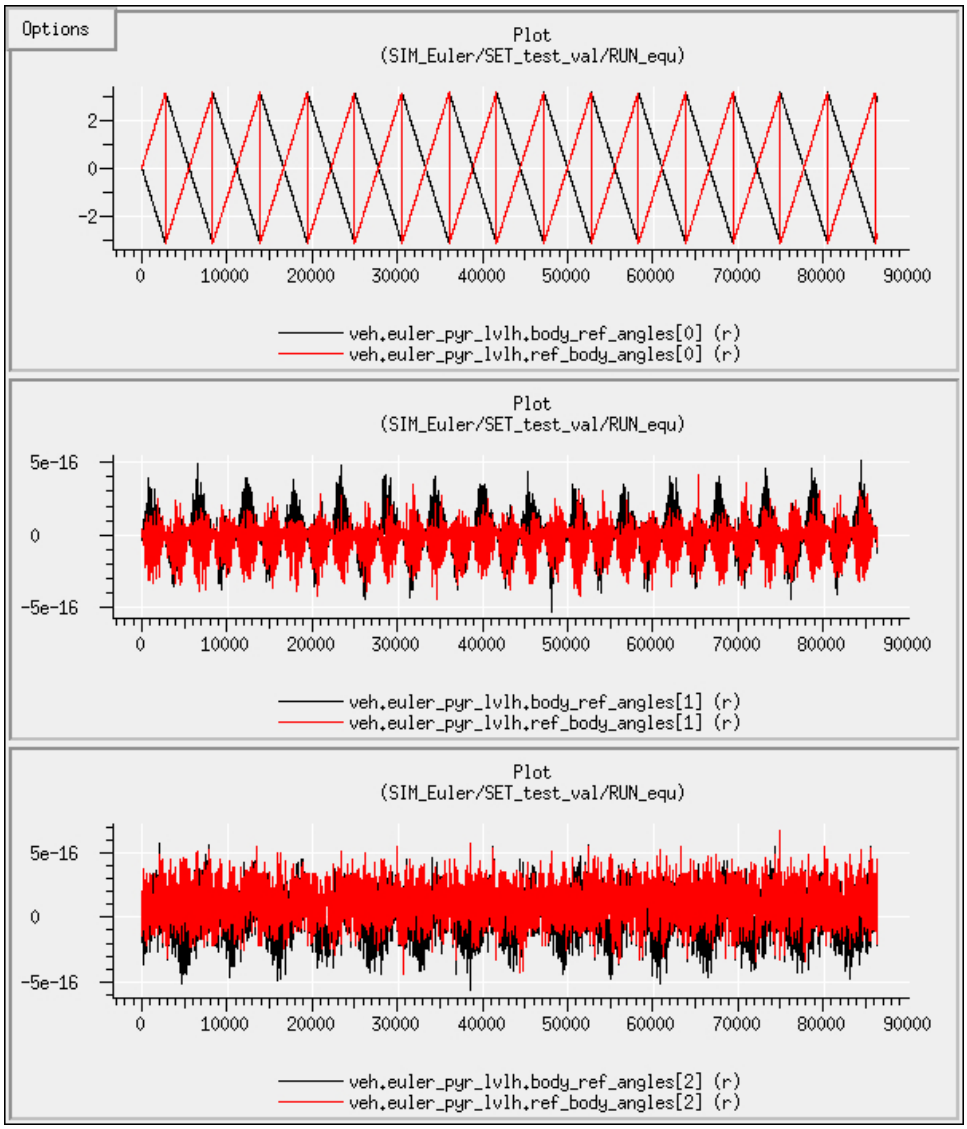


Figure 10.1: The Euler angles for the case in which the LVLH frame and body frame were initially aligned, and rotating with respect to each other on their respective pitch axes, expressed in a Pitch-Yaw-Roll sequence.

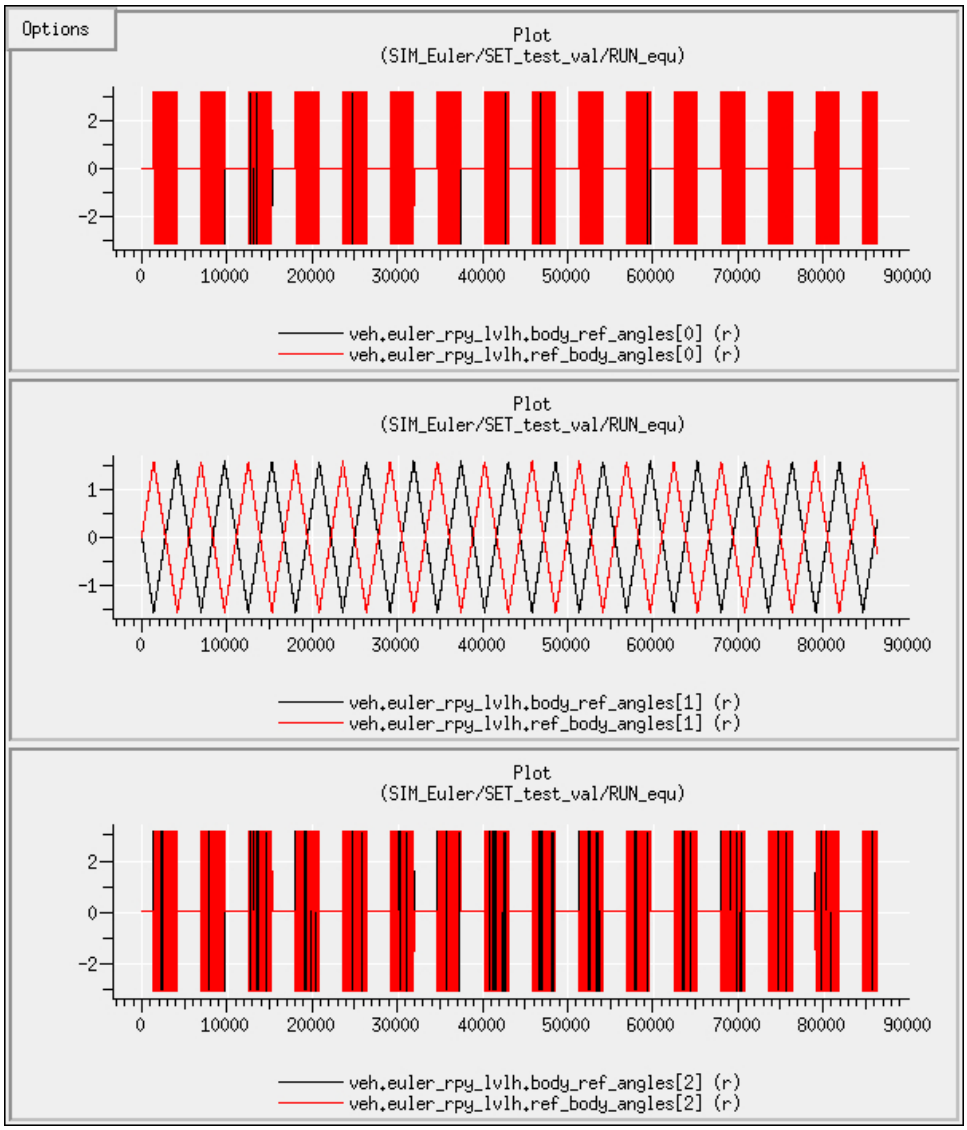


Figure 10.2: The Euler angles for the case in which the LVLH frame and body frame were initially aligned, and rotating with respect to each other on their respective pitch axes, expressed in a Roll-Pitch-Yaw sequence. Notice that the Pitch has a total range of only π radians; this is compensated for by the combined effect of adding in π rotations on the yaw and roll axes.

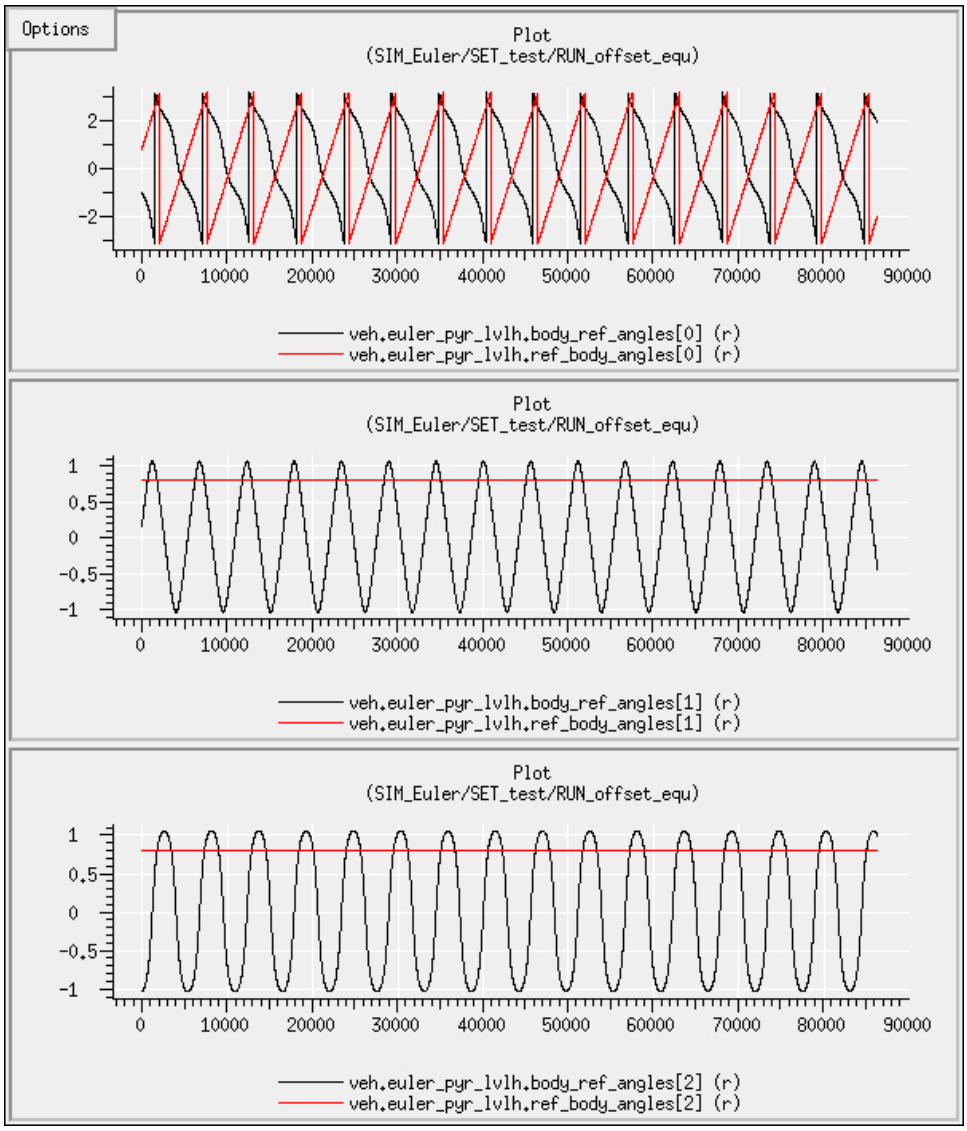


Figure 10.3: The Euler angles for the case in which the LVLH frame and body frame were initially un-aligned, and rotating with respect to each other on the pitch axis of the LVLH reference frame. Angles are expressed in a Pitch-Yaw-Roll sequence. Notice that when Pitch is first in the sequence, the angles associated with the transformation from the LVLH frame to the body frame (expressed in the LVLH frame) show a similar pattern to that in Figure 10.1, but that is not the case when representing the relative attitude from the body frame to the LVLH frame (this is expressed in the body frame).

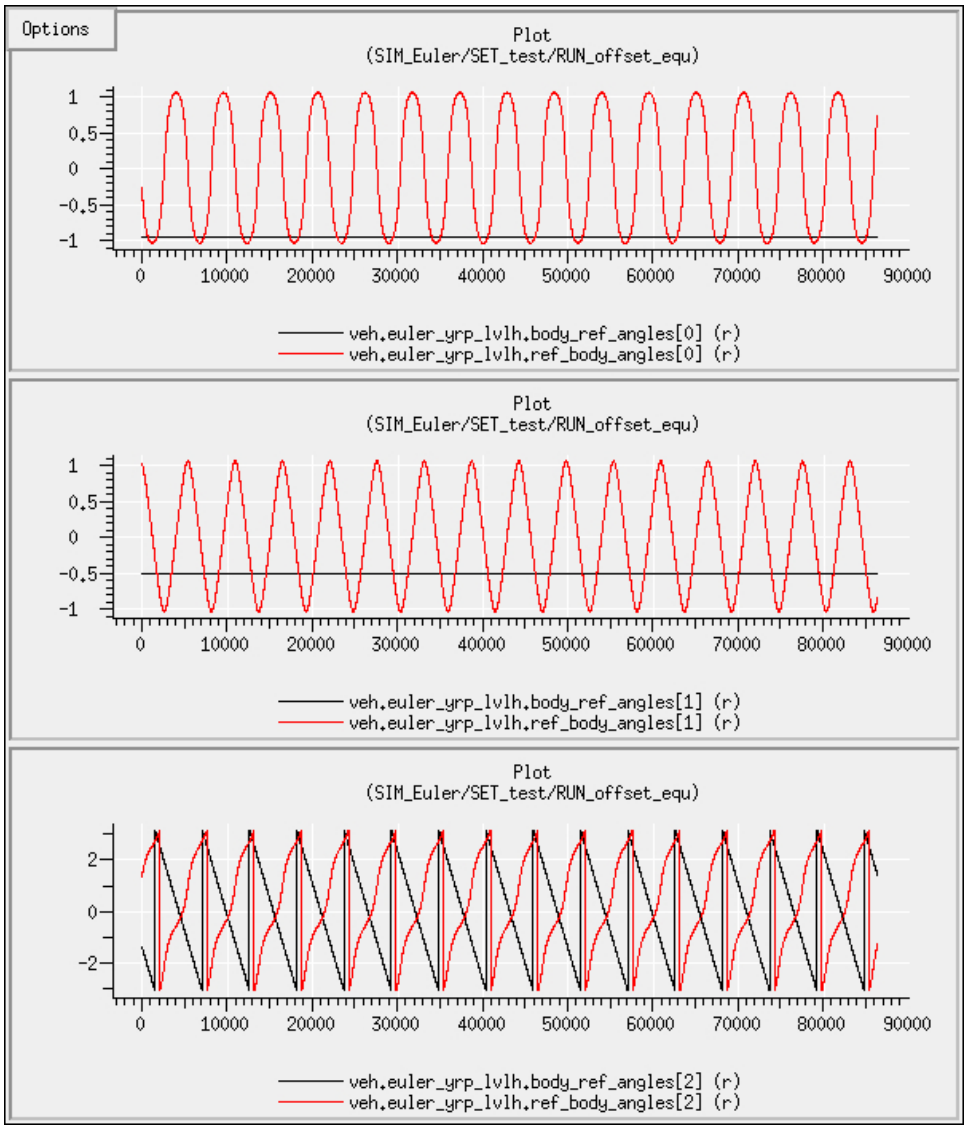


Figure 10.4: The Euler angles for the case in which the LVLH frame and body frame were initially un-aligned, and rotating with respect to each other on the pitch axis of the LVLH reference frame. Angles are expressed in a Yaw-Roll-Pitch sequence. Notice that when Pitch is last in the sequence, the angles associated with the transformation from body frame to LVLH (expressed in body frame) show a similar pattern to that in Figure 10.1, but that is not the case when representing the relative attitude from the LVLH frame to the body frame (this is expressed in the LVLH frame).

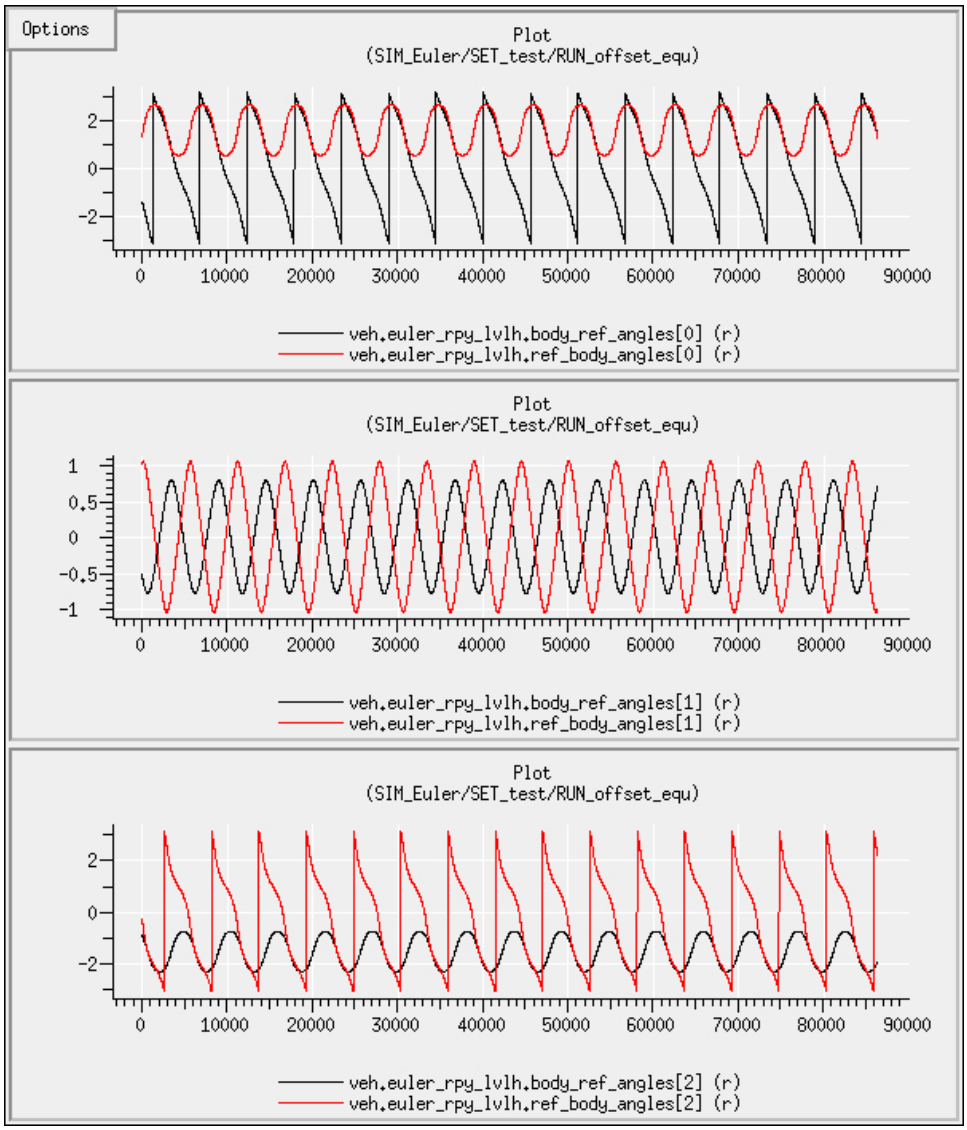


Figure 10.5: The Euler angles for the case in which the LVLH frame and body frame were initially un-aligned, and rotating with respect to each other on the pitch axis of the LVLH reference frame. Notice that the variation is non-sinusoidal, and that the amplitudes differ between the two representations of the transformation.

Part III

Local-Vertical-Local-Horizontal (LVLH) Reference Frame

Chapter 11

Introduction

11.1 Purpose and Objectives of the Local-Vertical-Local-Horizontal (LVLH) Reference Frame

The Local-Vertical-Local-Horizontal (LVLH) Reference Frame is used to express the state of a vehicle in terms of the local vertical (opposing the unit radial vector from the defined planetary reference frame origin - usually the planet about which it is orbiting), and two vectors in the local horizontal plane; the first is defined as the unit orbital angular momentum vector (so must be perpendicular to the local vertical), the second completes the right-hand coordinate system.

While several of the Derived State values simply describe the state of a vehicle in some other defined reference frame, the LVLH Derived State defines a new reference frame itself. Furthermore, since the reference frame is defined *by* the vehicle, it has no state in this frame. Instead, the LVLH frame, defined by the LVLH Derived State, can be used to express the state of another vehicle.

The LVLH reference frame, created by this model, is a recti-linear frame, so for two vehicles at the same altitude, the state of one in the LVLH frame of another will exhibit a positive vertical (i.e., down) component. See Figure 21.1 on page 71 for an illustration.

Chapter 12

Product Requirements

Requirement DerivedState_4: Local-Vertical-Local-Horizontal representation

Requirement:
The LVLH Derived State will provide the capability for expressing the state of one body in the LVLH reference frame based on another body.

Rationale:
Capability from JEOD 1.5.

Verification:
Test

12.1 Requirements Traceability

Table 12.1: Requirements Traceability

Requirement	Inspection and Testing
DerivedState_4 - Local-Vertical-Local-Horizontal representation	Test DerivedState_2

Chapter 13

Product Specification

13.1 Conceptual Design

The Local-Vertical-Local-Horizontal (LVLH) Reference Frame is used to express the state of some vehicle with respect to a reference frame that is aligned with the local vertical and horizontal axes at some other point (referred to below as the defining point), which is often another vehicle. A planetary object (referred to below as the specified planet) is used to define "vertical", and also to orient the axes in the horizontal plane.

The axes are defined as:

$$\hat{i} = \hat{j} \times \hat{k}$$
$$\hat{j} = -\hat{h}$$
$$\hat{k} = -\hat{r}$$

$\hat{h}$ represents the unit vector oriented with the angular momentum vector, associated with the translational motion of the defining point with respect to the specified planet.

$\hat{r}$ represents the unit vector oriented with the radial vector, from the specified planet to the defining point.

The LVLH reference frame fits into the reference frame tree as a child of the inertial reference frame associated with the specified planet.

The LVLH reference frame provided by this derived state model is a recti-linear LVLH representation.

13.2 Mathematical Formulations

The Local-Vertical-Local-Horizontal (LVLH) Reference Frame includes the method *compute_lvlh_frame*, which calculates the transformation matrix from the planet-centered inertial reference frame to the LVLH reference frame. This method is executed as a routine part of the LVLH update process.

The three vectors that define LVLH are as follows

- Opposite to the radial vector from planet center ($\hat{k} = -\hat{r}$)
- Opposite to the orbital angular momentum vector ($\hat{j} = -\hat{r} \times \hat{v}$).
- Completing the right handed coordinate system ($\hat{i} = \hat{j} \times \hat{k}$)

The $\hat{i}$ axis also represents the projection of the velocity vector, or trajectory, onto the horizontal plane. For circular orbits, the $\hat{i}$ axis lies along the trajectory of the vehicle, but that is not a definition.

Suppose the position and velocity are known in the planet-centered inertial reference frame.

$\vec{r}_{inrtl} = [r_{x,inrtl}, r_{y,inrtl}, r_{z,inrtl}]$

$\vec{v}_{inrtl} = [v_{x,inrtl}, v_{y,inrtl}, v_{z,inrtl}]$

and we want to define the transformation matrix from inertial to LVLH, such that

$$\vec{x}_{LVLH} = T_{inrtl->LVLH}\ \vec{x}_{inrtl}$$

$$T_{inrtl->lvlh} = \begin{bmatrix} p_1 & p_2 & p_3 \\ -h_1 & -h_2 & -h_3 \\ -r_1 & -r_2 & -r_3 \end{bmatrix}$$

where r_i are the components of the unit radial vector, h_i are the components of the unit angular momentum vector, and p_i are the normalized values resulting from the cross product of $\hat{r}$ with $\hat{h}$ (the normalization should be unnecessary, since the two vectors in the cross product are perpendicular unit vectors, but is included to ensure that numerical innaccuracies do not propagate).

Because the LVLH frame is rotating with respect to its parent frame (one revolution per orbit), the relative velocity with respect to the LVLH frame requires the relative angular velocity of the LVLH frame with respect to the inertial frame. Expressed in the LVLH frame, that relative angular velocity is oriented along the $-\hat{j}$ axis only, by definition.

Then,

$$\begin{aligned} \vec{v}_{veh1_wrt_veh2:LVLH} = \ & T_{inrtl->LVLH}\,(\vec{v}_{veh1_wrt_inrtl:inrtl} - \vec{v}_{veh2_wrt_inrtl:inrtl}) \\ & - \vec{\omega}_{LVLH_wrt_inrtl:LVLH} \times \vec{x}_{veh1_wrt_veh2:LVLH} \end{aligned} \tag{13.1}$$

Using

$$\vec{\omega}_{LVLH_wrt_inrtl:LVLH} = -|\omega|\hat{j}$$

as the angular momentum vector yields the correct interpretation from the Reference Frames calculations.

13.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

13.3.1 Process Architecture

The process architecture for the Local-Vertical-Local-Horizontal (LVLH) Reference Frame is trivial; the Local-Vertical-Local-Horizontal (LVLH) Reference Frame comprises only methods that are independent of one another.

13.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The Local-Vertical-Local-Horizontal (LVLH) Reference Frame contains only one class:

- **Class: LvlhDerivedState** **Inheritance:** *DerivedState* (on page 8)

  This contains the methods *compute_lvlh_frame*, *initialize*, and *update*:

  1. **compute_lvlh_frame**

     This method follows the process outlined in the *Mathematical Formulations section* (on page 45) for deriving the transformation matrix, and uses the *compute_quaternion* method from the Reference Frames model to compute the equivalent quaternion representation (see *Reference Frames* documentation [10] for details).

  2. **initialize**

     The initialization process comprises the following steps:

     (a) The generic DerivedState initialization routine is called to establish the naming convention of the reference object (i.e., the planet about which the vehicle is orbiting) and state identifier.

     (b) The DerivedState method *find_planet* is called (which subsequently calls the *Dynamics Manager* [2]) method of the same name to find a planet by name; that name is stored as *reference_name*, and is usually assigned in an input file.

     (c) The name of the LVLH state is set as *vehicle_name.planet_name.lvlh*.

     (d) The element *planet_centered_inertial* is set to be the inertially-oriented reference frame attached to the planet, identified by *reference_name*.

     (e) The LVLH reference frame is added as a child to the *planet_centered_inertial* frame.

     (f) The LVLH frame is registered with the Dynamics manager (optional, user can specify with the *register_frame* flag, default is true).

     (g) The angular momentum vector of the LVLH frame with respect to the inertial reference frame is then initialized to zero. This vector is to be expressed in the LVLH frame, so it will have only a y-component (the frame rotates about the orbital angular momentum vector, the y-axis of the LVLH frame). The unit vector is set accordingly.

3. **update**

   The LVLH frame is a child of the inertially-oriented frame attached to some planet. If that inertial frame is the same as the integration frame, then the update comprises simply a call to *compute_lvlh_frame*. Otherwise, it is first necessary to compute the relative state of the vehicle with respect to that inertial frame; this is achieved through the DynBody method *compute_relative_state* (see *Dynamic Body* documentation [10] for details)

Chapter 14

User's Guide

The Analysis section of the User's Guide is intended primarily for users of pre-existing simulations. It contains:

- A description of how to modify Local-Vertical-Local-Horizontal (LVLH) Reference Frame variables after the simulation has compiled, including an in-depth discussion of the input file,
- An overview of how to interpret (but not edit) the S_define file,
- A sample of some of the typical variables that may be logged.

The Integration section of the User's Guide is intended for simulation developers. It describes the necessary configuration of the Local-Vertical-Local-Horizontal (LVLH) Reference Frame within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section of the user guide. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 13).

The Extension section of the User's Guide is intended primarily for developers needing to extend the capability of the Local-Vertical-Local-Horizontal (LVLH) Reference Frame. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 13).

14.1 Analysis

It must be reiterated that the Local-Vertical-Local-Horizontal (LVLH) Reference Frame does not provide a state for the vehicle with which it is associated, it provides a *reference frame*. The LvlhDerivedState does contain a member called *lvlh_frame*, which contains a member called *state*, but this is the state of the reference frame with respect to the parent planet-centered inertial reference frame. This is identical to the inertial state of the vehicle with respect to the same planet; it is **not** the LVLH state of the vehicle with respect to the same planet.

14.1.1 Identifying the Local-Vertical-Local-Horizontal (LVLH) Reference Frame

If the LVLH reference frame has been included in the simulation, there will be an instance of *LvlhDerivedState* located in the S_define file. This would typically be found in either the vehicle object, or a separate relative-state object. There should be an accompanying call to an initialization routine, which takes a reference to the *subject_body* as one of its inputs, and an accompanying call to an update function. The essential variable *reference_name* is defined elsewhere, often in the input file.

The LVLH reference frame, generated by the Local-Vertical-Local-Horizontal (LVLH) Reference Frame model, may also be used to represent the state of another vehicle. In this case, there will be further inclusion of a RelativeDerivedState instance; see the *RelativeDerivedState User's Guide* (on page 130) for details on the implementation of RelativeDerivedState.

Example:

```
sim_object{
dynamics/derived_state:    LvlhDerivedState example_of_lvlh_state;

(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_lvlh_state.initialize (
    Inout DynBody &      subject_body = vehicle_1.dyn_body,
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);

{environment} dynamics/derived_state:
example_of_rel_state_object.example_of_lvlh_state.update ( )

} example_of_rel_state_object;
```

Then the input file may have an entry comparable to:

```
example_of_rel_state_object.example_of_lvlh_state.reference_name = "Earth";
```

14.1.2 Editing the Local-Vertical-Local-Horizontal (LVLH) Reference Frame

The planetary identification (*reference_name*) is open to edit by the analyst.

14.1.3 Output Data

The direct output from the Local-Vertical-Local-Horizontal (LVLH) Reference Frame comprises only the inertial state of the origin of the LVLH reference frame. This is typically not useful. The strength of the Local-Vertical-Local-Horizontal (LVLH) Reference Frame comes in its applicability to a RelativeDerivedState; see the *RelativeDerivedState User's Guide* (on page 130) for details on the implementation of RelativeDerivedState.

14.2 Integration

Simply including the Local-Vertical-Local-Horizontal (LVLH) Reference Frame is straightforward; using it to its full capacity is more challenging.

14.2.1 Generating the S_define

The LVLH model should be used in conjunction with a RelativeDerivedState. The ordering of their evaluation is very important; the LvlhDerivedState must be evaluated before the RelativeDerivedState in order to get the current relative state expressed in the current LVLH orientation.

An important consequence of using a RelativeDerivedState to express the relative states of two objects in an LVLH reference frame requires the consideration of the class structure of both LvlhDerivedState and RelativeDerivedState. There are two types of reference frames in use, BodyRefFrame, and RefFrame. The BodyRefFrame is associated with a DynBody (a body with dynamic properties), while the RefFrame need not be, it can be associated with some fixed (non-dynamic) body or point.

The LVLH reference frame in an LvlhDerivedState is an instance of a RefFrame, there is no requirement that this be attached to a DynBody. However, the RelativeDerivedState contains both a BodyRefFrame (*subject_frame*) and a RefFrame (*target_frame*); the intention is that the state of a body can be expressed in some other reference frame. Therefore, when generating the relative states, it is *not* possible to use the generated LVLH reference frame as the subject frame, it must instead be the target frame. Then, the vehicle whose state is being expressed must be associated with the subject frame, and the *subject_frame_name* and *target_frame_name* must be set appropriately. Then, the *direction_sense* of the Relative Derived State must be *ComputeSubjectStateinTarget*.

An illustration of this is found in the verification simulations for the LVLH model (verif/SIM_LVLH). Here, the relative state instance that expresses the state of vehicle1 (*veh*) with respect to vehicle2 (*veh2*), *veh_wrt_veh2*, has:

```
subject_frame_name = "vehicle1.composite_body";
target_frame_name = "vehicle2.Earth.lvlh";
direction_sense = RelativeDerivedState::ComputeSubjectStateinTarget;
```

Notice also in these simulations that the LvlhDerivedState is contained within the respective vehicle object, while the RelativeDerivedState is contained within the *rel_state* object. This assignment is intentional, though not necessary. What is necessary is that the state of the two vehicles be evaluated before the relative state, and that the LVLH reference frame be appropriately oriented, based on the state of its parent vehicle, before the relative state can be expressed in that frame. Therefore, it was chosen to move the relative state calculations to the end of the S_define, where they will be processed only after the vehicle and LVLH reference frames have been processed. This is recommended procedure.

14.2.2 Generating the Input File

The input file (or Modified Data file) must contain the name of the planet, identified as *reference_name*:

```
example_of_rel_state_object.example_of_lvlh_state.reference_name = "Earth";
```

14.2.3 Logging the Data

See the *Analysis* (on page 130) for a list of available output data from a RelativeDerivedState.

14.3 Extension

The Local-Vertical-Local-Horizontal (LVLH) Reference Frame is not intended for extension beyond its current capability. A potential area for improvement may be the development of a curvi-linear LVLH reference frame, but this would be a separate entity, not an extension of the recti-linear LVLH reference frame.

Chapter 15

Verification and Validation

15.1 Verification

For testing the Local-Vertical-Local-Horizontal (LVLH) Reference Frame a simulation of two vehicles was established, with the two vehicles in identical orbits, separated by 1 degree. Both vehicles generate a LVLH reference frame, and the relative state of the other vehicle is calculated in both. Three such cases are run, with the vehicles in an equatorial, circular orbit; an inclined, circular orbit; and an equatorial, eccentric orbit.

Test DerivedState_2: Verification of Local-Vertical-Local-Horizontal (LVLH) Reference Frame Output Data for Generic Orbits

Purpose:
To demonstrate that the output from the Local-Vertical-Local-Horizontal (LVLH) Reference Frame provides meaningful data.

Requirements:
Satisfactory conclusion of this test satisfies requirement DerivedState_4

Procedure:
The data from the LVLH output was compared against theory for the following features:

1. Orientation of the velocity vector.
   The orientation of the velocity vector, expressed in the LVLH frame, was considered.
2. Calculation of Transformation matrix elements.
   The inertial position outputs from the simulations were differenced, transformed into the respective LVLH frame (position of vehicle 1 with respect to vehicle 2 was transformed into the LVLH frame associated with vehicle 2, and vice versa), and compared against the output frm the relative state, expressed in LVLH.

   Similarly, the relative velocity vectors were transformed and the additional term for

frame rotation introduced. Using equation 13.2 on page 46, the term

$$-\vec{\omega}_{LVLH2_wrt_inrtl:LVLH2} \times \vec{x}_{veh1_wrt_veh2:LVLH2} = \begin{bmatrix} |\omega|x_3 \\ 0 \\ -|\omega|x_1 \end{bmatrix}$$

where $LVLH2$ is the LVLH frame associated with vehicle 2.

This term was added to the straightforward transformation of the velocity difference, and then compared to the output from the LVLH frame.

3. Symmetry between vehicles.
   The output from each LVLH frame was compared with the output from the other.

Predictions:

1. Orientation of velocity vector
   The velocity vector should be oriented in the x-z plane at all times, and along the x-axis only for the two circular orbits. The elliptical orbit should show some variation in the z-component of its velocity vector.
2. Calculation of Transformation matrix elements
   The transformed inertial output should be identical to the LVLH frame output.
3. Symmetry between vehicles
   The y-axis output for both velocity and position should be zero for all time, for all representations. For the circular orbits, the x-components of the relative positions should be constant and symmetric, with the value in one direction being negative the value in the other. For the elliptical orbit, the x-component of position should oscillate, but never cross zero. While the vehicles are close, the two outputs should approximate mirror images, but this should not hold true on close inspection due to the difference in curvature of the two orbits at the locations of the two vehicles. The z-component of position should be a constant positive value for the circular orbits in both frames. The z-component of position on the elliptical orbit will oscillate, with more elliptic orbits exhibiting both positive and negative values in the oscillation. As with the y-component, the oscillation should exhibit approximate reflection symmetry between the two frames, but only approximate. The circular orbits should return zero velocity values across all components. The elliptical orbit should return features consistent with the position data, i.e. approximate reflection symmetry, with non-sinusoidal oscillations in the x- and z- components.

Results:

All output data confirmed expectations, within numerical precision. Constants were not quite constant, but exhibited some very small oscillatory behavior.

15.2 Validation

There is no independent validation of the LVLH model.

Part IV

Local-Vertical-Local-Horizontal (LVLH) Relative Derived State

Chapter 16

Introduction

16.1 Purpose and Objectives of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State

The Local-Vertical-Local-Horizontal (LVLH) Relative Derived State provides a means to express the state of a vehicle with respect to the Local-Vertical-Local-Horizontal (LVLH) of any translational state (position and velocity) and planet for which LVLH can be defined as in 13. In addition to the customary rectilinear LVLH system, the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State offers a curvilinear option in which the rectilinear X-axis is replaced by a planet-centered circle through the system origin and within its plane of motion.

Chapter 17

Product Requirements

Requirement DerivedState_5: Rectilinear LVLH representation

Requirement:
The LVLH Relative Derived State shall provide the capability to express the state of one body in the rectilinear LVLH reference frame based on a reference point and reference planet.

Rationale:
Capability from JEOD 1.5.

Verification:
Test

Requirement DerivedState_6: Curvilinear LVLH representation

Requirement:
The LVLH Relative Derived State shall provide the capability to express the state of one body in the curvilinear LVLH reference frame based on a reference point and reference planet.

Rationale:
Capability from JEOD 1.5.

Verification:
Test

17.1 Requirements Traceability

Table 17.1: Requirements Traceability

Requirement	Inspection and Testing
DerivedState_5 - Rectilinear LVLH representation	Test DerivedState_3
DerivedState_6 - Curvilinear LVLH representation	Test DerivedState_4

Chapter 18

Product Specification

18.1 Conceptual Design

The Local-Vertical-Local-Horizontal (LVLH) Relative Derived State is used to express the state of a subject frame relative to a system that is aligned with the local vertical and horizontal of a given point with respect to a planetary object. The relative state includes the full 6-DoF state of a given reference frame as viewed from the LVLH frame. Unlike a true reference frame, this relative state is not part of the reference frame tree structure. Although it has no parent, the relative state can be considered to be a child of the LVLH frame, especially in the rectilinear case. Introduction of curvilinear coordinates changes the way distances, velocities and directions are computed, but the relative state still represents the way the subject looks from the point of view of the LVLH frame.

18.2 Mathematical Formulations

the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State depends on the *LVLH Frame model* to supply a rectangular LVLH reference frame. In this discussion, we will consider two reference frames, Q (subject) and R (target) with origin respectively located at $\vec{Q}$ and $\vec{R}$ where both Q and R are children of planet-centered inertial. Suppose that R is the LVLH frame associated with $\vec{R}$ with respect to a reference planet. Note that in order to construct a canonical LVLH frame at $\vec{R}$, the angular momentum $\vec{H} = \vec{R} \times \dot{\vec{R}}$ must be non-zero. Let $\hat{\vec{H}}$ be the unit vector in the direction of $\vec{H}$, thus $\hat{\vec{H}} = \frac{\vec{H}}{|\vec{H}|}$. From the documentation of the LVLH Frame Model, found in models/utils/lvlh_frame/docs/lvlh_frame.pdf, the axis unit vectors of R $\hat{\vec{X}}$, $\hat{\vec{Y}}$ and $\hat{\vec{Z}}$ are given by

$$\begin{aligned} \hat{\vec{Z}} &= -\frac{\vec{R}}{|\vec{R}|} \\ \hat{\vec{Y}} &= -\hat{\vec{H}} \\ \hat{\vec{X}} &= \hat{\vec{Y}} \times \hat{\vec{Z}} \end{aligned} \tag{18.1}$$

In the rectangular case, we obtain the relative state by simply invoking the *compute_relative_state* method of the subject frame with respect to the rectilinear LVLH frame (R) which is generated by

the *LvlhFrame* class and is available from the dynamics manager in the reference frame tree. Specifically, the function call Q.*compute_relative_state*(R,*recti_state*) provides the state of Q expressed in R.

In general, tracking the action of the *compute_relative_state* method of the *RefFrame* class can be challenging, but since Q and R have a common parent I, the planetary inertial frame, all the work is done by a single call to the *decr_left* method of the *RefFrameStateClass*. For rectilinear coordinates, it is not necessary to understand the inner workings of the transformations because they are handled automatically by *compute_relative_state*. However, as we shall see in 18.2.1, we will replace the frame R by the *RefFrameState* R_q that differs from R by a Y-rotation through the *phase angle* θ. For technical reasons, R_q will not be a reference frame in the JEOD sense, so we will need to manually implement the calculations that would otherwise be done by *RefFrameState::decr_left*. Here are the equations as applied to generic *RefFrameStates* A, B and C.

$$\begin{aligned} T_{B:C} &= T_{A:C}T_{A:B}^T \\ \omega_{B:C} &= \omega_{A:C} - T_{B:C}\omega_{A:B} \\ x_{B:C} &= T_{A:B}(x_{A:C} - x_{A:B}) \\ v_{B:C} &= T_{A:B}(v_{A:C} - v_{A:B}) - \omega_{A:B} \times x_{B:C} \end{aligned} \tag{18.2}$$

18.2.1 Curvilinear Coordinates

The curvilinear system replaces the LVLH X-axis by the circle centered at the planet center and tangent to the LVLH X-axis at the origin $\vec{R}$ of R, and the x' coordinate of the curvilinear system represents distance measured on this circle. The curvilinear Z coordinate is measured radially from the circle with the positive Z direction being toward the planet. The Y coordinate axis of the curvilinear system is the same as that of R.

The Local-Vertical-Local-Horizontal (LVLH) Relative Derived State obtains a curvilinear representation of Q with respect to R by first computing the rectilinear LVLH state and then invoking *convert_rect_to_circ* on the rectilinear LVLH state (*recti_state*).

18.2.2 Translational State

The translational part of the curvilinear state is defined by common practice, and as such, is not an exact analog of the translational component of a rectilinear state. Where analogs exist, they will be identified and explained.

A key parameter associated with the curvilinear representation is the *phase angle* θ which describes the angular separation of $\vec{Q}$ and $\vec{R}$ as measured in the orbital plane of $\vec{R}$. A second key quantity is the vector $\vec{Q}'$ which is the projection of $\vec{Q}$ on the orbital plane.

The incoming parameter *recti_state* contains some important fields which are extracted as follows:

$$\begin{aligned} x &= \text{recti_state.trans.position[0]} \\ y &= \text{recti_state.trans.position[1]} \\ z &= \text{recti_state.trans.position[2]} \\ \dot{x} &= \text{recti_state.trans.velocity[0]} \\ \dot{y} &= \text{recti_state.trans.velocity[1]} \\ \dot{z} &= \text{recti_state.trans.velocity[2]} \end{aligned} \tag{18.3}$$

18.2.2.a Projecting $\vec{Q}$ onto the X-Z Plane of R

We first obtain the projection $\vec{Q}'$ of $\vec{Q}$ onto the X-Z plane of R.

$$\vec{Q}' = \vec{R} + x\hat{\vec{X}} + z\hat{\vec{Z}} \tag{18.4}$$

where $\hat{\vec{X}}$ and $\hat{\vec{Z}}$ are as in 18.2.

18.2.2.b The Phase Angle θ

The points 0, $\vec{Q}'$ and $\vec{R} - z\hat{\vec{Z}}$ form a right triangle in the X-Z plane of R, and the angle θ at the origin is said to be the *phase angle* of $\vec{Q}$ with respect to $\vec{R}$. If $r = |\vec{R}|$, we then have the following equations for x', y', z', q' and θ, the position coordinates in the curvilinear system, $|\vec{Q}'|$ and the phase angle.

$$\begin{aligned} q' &= \sqrt{x^2 + (r-z)^2} \\ \theta &= \tan^{-1}\frac{x}{r-z} \\ x' &= r\theta \\ y' &= y \\ z' &= r - q' \end{aligned} \tag{18.5}$$

The translational velocity in the curvilinear system is the same as the rectilinear LVLH velocity except that it is expressed in a frame rotated by θ. We effectively express the curvilinear velocity with respect to a frame co-moving with R but rotated to the LVLH of the point $\vec{Q}'$. Of course, this is physical fiction because $\vec{Q}'$ does not co-move with $\vec{R}$, and this introduces some small physical inconsistencies which are described in 18.2.3. Let us refer to the co-moving frame as R_q. We then introduce the Y-rotation M_θ which, in the parlance of 18.3 can be identified as $T_{R:R_q}$.

$$M_\theta = \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix} \tag{18.6}$$

Then the velocity field of *rel_state* is computed as follows:

$$\text{rel_state.trans.velocity} = M_\theta \cdot \text{ recti_state.trans.velocity} \tag{18.7}$$

18.2.3 Rotational State

For attitude and attitude rate (angular velocity) to be meaningful, they must be relative to some coordinate system. In the rectangular case, the rectangular LVLH frame R served as the reference. In the curvilinear case, R_q will serve as the reference. Note that we have specified a velocity (same as R) and an orientation (R rotated by θ), but not an origin. If we place the origin of R_q at $\vec{Q}'$, then we are inconsistent because $\vec{Q}'$ and $\vec{R}$ have non-zero relative motion. If instead, we place the origin of R_q at $\vec{R}$, then any rotation of R_q relative to R will introduce an $\vec{\omega} \times \vec{X}$ term into 18.7. Practically speaking, this issue can be safely ignored. We certainly want R_q to co-move with R so that the curvilinear velocity will "see" the translational motion of Q with respect to R. While we do account for a rotational motion of R_q with respect to R, in virtually all operational scenarios such as rendezvous maneuvers, the differential rotation rate is on the order of 10^{-6} radians per second. The default setting is to ignore it.

now it is time to use the equations of 18.3 to understand how using R_q rather than R as a reference will change attitude and angular velocity calculations. Note that the incoming parameter *recti_state* already contains the R-relative quantities, so we only need account for the differences introduced by using a different reference.

The easiest place to start is with the attitude calculation, the first identity of 18.3. If we let $A = I$, $B = R$, and $C = Q$, then

$$\begin{aligned} \text{recti_state.rot.T_parent_this} = T_{R:Q} = T_{I:Q}T^t_{I:R}, where \\ T_{I:Q}T^t_{I:R} = Q.\text{rot.T_parent_this} \cdot R.\text{rot.T_parent_this}^t \end{aligned} \tag{18.8}$$

Further, $T_{R:R_q} = M_\theta$, so $T_{I:R_q} = M_\theta T_{I:R}$. Now applying 18.3 with $A = I$, $B = R_q$ and $C = Q$, then

$$T_{R_q:Q} = T_{I:Q}T^t_{I:R_q} = (T_{I:Q}T^t_{I:R})M^t_\theta = \text{recti_state.rot.T_parent_this}M^t_\theta \tag{18.9}$$

The next part to tackle is the angular velocity as described by the second identity of 18.3. Again, let $A = I$, $B = R$ and $C = Q$. This gives

$$\omega_{R:Q} = \omega_{I:Q} - T_{R:Q}\omega_{I:R} \tag{18.10}$$

If instead, we let the middle frame $B = R_q$ we have

$$\omega_{R_q:Q} = \omega_{I:Q} - T_{R_q:Q}\omega_{I:R_q} \tag{18.11}$$

If we subtract 18.10 from 18.11 we derive a formula for the difference between the rectilinear angular velocity *recti_state.rot.ang_vel_this* and the curvilinear value *rel_state.rot.ang_vel_this*. Specifically,

$$\begin{aligned} \text{rel_state.rot.ang_vel_this} = \text{recti_state.rot.ang_vel_this} + T_{R:Q}\omega_{I:R} - T_{R_q:Q}\omega_{I:R_q} \\ = T_{R:Q}\omega_{I:R} - T_{I:Q}M^t_\theta\omega_{I:R_q} \\ = T_{R:Q}(\omega_{I:R} - M^t_\theta\omega_{I:R_q}) \\ = T_{R:Q}(\omega_{I:R} - \omega_{I:R_q}) \end{aligned} \tag{18.12}$$

where we take advantage of the fact the M_θ^t is a Y-rotation, and thus leaves $\omega_{I:R_q}$ unchanged because $\omega_{I:R_q}$ and, for that matter, $\omega_{I:R}$ are of the form $\begin{pmatrix} 0 \\ \omega \\ 0 \end{pmatrix}$. Moreover, $\omega_{I:R} - \omega_{I:R_q} = \begin{pmatrix} 0 \\ \dot{\theta} \\ 0 \end{pmatrix}$. Therefore

$$\text{rel_state.rot.ang_vel_this} = \text{recti_state.rot.ang_vel_this} + \text{recti_state.T_parent_this} \begin{pmatrix} 0 \\ \dot{\theta} \\ 0 \end{pmatrix} \tag{18.13}$$

It is straightforward to differentiate the expression for θ in 18.6 with respect to time if the derivatives of r, x and z are available. The derivatives of x and z are part of the translational portion of *recti_state*, and $\dot{r} = \frac{\vec{R}\cdot\dot{\vec{R}}}{r}$, therefore

$$\dot{\theta} = \frac{\dot{x}(r-z) - x(\frac{\vec{R}\cdot\dot{\vec{R}}}{r} - \dot{z})}{x^2 + (r-z)^2} \tag{18.14}$$

which, with 18.13, completes the calculation of *rel_state.rot.ang_vel_this*.

As already noted, accounting for $\dot{\theta}$ is not necessary for many applications and is not strictly consistent with physics. For these reasons, the $\dot{\theta}$ correction is turned off by default in JEOD, and in that case, the rectilinear and curvilinear angular velocities are identical. The calculations are only included for completeness.

18.2.4 *convert_circ_to_rect*

Now that the equations of *convert_rect_to_circ* have been explained, their inversions will be presented with minimal comment. The *convert_circ_to_rect* method takes a *RefFrameState* argument, say *curvi_state*, and fills in the *rel_state* field of *RelativeDerivedState* with the rectilinear equivalents. It is assumed that *curvi_state* shares the same conventions for curvilinear coordinates described in 18.2.1. We also have access to the target frame R, thus r and $\vec{R}$ are known. Since we know the curvilinear position $\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}$ = curvi_state.trans.position, we can invert the equations of 18.2.1 as follows:

$$\begin{aligned} \theta &= \tfrac{x'}{r} \\ q' &= r - z' \\ x &= q' \sin\theta \\ y &= y' \\ z &= r - q' \cos\theta \end{aligned} \tag{18.15}$$

We can invert 18.7 to obtain the rectilinear LVLH velocity.

$$\text{rel_state.trans.velocity} = M_\theta^t \ \text{curvi_state.trans.velocity} \tag{18.16}$$

Similarly, we can invert 18.9 to obtain the orientation of the rectilinear LVLH frame.

$$\text{rel_state.rot.T_parent_this} = \text{curvi_state.t_parent_this} M_\theta \tag{18.17}$$

Now that the rectilinear translational state and orientation of Q with respect to R are known, it is trivial to invert 18.13 for the rectilinear angular velocity.

$$\text{rel_state.rot.ang_vel_this} = \text{curvi_state.rot.ang_vel_this} - \text{rel_state.rot.T_parent_this} \begin{pmatrix} 0 \\ \dot{\theta} \\ 0 \end{pmatrix} \tag{18.18}$$

18.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

18.3.1 Process Architecture

The process architecture for the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State is trivial; the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State comprises the usual *initialize* and *update* methods and two public methods for conversion between rectilinear and circular curvilinear coordinates.

18.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The Local-Vertical-Local-Horizontal (LVLH) Relative Derived State contains only one class:

- **Class: LvlhRelativeDerivedState** **Inheritance:** *RelativeDerivedState* (on page 128)

  This contains the public methods *convert_rect_to_circ*, *convert_circ_to_rect*, *initialize*, and *update*:

  1. **convert_rect_to_circ**
     This method follows the process outlined in the *Mathematical Formulations section* (on page 58) for converting a state from rectilinear coordinates to the circular curvilinear system also described in *math section* (on page 58).
  2. **convert_circ_to_rect**
     This method follows the process outlined in the *Mathematical Formulations section* (on page 58) for converting a state from the circular curvilinear coordinates to standard rectilinear LVLH.
  3. **initialize**
     This method is a simple passthrough to the *initialize* method of *RelativeDerivedState*.

4. **update**

   The *update* method first does a standard conversion to rectilinear LVLH. If *lvlh_type* is set to *CircularCurvilinear*, then the method *convert_rect_to_circ* is called to perform the conversion.

Chapter 19

User's Guide

The layout of this chapter will be slightly different from what is seen in the User Guide chapters of most JEOD models. The Analyst and Integrator sections will be combined, and since we do not expect this model to be extended, the Extender section is not included.

The Local-Vertical-Local-Horizontal (LVLH) Relative Derived State provides the capability to maintain a relative derived state of a subject vehicle with respect to an LVLH target frame. The target frame is generated independently by the LVLH frame model found in the utilities section. This LVLH frame need not be associated with a vehicle. Its definition requires only a reference planet and a translational state in planetary inertial coordinates. For more information, see the documentation for the LVLH frame model.

The Local-Vertical-Local-Horizontal (LVLH) Relative Derived State is a specialization of the *RelativeDerivedState* whose target frame is a rectilinear LVLH frame, possibly, but not necessarily, of a target vehicle. The *rel_state* field of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State indicates the state of the subject vehicle in rectilinear LVLH or optionally in a circular curvilinear system based on the target frame.

19.1 Using the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State in a Simulation

If your goal is to define and maintain an LVLH relative derived state, then you will need an instance of *LvlhRelativeDerivedState* in your S_define file. You normally want the initializations and updates of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State to occur subsequent to those of the *LvlhFrame*, so the *LvlhRelativeDerivedState* instance is often in a *SimObject* near the end of the S_define file.

You will also need to have an instance of *LvlhFrame* in your simulation. See the User Guide chapter of the documentation for the LVLH frame for details. The initialization process for LVLH frame registers the frame with the *DynManager* so that the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State can access it by name. Just as for *RelativeDerivedState*, you supply the names of the subject and target frames in the inputfile.

Example:

```
sim_object.lvlh_RelDerState.subject_frame_name =
"vehicleB.composite_body"
...
...
sim_object.lvlh_RelDerState.target_frame_name =
"vehicleA.composite_body.ref_planet.lvlh"
```

19.2 Other Options

There are two other optional values which you can set. The field *lvlh_type* should be one of two enumerated values, either *LvlhType.Rectilinear* for standard rectangular LVLH or *LvlhType.CircularCurvilinear* for output in circular curvilinear coordinates. The default is *LvlhType.Rectilinear*.

There is also a boolian *use_theta_dot_correction*. If set to true, the circular curvilinear angular velocity will account for rate of change of the phase angle θ. The default value is false.

19.3 Output Data

The *rel_state* field of *LvlhRelativeDerivedState* contains the state of the subject with respect to the target frame. The following code example shows how to log the entire state.

```
  for ii in range(0,3) :
    dr_group.add_variable("rel_state.vehicleA_wrt_vehicleB_in_B.
                          rel_state.trans.position[" + str(ii) + "]" )
  for ii in range(0,3) :
    dr_group.add_variable("rel_state.vehicleA_wrt_vehicleB_in_B.
                          rel_state.trans.velocity[" + str(ii) + "]" )
  for ii in range(0,3) :
    dr_group.add_variable("rel_state.vehicleA_wrt_vehicleB_in_B.
                          rel_state.rot.ang_vel_this[" + str(ii) + "]" )
  for ii in range(0,3) :
    for jj in range (0,3) :
      dr_group.add_variable("rel_state.vehicleA_wrt_vehicleB_in_B.
                            rel_state.rot.T_parent_this[" +
                            str(ii) + "][" + str(jj) + "]")
```

Chapter 20

Verification and Validation

20.1 Verification

A simulation was developed to test the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State. The simulation includes two vehicles, and LVLH relative derived states, both rectilinear and curvilinear, are computed. A software program is provided to check the derived state outputs in the generic cases. Two run cases are provided; one generic which tests most aspects of the model which can be independently verified by the software program and a second case which allows verification of the rotational curvilinear state by inspection.

Test DerivedState_3: Verification of Local-Vertical-Local-Horizontal (LVLH) Relative Derived State Output Data for Rectilinear LVLH

Purpose:
To demonstrate that the output of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State in the rectilinear case is consistent with existing LVLH code.

Requirements:
Satisfactory conclusion of this test satisfies requirement DerivedState_5

Procedure:
The data from the rectilinear LVLH output was compared against the Local-Vertical-Local-Horizontal (LVLH) Reference Frame for the following features:

1. Position.
2. Velocity.
3. Angular Velocity.
4. Calculation of Transformation matrix elements.

Predictions: The values should be identical.

Results:
The outputs were identical.

Test DerivedState_4: Verification of Local-Vertical-Local-Horizontal (LVLH) Relative Derived State Output Data for Curvilinear LVLH

Purpose:

To demonstrate that the output of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State in the curvilinear case matches theoretical predictions.

Requirements:

Satisfactory conclusion of this test satisfies requirement DerivedState_6

Procedure:

The data from the curvilinear output was compared against that of a computer program which used an independent method for calculating the following features:

1. Position.
2. Velocity.
3. Angular Momentum.

Predictions: The values should match to machine precision.

Results:

The outputs matched to machine precision.

Hand Calculation of Transformation Matrix

A second run case was included in which the transformation from curvilinear LVLH to the subject frame could easily be calculated by hand. The output of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State was identical to the hand calculations. The case was constructed in such a way that the curvilinear and rectilinear frames would differ by a 90 degree rotation, thus accentuating the effectiveness of the test.

20.2 Validation

There is no independent validation of the Local-Vertical-Local-Horizontal (LVLH) Relative Derived State.

Part V

North-East-Down Reference Frame

Chapter 21

Introduction

21.1 Purpose and Objectives of the North-East-Down Reference Frame

The North-East-Down Reference Frame is used to express the state of a vehicle with respect to some planet (usually the one about which it is orbiting) in terms of planetary North, planetary East, and local Down (with respect to the planet). The planetary basis can be specified either as spherical (geocentric), or elliptical (geodetic). For more discussion on the difference between spherical (geocentric) and elliptical (geodetic) representations, see the *Planetary Derived State Model* (on page 101).

While several of the Derived State values simply describe the state of a vehicle in some other defined reference frame, the North-East-Down Derived State also defines the reference frame itself. The state of the vehicle in the North-East-Down representation is identical to that in the Planetary representation, with both elliptical (geodetic) and spherical (geocentric) representations available. The North-East-Down reference frame is based on *either* the elliptical *or* the spherical representation; both are not available simultaneously unless two different instances of NedDerivedState are instantiated.

The North-East-Down reference frame, created by this model, is a recti-linear frame, so for two vehicles at the same altitude, the state of one in the NED frame of another will exhibit a "Down" component. See Figure 21.1 for an illustration.

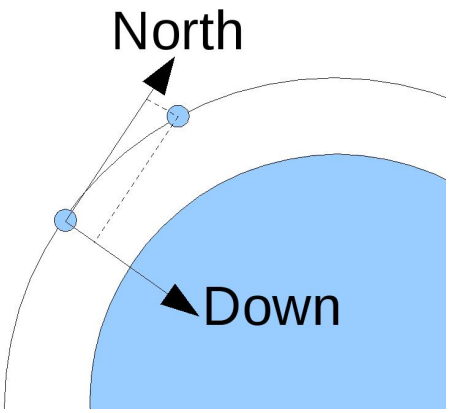


Figure 21.1: The effect of the curvature of an orbit on the relative state using rectilinear coordinates.

Chapter 22

Product Requirements

Requirement DerivedState_7: North-East-Down representation

Requirement:
The North-East-Down Derived State will provide the capability for expressing the state of a vehicle in the North-East-Down oriented reference frame of a planet.

Rationale:
Capability from JEOD 1.5.

Verification:
Inspection, Test

Requirement DerivedState_8: North-East-Down frame generation

Requirement:
The North-East-Down Derived State will provide the capability for generating a reference frame oriented with North, East, and Down, as described in the introduction, through which the state of another vehicle may be expressed.

Rationale:
Capability from JEOD 1.5.

Verification:
Inspection, Test

22.1 Requirements Traceability

Table 22.1: Requirements Traceability

Requirement	**Inspection and Testing**
DerivedState_7 - North-East-Down representation	Insp. DerivedState_5
DerivedState_8 - North-East-Down frame generation	Insp. DerivedState_6

Chapter 23

Product Specification

23.1 Conceptual Design

The North-East-Down Reference Frame is used to express the state of a vehicle with respect to a planet-fixed, rotating reference frame in a reference frame oriented with local North, East, and Down directions. The reference frame is defined by East, parallel to the equator (y-axis); Down, perpendicular to the surface (z-axis); and North (x-axis) completes the right-handed coordinate system. North is not, in general, aligned with the magnetic field, nor the planetary spin axis.

The calculations associated with representing the state in this way are handled by the North-East-Down subset of the planet-fixed model; see the *Planet-Fixed document* [4] for details.

23.2 Mathematical Formulations

The North-East-Down Reference Frame contains no independent mathematical formulations; the mathematics utilized by the model is provided by the North-East-Down subset of the planet-fixed model, and by the Reference Frames model; see the *Planet-Fixed* [4] and the *Reference Frames* [10] documentation for details. These documents are also linked as appropriate in the detailed design section of this document.

23.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

23.3.1 Process Architecture

The architecture for the North-East-Down Reference Frame is trivial, the North-East-Down Reference Frame methods are largely independent of one another.

23.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The North-East-Down Reference Frame contains only one class:

- **Class: NedDerivedState** **Inheritance:** *DerivedState* (on page 8)

  This contains the methods *compute_ned_frame*, *set_use_alt_pfix*, *initialize*, and *update*:

  1. **compute_ned_frame**

     This method utilizes methods from the planet-fixed model, see the *Planet Fixed* documentation [4] for details on these methods.

     It uses the NorthEastDown methods (from the planet-fixed model) *set_ned_trans_states* and *build_ned_orientation* to generate the NED state. The former, in turn, calls the PlanetFixedPosition method *update_from_cart* to convert the Cartesian representation into spherical and elliptical representations. The latter uses these generated latitude and longitude values to produce a transformation matrix and corresponding quaternions for going from the planet-fixed reference frame to the NorthEastDown reference frame.

  2. **set_use_alt_pfix**

     This method sets the flag indicating whether to use the default planet-centered, planet-fixed reference frame associated with the planet or the alternate one, if one is available for the planet.

  3. **initialize**

     The initialization process comprises the following steps:

     (a) The generic DerivedState initialization routine is called to establish the naming convention of the reference object (i.e., the planet about which the vehicle is orbiting) and state identifier.

     (b) The DerivedState method *find_planet* is called (which subsequently calls the *Dynamics Manager* [2]) method of the same name to find a planet by name; that name is stored as *reference_name*, and is usually assigned in an input file.

     (c) Depending on the value of *use_alt_pfix*, the element *pfix_ptr* is set to either the default planet-fixed reference frame or the alternate planet-fixed reference frame of the planet identified by *reference_name* and the frame is subscribed.

     (d) The name of the NED state is set as *vehicle_name.planet_name.ned*.

     (e) The North-East-Down reference frame is added as a child to the *planet_centered_planet_fixed* frame.

     (f) The North-East-Down frame is registered with the Dynamics manager (optional, user can specify with the *register_frame* flag, default is true).

     (g) The identified planet is then passed into the initialize method for the PlanetFixedPosition object, *ned_state* (*ned_state* is a NorthEastDown state, which inherits from PlanetFixedPosition), which simply records a pointer to the planet just found in the PlanetFixedPosition class (see *Planet Fixed* documentation [4]).

  4. **update**

     This method uses the RefFrame method *compute_relative_state* (see *Reference Frames* documentation [10]) to calculate the full state of the *subject* in the planet fixed reference

frame. Then, it passes the translational part of that state (i.e. position and velocity) into the *compute_ned_frame* method.

Chapter 24

User's Guide

The Analysis section of the User's Guide is intended primarily for users of pre-existing simulations. It contains:

- A description of how to modify North-East-Down Reference Frame variables after the simulation has compiled, including an in-depth discussion of the input file,
- An overview of how to interpret (but not edit) the S_define file,
- A sample of some of the typical variables that may be logged.

The Integration section of the User's Guide is intended for simulation developers. It describes the necessary configuration of the North-East-Down Reference Frame within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section of the user guide. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 23).

The Extension section of the User's Guide is intended primarily for developers needing to extend the capability of the North-East-Down Reference Frame. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 23).

24.1 Analysis

It must be reiterated that the North-East-Down Reference Frame does not provide a state for the vehicle with which it is associated, it provides a *reference frame*. The NedDerivedState does contain a member called *ned_state*, which contains the state of the reference frame with respect to the parent planet-centered planet-fixed reference frame. This is identical to the planet-fixed state (PlanetaryDerivedState) of the vehicle with respect to the same planet; it is **not** the North-East-Down state of the vehicle with respect to the same planet.

24.1.1 Identifying the North-East-Down Reference Frame

If the North-East-Down reference frame has been included in the simulation, there will be an instance of *NedDerivedState* located in the S_define file. This would typically be found in either the vehicle object (usually if the vehicle is to be described in a North-East-Down sense), or a separate relative-state object (usually if a relative state is to be expressed as such). There should be an accompanying call to an initialization routine, which takes a reference to the *subject_body* as one if its inputs, and an accompanying call to an update function. The essential variables *reference_name* and *ned_state.altlatlong_type* are defined elsewhere, often in the input file.

The North-East-Down reference frame, generated by the North-East-Down Reference Frame, may also be used to represent the state of another vehicle. In this case, there will be further inclusion of a RelativeDerivedState instance; see the *RelativeDerivedState User's Guide* (on page 130) for details on the implementation of RelativeDerivedState.

Example:

```
sim_object{
dynamics/derived_state:    NedDerivedState example_of_ned_state;

(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_ned_state.initialize (
    Inout DynBody &      subject_body = vehicle_1.dyn_body,
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);

{environment} dynamics/derived_state:
example_of_rel_state_object.example_of_ned_state.update ( )

} example_of_rel_state_object;
```

Then the input file may have entries comparable to:

```
example_of_rel_state_object.example_of_ned_state.reference_name = "Earth";
example_of_rel_state_object.example_of_ned_state.ned_state.altlatlong_type =
                                                    NorthEastDown::spherical;
```

24.1.2 Editing the North-East-Down Reference Frame

The planetary identification, and the planetary surface interpretation (spherical / elliptical) are open to edit by the analyst.

24.1.3 Output Data

The following outputs are available from the North-East-Down Reference Frame, but once again, these are simply the planetary derived states of the origin of the North-East-Down reference frame.

The strength of the North-East-Down Reference Frame comes in its applicability to a RelativeDerivedState; see the *RelativeDerivedState User's Guide* (on page 130) for details on the implementation of RelativeDerivedState.

```
example_of_rel_state_object.example_of_ned_state.ned_state.cart_coords[0-2]
example_of_rel_state_object.example_of_ned_state.ned_state.sphere_coords.altitude
example_of_rel_state_object.example_of_ned_state.ned_state.sphere_coords.latitude
example_of_rel_state_object.example_of_ned_state.ned_state.sphere_coords.longitude
example_of_rel_state_object.example_of_ned_state.ned_state.ellip_coords.altitude
example_of_rel_state_object.example_of_ned_state.ned_state.ellip_coords.latitude
example_of_rel_state_object.example_of_ned_state.ned_state.ellip_coords.longitude
```

24.2 Integration

Simply including the North-East-Down Reference Frame is straightforward; using it to its full capacity is more challenging.

24.2.1 Generating the S_define

It should be reiterated that if the desire is simply to express the state of a vehicle with respect to the planet, the NedDerivedState and PlanetaryDerivedState perform the same task. However, the PlanetaryDerivedState is preferred in this situation, because it does not perform the additional task of generating a reference frame.

The North-East-Down model should be used when the state of a vehicle is desired with respect to some point that is not at the center of the planet, e.g., a launch facility. Under these circumstances, it becomes necessary to instantiate both a NedDerivedState, and a RelativeDerivedState, with the understanding that the NedDerivedState must be evaluated before the RelativeDerivedState (otherwise the RelativeDerivedState output will be based on an out-of-date orientation of the NED reference frame).

An important consequence of using a RelativeDerivedState to express the relative states of two objects in a North-East-Down reference frame requires the consideration of the class structure of both NedDerivedState and RelativeDerivedState. There are two types of reference frames in use, BodyRefFrame, and RefFrame. The BodyRefFrame is associated with a DynBody (a body with dynamic properties), while the RefFrame need not be, it can be associated with some fixed (non-dynamic) body or point.

The North-East-Down reference frame in a NedDerivedState is an instance of a RefFrame, there is no requirement that this be attached to a DynBody. However, the RelativeDerivedState contains both a BodyRefFrame (*subject_frame*) and a RefFrame (*target_frame*); the intention is that the state of a body can be expressed in some other reference frame. Therefore, when generating the relative states, it is *not* possible to use the generated North-East-Down reference frame as the subject frame, it must instead be the target frame. Then, the vehicle whose state is being expressed must be associated with the subject frame, and the *subject_frame_name* and *target_frame_name* must be set appropriately. Then, the *direction_sense* of the Relative Derived State must be *Com-*

puteSubjectStateinTarget.

An illustration of this is found in the verification simulations for the North-East-Down model (verif/SIM_NED). Here, the relative state instance that expresses the state of vehicle1 (*veh*) with respect to vehicle2 (*veh2*), *veh_wrt_veh2*, has:

```
subject_frame_name = "vehicle1.composite_body";
target_frame_name = "vehicle2.Earth.ned";
direction_sense = RelativeDerivedState::ComputeSubjectStateinTarget;
```

Notice also in these simulations that the NedDerivedState is contained within the respective vehicle object, while the RelativeDerivedState is contained within the *rel_state* object. This assignment is intentional, though not necessary. What is necessary is that the state of the two vehicles be evaluated before the relative state, and that the NED reference frame be appropriately oriented, based on the state of its parent vehicle, before the relative state can be expressed in that frame. Therefore, it was chosen to move the relative state calculations to the end of the S_define, where they will be processed only after the vehicle and NED reference frames have been processed. This is recommended procedure.

24.2.2 Generating the Input File

The input file (or Modified Data file) must contain the name of the planet, identified as *reference_name*, and the identification of whether the North-East-Down reference frame should be based on a spherical (geocentric) or elliptical (geodetic) surface.

```
example_of_rel_state_object.example_of_ned_state.reference_name = "Earth";
example_of_rel_state_object.example_of_ned_state.ned_state.altlatlong_type =
                                                    NorthEastDown::spherical;
```

or

```
example_of_rel_state_object.example_of_ned_state.reference_name = "Earth";
example_of_rel_state_object.example_of_ned_state.ned_state.altlatlong_type =
                                                  NorthEastDown::elliptical;
```

The user may specify which planet-centered, planet-fixed reference frame (default or alternate) to use as the basis of the relative state calculations for the North-East-Down reference frame. Note: this only works with Planets that have an alternate reference frame defined.

```
example_of_rel_state_object.example_of_ned_state.set_use_alt_pfix(False);
```

or

```
example_of_rel_state_object.example_of_ned_state.set_use_alt_pfix(True);
```

24.2.3 Logging the Data

See the *Analysis* (on page 77) section for a list of available output data, although again, typically, the desired output results from the relative state implementation of the NED reference frame, rather than from the NED state. See the *Analysis* (on page 130) section for a list of available output data from a RelativeDerivedState.

24.3 Extension

The North-East-Down Reference Frame is not intended to be extensible. A potential area for further development may be the addition of a comparable reference frame, such as East-North-Up, but this would be a separate entity, not an extension of the NED reference frame.

Chapter 25

Verification and Validation

25.1 Verification

25.1.1 Inspection of Modeling Requirements

Inspection DerivedState_3: State Encapsulation

The North-East-Down Reference Frame is capable of outputting data as desired, as demonstrated in simulations located at *derived_state/verif/SIM_NED/*, thereby satisfying requirement Derived-State_7 at the inspection level.

Inspection DerivedState_4: Frame Generation

The North-East-Down Reference Frame is capable of generating a reference frame, as demonstrated in simulations located at *derived_state/verif/SIM_NED/*, thereby satisfying requirement Derived-State_8 at the inspection level.

25.1.2 Verification of Model

For testing the North-East-Down Reference Frame, a simulation of two vehicles was established in which the two vehicles are separated by 1 degree on a common orbital trajectory. Both vehicles generate a North-East-Down reference frame, and the relative state of the other vehicle is calculated for both.

Test DerivedState_5: Verification of North-East-Down Reference Frame Output Data for Generic Orbits

Purpose:
To demonstrate that the data output by the North-East-Down Reference Frame provides meaningful data.

Requirements:

Satisfactory conclusion of this test satisfies requirement DerivedState_7

Procedure:

The data from the North-East-Down output was compared against that from a Planetary Derived State for an identical orbit. Polar, inclined, and equatorial orbital characteristics were each tested.

Predictions: The state output from this simulation should match that from a Planetary Derived State, which has been verified independently in the *PlanetaryDerivedState Verification Section* (on page 109).

Results: There appeared to be no, or negligible, differences between the two states.

Test DerivedState_6: Verification of the reference frame orientation

Purpose:

To demonstrate that the reference frame generated by the North-East-Down Reference Frame is oriented along the intended axes.

Requirements:

Satisfactory conclusion of this test satisfies requirement DerivedState_8

Procedure:

The relative state data between the two vehicles was compared against theoretical prediction for consistency for polar, inclined, and equatorial orbital characteristics.

Predictions:

1. Polar orbit.
   (a) North. The lead vehicle should be North of the trailing vehicle while latitude is increasing, and South of the trailing vehicle while the latitude is decreasing. The transition should be sharp, though not coincident between the two frames; as the vehicles cross the pole, the state of the trailing vehicle in the NED frame associated with the lead vehicle should transition first, resulting in a short time when both vehicles are north (or south) of each other.
   (b) East. The relative states should show zero East component in their respective positions.
   (c) Down. Because the NED states are rectilinear, both vehicles should be represented as having a state with a positive down component at all times. See Figure 21.1 for a partial illustration of this counter-intuitive phenomenon. The behavior is also dependent upon the planetary geometry:
      - Spherical. The Down component should be constant, at a value equal to $r - r\cos\Delta\theta$. For these simulations, this value equates to $1.03km$.
      - Elliptical. The Down component should oscillate. In these simulations, the orbit is approximately circular, whereas the definition of *down* is elliptical. As the vehicles leave the equator, the trailing vehicle should be significantly further below the leading vehicle than the leading vehicle below the trailing vehicle.

As the vehicles approach the poles, the two values should equalize to $1.03km$, due to symmetry, then drift apart again heading back to the equator; this time, however, the leading vehicle should have the larger down component. As the vehicles approach the equator again, the values should again equalize, again due to symmetry. An exaggerated illustration of this is shown in Figure 25.1.

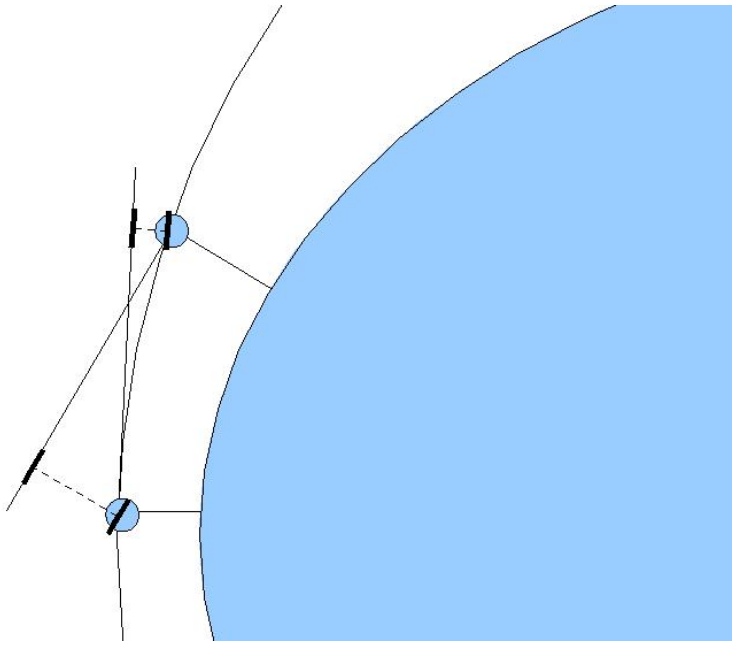


Figure 25.1: The effect of the difference between the curvature of an orbit and that of the reference ellipse on the relative state using rectilinear coordinates. The vehicle closer to the equator will exhibit a larger down component in its state than the vehicle farther from the equator.

2. Inclined orbit.
   (a) North. As for the polar orbit, the lead vehicle should be North of the trailing vehicle while latitude is increasing, and South of the trailing vehicle while the latitude is decreasing. Conversely, the transitions at the poles will differ from those observed for the polar orbit; for the inclined orbit, the North component of the relative state should be oscillatory, and ninety degrees out-of-phase from the latitude measurement (so that the North component of the relative state reaches an extremum when the vehicles are at the equator, and goes to zero when the vehicles are at their extreme latitudes).
   (b) East. The lead vehicle should always be East of the trailing vehicle, and the trailing vehicle negative-East of the lead vehicle.
   (c) Down. By the same argument put forth for the polar orbit, both vehicles should be represented as having a state with a positive down component at all times, constant for spherical geometry and oscillating (though with smaller magnitude due to the variation of the curvature being less significant on an inclined orbit) for the elliptical geometry.
3. Equatorial orbit.

(a) North. Both relative states should have zero North component.

(b) East. The lead vehicle should always be East of the trailing vehicle, and the trailing vehicle negative-East of the lead vehicle.

(c) Down. By the same argument put forth for the polar orbit, both vehicles should be represented as having a state with a positive down component at all times, but now be constant for both spherical and elliptical geometries.

Results:

1. Polar orbit.
   The results were mostly as expected. Figures 25.2 and 25.3 show the results from the elliptical-geometry reference frame, and Figure 25.4 the results from the spherical-geometry reference frame.

   (a) North. The instantaneous switch is seen at polar crossings, with the offset between the two states illustrated better in Figure 25.3. This is at a South pole crossing, and it is clear that the trailing vehicle first transitions to being South of the leading vehicle, then the leading vehicle transitions to being North of the trailing vehicle.

   (b) East. The East component is not uniformly zero, it has some instantaneous spikes in its value. This is attributed to numerical error; those spikes all occur as the vehicles transit the poles, where North and East are not well defined.

   (c) Down. For the spherical geometry, the down component shows some oscillation, but that oscillation has magnitude less than $1cm$, and the mean has the expected value of $1.03km$. For the elliptical geometry, the oscillation can clearly be seen. Figure 25.3 shows the oscillation from equator heading North (trailing vehicle has larger down component), through the pole at about $1400s$, then back South to the equator again at approximately $2800s$.

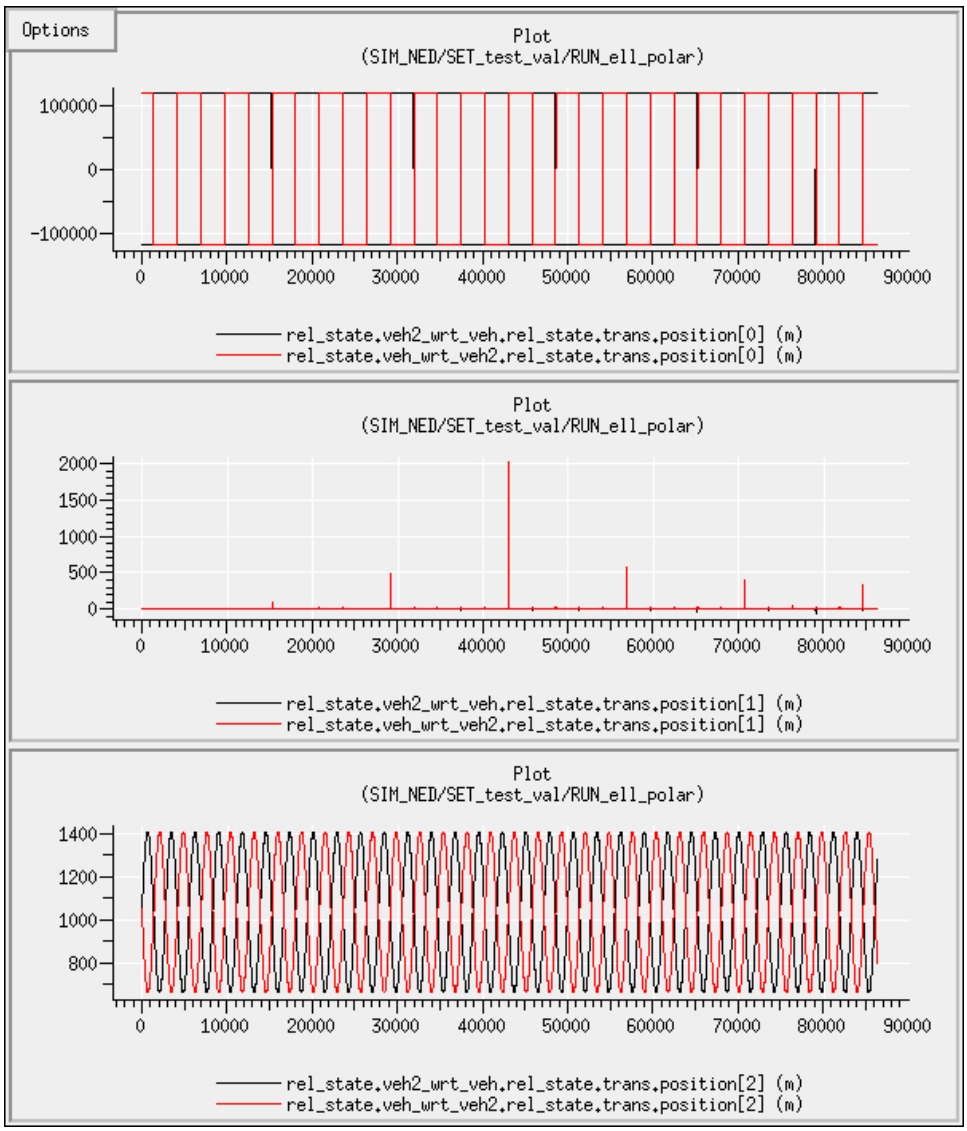


Figure 25.2: The relative states for a polar orbit, with elliptical geometry used to define the reference frame.

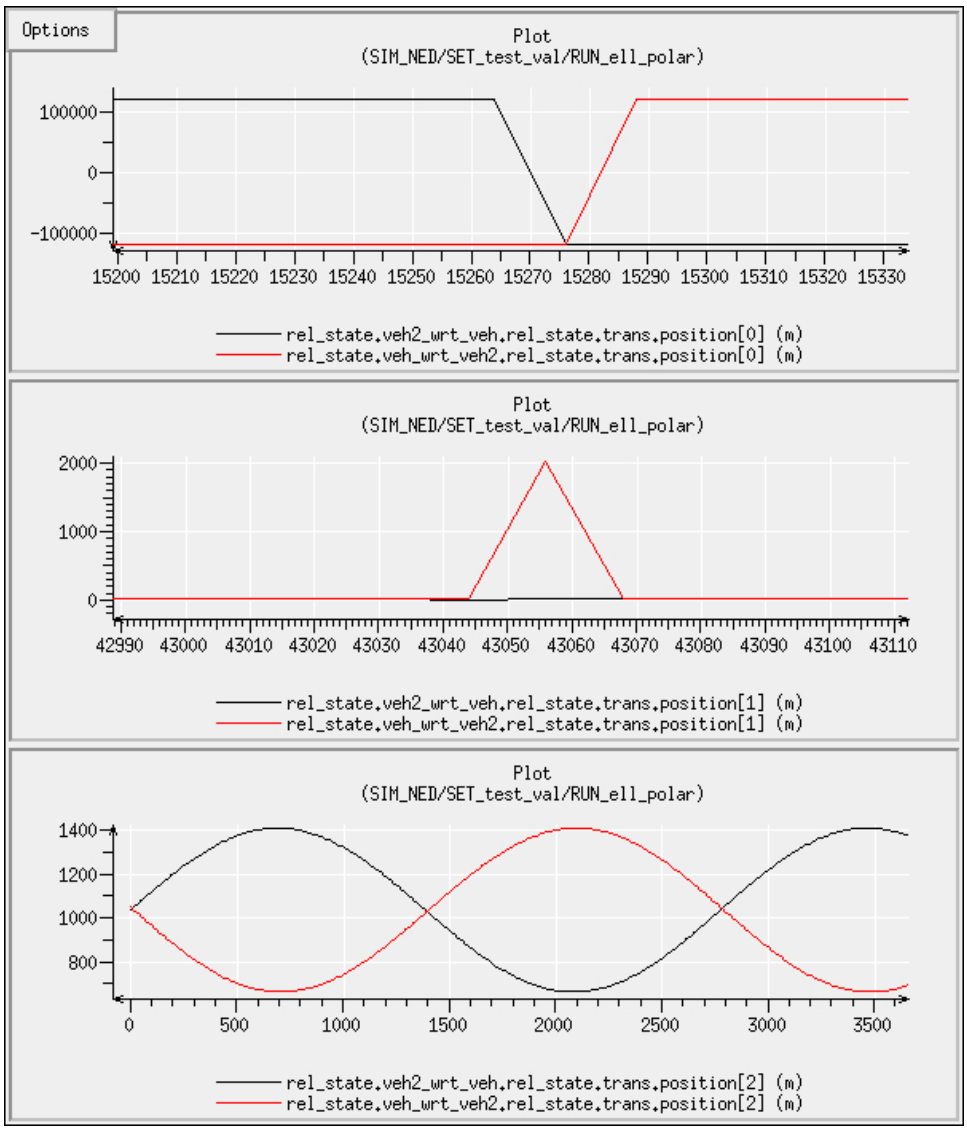


Figure 25.3: The relative states for a polar orbit, with elliptical geometry used to define the reference frame, zoomed in to locations of interest.

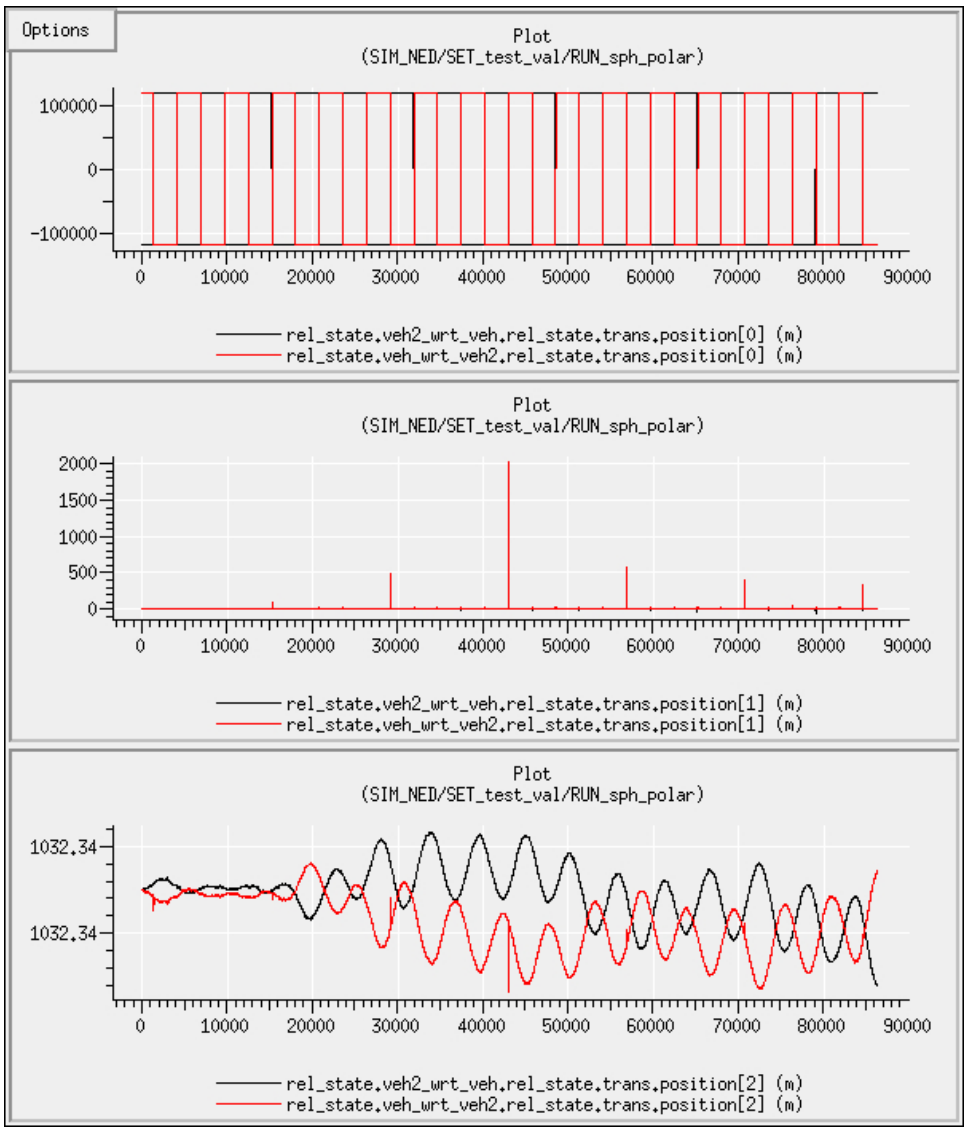


Figure 25.4: The relative states for a polar orbit, with spherical geometry used to define the reference frame.

2. Inclined Orbit.
   The results match with expectation. Figure 25.5 shows the North and East components with the latitude history. The Down components had a similar oscillation to those seen in the polar orbit and are not shown. A small difference exists of order meters between the North components of the spherical and elliptical geometries, East shows no dependence on geometry, and Down shows a large dependence, as it did in the polar orbit.

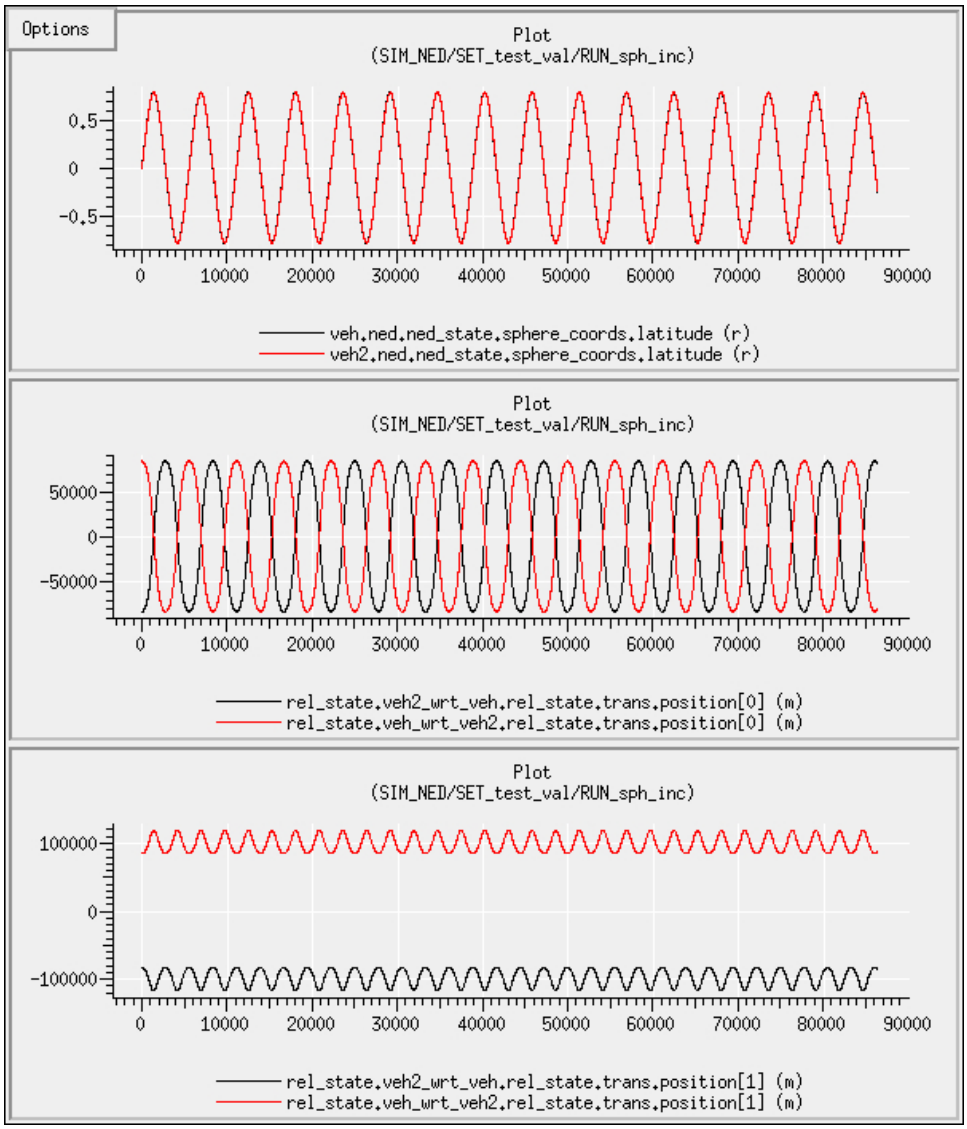


Figure 25.5: The variation with time of latitude and of the North and East components of the relative states for an orbit inclined at 45 degrees, with spherical geometry used to define the reference frame.

3. Equatorial Orbit.
   The results match with expectation. Figure 25.6 shows the state components; there was no noticeable difference between the spherical and elliptical geometries.

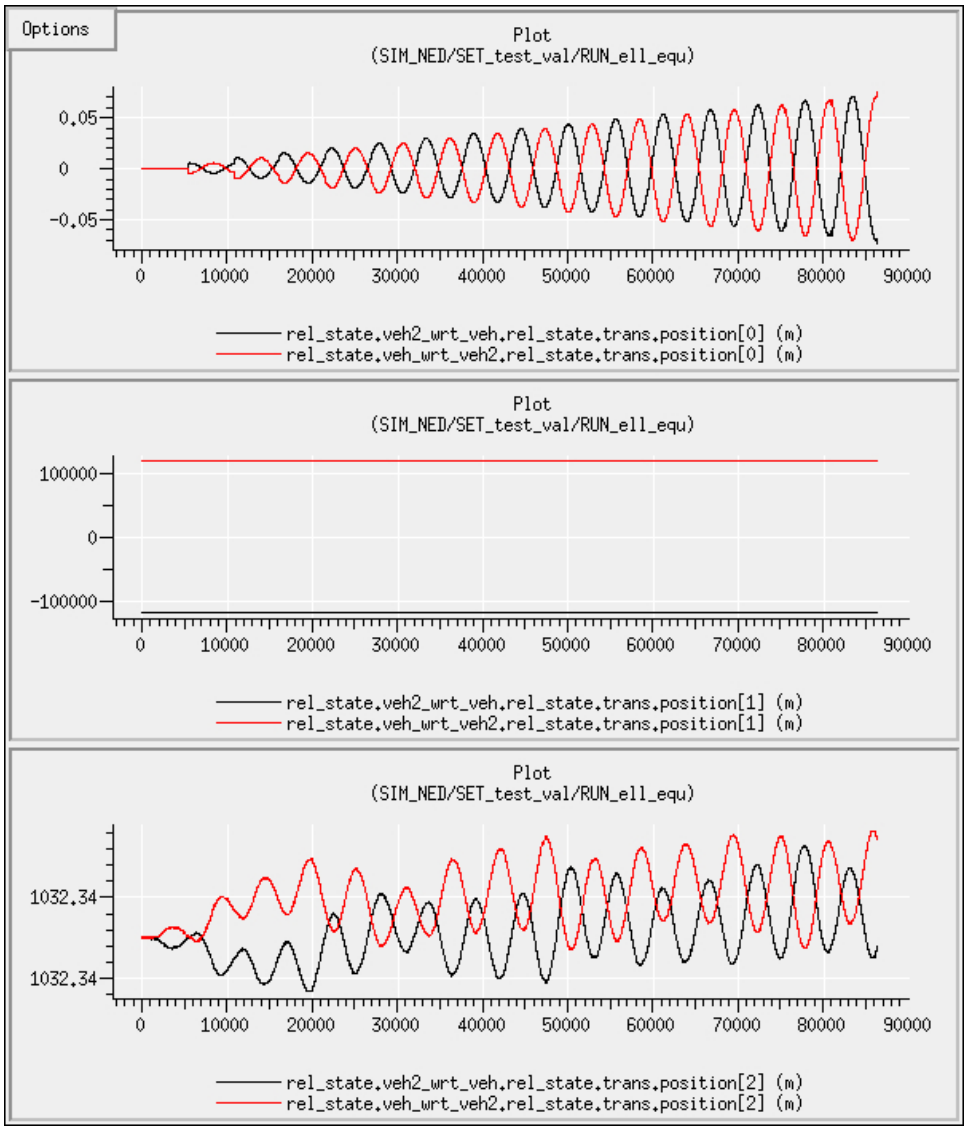


Figure 25.6: The relative states for an equatorial orbit with spherical geometry used to define the reference frame.

25.2 Validation

There is no independent validation of this model.

Part VI

Orbital Elements State Representation

Chapter 26

Introduction

26.1 Purpose and Objectives of the Orbital Elements State Representation

The Orbital Elements state representation is used to express the state of a vehicle with respect to the planet about which it is orbiting in terms of its orbital elements:

- Semi-major axis
- Semiparameter, or semi-latus rectum
- Eccentricity
- Inclination
- Argument of periapsis
- Longitude of ascending node
- Anomaly (true, mean, and either eccentric, hyperbolic, or parabolic)
- Mean motion
- Specific orbital energy
- Specific orbital angular momentum

Chapter 27

Product Requirements

Requirement DerivedState_9: Cartesian - Orbital Elements conversion

Requirement:
The Orbital Elements Derived State shall provide the capability of outputting data from a cartesian framework as an equivalent set of Keplerian orbital elements.

Rationale:
requirement from JEOD 1.5.

Verification:
Inspection

27.1 Requirements Traceability

Table 27.1: Requirements Traceability

Requirement	Inspection and Testing
DerivedState_9 - Cartesian - Orbital Elements conversion	Insp. DerivedState_5

Chapter 28

Product Specification

28.1 Conceptual Design

The entire process of generating the orbital elements is handled by the Orbital Elements model; see the Orbital Elements document [9] for details.

28.2 Mathematical Formulations

The Orbital Elements state representation has no mathematical formulations, it relies entirely on the Orbital Elements model.

28.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

28.3.1 Process Architecture

The architecture for the Orbital Elements state representation is trivial, comprising few methods that are largely independent.

28.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The Orbital Elements state representation contains only one class:

- **Class: OrbElemDerivedState** **Inheritance:** *DerivedState* (on page 8)

  This contains only the methods *compute_orbital_elements*, *set_use_alt_inertial*, *initialize* and *update*:

1. **compute_orbital_elements**

   Simply calls the Orbital Elements method *from_cartesian* to convert the data from a Cartesian frame to a set of orbital elements. See the Orbital Elements document [9] for details on this method.

2. **set_use_alt_inertial**

   This method sets the flag indicating whether to use the default inertial reference frame associated with the planet or the alternate one, if one is available for the planet.

3. **initialize**

   The initialization process comprises the following steps:

   (a) The generic DerivedState initialization routine is called to assign the *subject* value, establish the naming convention for the state identifier.

   (b) Calls the Derived State method, *find_planet* to identify the planet with name specified by *reference_name*.

   (c) Calls the Orbital Elements routine *set_object_name* to set the name of the vehicle.

   (d) Depending on the value of *use_alt_inertial*, the element *inertial_ptr* is set to either the default inertial frame or the alternate inertial frame of the planet identified by *reference_name* and the frame is subscribed.

   (e) Calls the Orbital Elements routine *set_planet_name* to set the name of the planet.

4. **update**

   Calls the *compute_orbital_elements* routine on the planet-relative state. If the integration frame is planet centered inertial, it uses the basic state of the vehicle. Otherwise, it calculates the relative state of the vehicle with respect to planet-centered inertial, and uses that as the input to *compute_orbital_elements*.

Chapter 29

User's Guide

The Analysis section of the User's Guide is intended primarily for users of pre-existing simulations. It contains:

- A description of how to modify Orbital Elements state representation variables after the simulation has compiled, including an in-depth discussion of the input file,
- An overview of how to interpret (but not edit) the S_define file,
- A sample of some of the typical variables that may be logged.

The Integration section of the User's Guide is intended for simulation developers. It describes the necessary configuration of the Orbital Elements state representation within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section of the user guide. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 28).

The Extension section of the User's Guide is intended primarily for developers needing to extend the capability of the Orbital Elements state representation. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 28).

29.1 Analysis

29.1.1 Identifying the Orbital Elements State Representation

If Orbital Elements have been included in the simulation, there will be an instance of *OrbElemDerivedState* located in the S_define file. This would typically be found in either the vehicle object, or a separate relative-state object. There should be an accompanying call to an initialization routine, which takes a reference to the *subject_body* as one if its inputs, and an accompanying call to an update function. The essential *reference_name* is defined elsewhere, often in the input file.

Example:

```
sim_object{
dynamics/derived_state:    OrbElemDerivedState example_of_orb_elem_state;

(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_orb_elem_state.initialize (
    Inout DynBody &      subject_body = vehicle_1.dyn_body,
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);

{environment} dynamics/derived_state:
example_of_rel_state_object.example_of_orb_elem_state.update ( )

} example_of_rel_state_object;
```

Then the input file may have entries comparable to:

```
example_of_rel_state_object.example_of_orb_elem_state.reference_name = "Earth";
```

29.1.2 Editing the Orbital Elements State Representation

There is nothing to edit in the Orbital Elements state representation. The only public data elements are the output data.

29.1.3 Output Data

The various outputs from the Orbital Elements state representation, listed in the introduction, are accessible as follows:

```
example_of_rel_state_object.example_of_orb_elem_state.elements.semi_major_axis,
example_of_rel_state_object.example_of_orb_elem_state.elements.semiparam,
example_of_rel_state_object.example_of_orb_elem_state.elements.e_mag,
example_of_rel_state_object.example_of_orb_elem_state.elements.inclination,
example_of_rel_state_object.example_of_orb_elem_state.elements.arg_periapsis,
example_of_rel_state_object.example_of_orb_elem_state.elements.long_asc_node,
example_of_rel_state_object.example_of_orb_elem_state.elements.true_anom,
example_of_rel_state_object.example_of_orb_elem_state.elements.mean_anom,
example_of_rel_state_object.example_of_orb_elem_state.elements.orbital_anom,
example_of_rel_state_object.example_of_orb_elem_state.elements.mean_motion,
example_of_rel_state_object.example_of_orb_elem_state.elements.orb_energy,
example_of_rel_state_object.example_of_orb_elem_state.elements.orb_ang_momentum.
```

29.2 Integration

Including the Orbital Elements state representation is relatively straightforward.

29.2.1 Generating the S_define

Conventional practice would add the Orbital Elements state representation to the vehicle object, or (if one exists) to a separate relative-state object at the end of the S_define file. When adding the state to a specific vehicle, care must be taken to ensure that the calls to update the state occur after the planet position has been generated from the ephemerides, and the vehicle position integrated. There is no internal check that this ordering is achieved, the responsibility for ensuring this lies entirely on the Integrator.

The instance of *OrbElemDerivedState* needs to be defined, the model initialized, and a routine update scheduled. A simple example of how this may look is found in the *Analysis* (on page 96) section. A simple example simulation is found in the *derived_state/verif/OrbElem* verification simulations released with JEOD v5.4.

29.2.2 Generating the Input File

The input file (or Modified Data file) must contain the name of the planet, identified as *reference_name*.

The user may specify which inertial reference frame (default or alternate) to use as the basis of the orbital element calculations. Note: this only works with Planets that have an alternate reference frame defined.

```
example_of_rel_state_object.example_of_orb_elem_state.set_use_alt_inertial(False);
```

or

```
example_of_rel_state_object.example_of_orb_elem_state.set_use_alt_inertial(True);
```

See the *Analysis* (on page 96) section for an example of how the input data may appear.

29.2.3 Logging the Data

See the *Analysis* (on page 96) section for a list of available output data.

29.3 Extension

The Orbital Elements state representation is not intended to be extended.

Chapter 30

Verification and Validation

30.1 Verification

Inspection DerivedState_5: State Encapsulation

The Orbital Elements state representation is capable of outputting data as desired, as demonstrated in simulation *derived_state/verif/OrbElem/SIM_OrbElem/*, thereby satisfying requirement DerivedState_9 at the inspection level. See the Orbital Elements document [9] for verification of the Orbital Elements calculations.

30.2 Validation

There is no independent validation of the Orbital Elements state representation. See the Orbital Elements document [9] for validation of the Orbital elements calculations.

Part VII

Planetary Reference Frame State Representation

Chapter 31

Introduction

31.1 Purpose and Objectives of the Planetary Reference Frame State Representation

The Planetary Reference Frame state representation provides the state of a vehicle in three different representations:

- Cartesian coordinates
  Cartesian coordinates are centered and fixed with the planet
- Spherical coordinates
  These are centered on the planet, with ϕ measured from the Cartesian z-axis, and θ measured in the Cartesian x-y plane from the Cartesian x-axis toward the Cartesian y-axis. The position is represented as *latitude* $(\frac{\pi}{2} - \phi)$, *longitude* (θ), and *altitude* $(r - r_{equatorial})$.
- Elliptical coordinates
  These are also centered on the planet, with elliptical longitude and latitude being measured to the point on the reference ellipsoid (surface) that has a normal that passes through the point of interest. See Figure 31.1 for a graphical distinction between spherical and elliptical coordinates.

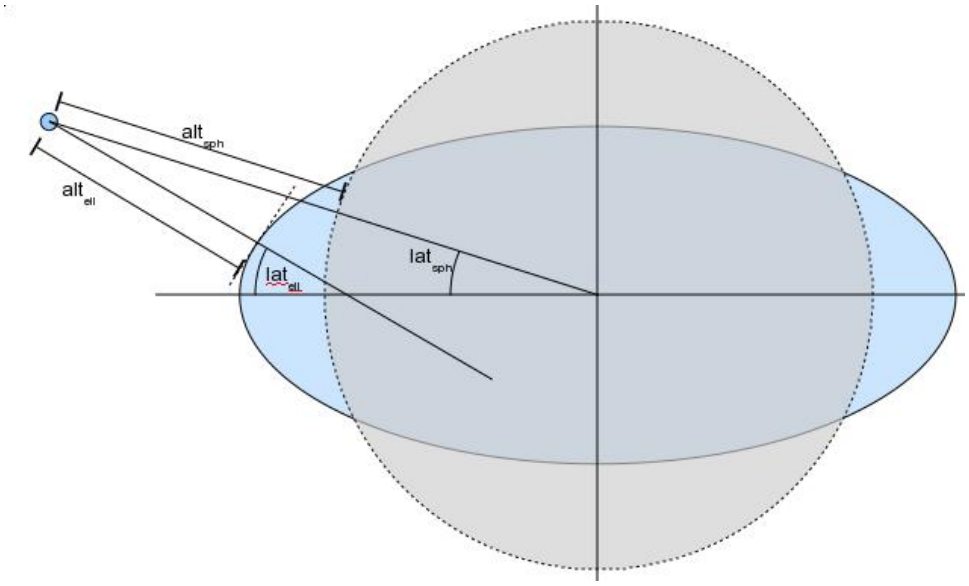


Figure 31.1: The differences between spherical and elliptical altitude and latitude.

Chapter 32

Product Requirements

Requirement DerivedState_10: Planetary representation

Requirement:
The Planetary Derived State will provide the capability for expressing the state of one body in a planetary reference frame.

Rationale:
Capability from JEOD 1.5.

Verification:
Test

32.1 Requirements Traceability

Table 32.1: Requirements Traceability

Requirement	Inspection and Testing
DerivedState_10 - Planetary representation	Test DerivedState_7

Chapter 33

Product Specification

33.1 Conceptual Design

The Planetary Reference Frame state representation is used to express the state of a vehicle with respect to a planet-fixed, rotating reference frame. This is useful, for example, for finding the position of a vehicle with respect to some fixed point on a planetary surface, or for plotting a vehicle trajectory over a map of a planet.

There are three methods for presenting the state, these are described in the *Introduction* (on page 101).

33.2 Mathematical Formulations

The mathematical details of the Planetary Reference Frame state representation are all handled by other models. See the Detailed Specification for links.

33.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

33.3.1 Process Architecture

The architecture for the Planetary Reference Frame state representation is trivial, comprising methods that are operationally independent.

33.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The Planetary Reference Frame state representation contains only one class:

- **Class: PlanetaryDerivedState** **Inheritance:** *DerivedState* (on page 8)

  This contains only the methods *set_use_alt_pfix*, *initialize*, and *update*:

  1. **set_use_alt_pfix**

     This method sets the flag indicating whether to use the default planet-centered, planet-fixed reference frame associated with the planet or the alternate one, if one is available for the planet.

  2. **initialize**

     The initialization process comprises the following steps:

     (a) The generic DerivedState initialization routine is called to establish the naming convention of the reference object (i.e., the planet about which the vehicle is orbiting) and state identifier.

     (b) The DerivedState method *find_planet* is called (which subsequently calls the *Dynamics Manager* [2]) method of the same name to find a planet by name; that name is stored as *reference_name*, and is usually assigned in an input file.

     (c) Depending on the value of *use_alt_pfix*, the element *pfix_ptr* is set to either the default planet-fixed reference frame or the alternate planet-fixed reference frame of the planet identified by *reference_name* and the frame is subscribed.

     (d) The identified planet is then passed into the initialize method for the PlanetFixedPosition object, *state*, which simply records a pointer to the planet just found in the PlanetFixedPosition class (see *Planet Fixed* documentation [4]).

  3. **update**

     This method uses the RefFrame method *compute_position_from* (see *Reference Frames* documentation [10]) to calculate the Cartesian position of the *subject* in the planet fixed reference frame. Then, it uses the PlanetFixedPosition method *update_from_cart* (see *Planet Fixed* documentation [4]) to convert the Cartesian representation into spherical and elliptical representations.

Chapter 34

User's Guide

The Analysis section of the User's Guide is intended primarily for users of pre-existing simulations. It contains:

- A description of how to modify Planetary Reference Frame state representation variables after the simulation has compiled, including an in-depth discussion of the input file,
- An overview of how to interpret (but not edit) the S_define file,
- A sample of some of the typical variables that may be logged.

The Integration section of the User's Guide is intended for simulation developers. It describes the necessary configuration of the Planetary Reference Frame state representation within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section of the user guide. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 33).

The Extension section of the User's Guide is intended primarily for developers needing to extend the capability of the Planetary Reference Frame state representation. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 33).

34.1 Analysis

34.1.1 Identifying the Planetary Reference Frame State Representation

If Planet-based states have been included in the simulation, there will be an instance of *Planetary-DerivedState* located in the S_define file. This would typically be found in either the vehicle object, or a separate relative-state object. There should be an accompanying call to an initialization routine, which takes a reference to the *subject_body* as one if its inputs, and an accompanying call to an update function. The *reference_name* is defined elsewhere, possibly in the input file; this is the name of the planet,with respect to which the state will be evaluated.

Example:

```
sim_object{
dynamics/derived_state:    PlanetaryDerivedState example_of_planetary_state;

(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_planetary_state.initialize (
    Inout DynBody &      subject_body = vehicle_1.dyn_body,
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);

{environment} dynamics/derived_state:
example_of_rel_state_object.example_of_planetary_state.update ( )

} example_of_rel_state_object;
```

Then the input file may have an entry comparable to:

```
example_of_rel_state_object.example_of_planetary_state.reference_name = "Earth";
```

34.1.2 Editing the Planetary Reference Frame State Representation

There is very little to edit in the Planetary Reference Frame state representation, apart from the *reference_name*, which will result in the planetary state being calculated with respect to a different planet. It should be noted that *reference_frame* is only used at initialization, and therefore changing this name *after* the initialization will have no effect. The *reference_name* is used to identify the appropriate planet memory location during initialization and thereafter the variable *planet* is used, not *reference_frame*.

34.1.3 Output Data

The output available from this model is the position of the vehicle with respect to the planet, in planet-fixed coordinates. This position is available as a Cartesian, spherical, or elliptical coordinate representation.

```
example_of_rel_state_object.example_of_planetary_state.state.cart_coords[0-2]
example_of_rel_state_object.example_of_planetary_state.state.sphere_coords.altitude
example_of_rel_state_object.example_of_planetary_state.state.sphere_coords.latitude
example_of_rel_state_object.example_of_planetary_state.state.sphere_coords.longitude
example_of_rel_state_object.example_of_planetary_state.state.ellip_coords.altitude
example_of_rel_state_object.example_of_planetary_state.state.ellip_coords.latitude
example_of_rel_state_object.example_of_planetary_state.state.ellip_coords.longitude
```

The rotational state is not available.

34.2 Integration

Including the Planetary Reference Frame state representation is relatively straightforward.

34.2.1 Generating the S_define

Conventional practice would add the Planetary Reference Frame state representation to the vehicle object, or (if one exists) to a separate relative-state object at the end of the S_define file. When adding the state to a specific vehicle, care must be taken to ensure that the calls to update the state occur after the planet position has been generated from the ephemerides, and the vehicle position integrated. There is no internal check that this ordering is achieved, the responsibility for ensuring this lies entirely on the Integrator.

The instance of *PlanetaryDerivedState* needs to be defined, the model initialized, and a routine update scheduled. A simple example of how this may look is found in the *Analysis* (on page 106) section. Simple example simulations are found in the *derived_state/verif/Planetary* verification simulations released with JEOD v5.4.

34.2.2 Generating the Input File

The input file (or Modified Data file) must contain the name of the planet, identified as *reference_name*.

In the input file, the user may specify which planet-centered, planet-fixed reference frame (default or alternate) to use as the basis of the position calculations. Note: this only works with Planets that have an alternate reference frame defined.

```
example_of_rel_state_object.example_of_planetary_state.set_use_alt_pfix(False);
```

or

```
example_of_rel_state_object.example_of_planetary_state.set_use_alt_pfix(True);
```

See the *Analysis* (on page 106) section for an example of how the input data may appear.

34.2.3 Logging the Data

See the *Analysis* (on page 106) section for a list of available output data.

34.3 Extension

The Planetary Reference Frame state representation is not intended to be extensible.

Chapter 35

Verification and Validation

35.1 Verification

35.1.1 Inspection of Modeling Requirements

Inspection DerivedState_6: State Encapsulation

The Planetary Reference Frame state representation is capable of outputting data as desired, thereby satisfying requirement DerivedState_10 at the inspection level. The validity of the output data is tested below.

35.1.2 Verification of Surface Representations

Test DerivedState_7: Verification of Planetary Reference Frame state representation Output Data for Generic Orbits

Purpose:
To demonstrate that the data output by the Planetary Reference Frame state representation provides meaningful data.

Requirements:
Satisfactory conclusion of this test satisfies requirement DerivedState_10

Procedure:
A simulation was developed containing an arbitrary vehicle initialized into one of several orbits:

1. Low Earth Orbit, equatorial.
   Initialized with an apo-altitude and peri-altitude of 400 km, and an inclination of zero.
2. Low Earth Orbit, inclined.
   Initialized with an apo-altitude and peri-altitude of 400 km, and an inclination of 45 degrees.

3. Low Earth Orbit, polar.
   Initialized with an apo-altitude and peri-altitude of 400 km, and an inclination of 90 degrees.
4. Eccentric Earth Orbit, equatorial.
   Initialized with an apo-altitude of 8,000 km, a peri-altitude of 400 km, and an inclination of zero.
5. Near-Geosynchronous Orbit.
   Initialized with an apo-altitude and peri-altitude of 35,786 km, and an inclination of zero.

The data from the five simulations were compared against theory and against each other.

Predictions:

1. Low Earth Orbit, equatorial
   - The Cartesian coordinate output from this data should be close to 0 on the z-axis, and be oscillatory on the x-and y- axes, with a period equal to the synodic period of the satellite, approximately 5,940 seconds.
   - The inertial position should oscillate on all three axes with a period equal to the sidereal period of the satellite, approximately 5,560 seconds.
   - The altitude should remain approximately constant. The longitude should have a saw-tooth appearance with a variation that is linear with time, rising to π radians, and resetting to $-\pi$ radians instantaneously, with a period equal to the synodic period of the satellite. The latitude should remain at zero.
2. Low Earth Orbit, inclined
   - The Cartesian coordinate output from this data should show the same periodic behavior as for the equatorial orbit on the x- and y-axes, with an additional 1-day oscillation in the magnitude of the x- and y- axis magnitudes due to the rotation of Earth underneath the orbit (when the orbital nodes lie close to the x-axis, the x-axis oscillation should be close to the full magnitude of 6,800 km, and the y-axis oscillation only at $6,778km\ \sin(\pi/2) = 4,800km$, and vice-versa. On the z-axis, there should be some oscillation with a magnitude of approximately $6,778km\ \sin(\pi/2) = 4,800km$, and a period equal to the sidereal period of approximately 5,560 seconds.
   - The inertial position should show a similar pattern to that for the equatorial orbit, with the same period.
   - The altitude should remain approximately constant when expressed in spherical coordinates, but differ significantly when expressed in elliptical (geodetic) coordinates to account for the reduced radius of Earth at higher latitudes. The difference between the elliptical altitude and spherical altitude should be maximized at high latitudes, and zero at equatorial latitudes. The period of this oscillation should be the sidereal period of the satellite. The longitude should exhibit a saw-tooth pattern, but the rise is not expected to be linear; the vehicle will be covering longitude more rapidly at high latitudes than near the equator; while its speed is constant, the vehicle is traveling perpendicular to lines of longitude at high latitudes, but not at equatorial latitudes. The period of this oscillation should be the synodic period

of the satellite. The latitude should oscillate between $\pm\frac{\pi}{4}$ with a period equal to the sidereal period.

3. Low Earth Orbit, polar
   - The Cartesian coordinate output from this data should continue the trend seen in going to a 45 degree inclination; the z-axis amplitude should increase to the full 6,800 km, and the day-long oscillation on the x-and y-axes should force their respective amplitudes to vary between 0 and the full 6,800 km, ninety degrees out of phase with each other.
   - The inertial position should should show a similar pattern to that for the equatorial orbit, with the same period.
   - The altitude should show the same pattern as exhibited for the inclined orbit, but with the geodetic altitude reaching a maximum of 21 km above the spherical altitude as the vehicle crosses the poles. The longitude should show a day-long trend, with a transition at every polar crossing of π radians. The latitude should show a linear triangular oscillation as the latitude varies uniformly while the vehicle heading is northerly and southerly at a constant rate.
4. Eccentric Earth Orbit, equatorial
   - The altitude will vary significantly. Because of the difference in speed between apoapsis and periapsis, this variation will not be sinusoidal, but should have a steeper gradient near periapsis. The altitude will oscillate with the sidereal period of the vehicle. The longitude should exhibit a quasi-sawtooth pattern, but will have significant variation in its gradient due to the varying speed and proximity of the vehicle (when the vehicle is closer, at the same speed, it should cover more longitude in any given time step; when the vehicle is closer, it is also moving faster, exacerbating this feature). The longitude oscillation will be at the synodic period of the vehicle, whereas the proximity of the vehicle to the planet oscillates at the sidereal period. Therefore, the "sawteeth" are not expected to be equal, the section of the tooth with a steep gradient (vehicle is close to the planet) will gradually migrate across the tooth edge. The latitude should remain at zero. The altitude should oscillate non-sinusoidally, but with constant amplitude on each axis since the orbit is not rotating with respect to the inertial axes (orbital precession is neglected since the orbit is so close to equatorial)
   - The Cartesian coordinate output from this data should exhibit oscillatory-on-oscillatory behavior, as the x- and y-axes rotate on and off the major axis of the orbit. The oscillation should not be sinusoidal, as observed for the altitude-longitude-latitude plot.
   - The inertial position should exhibit non-sinusoidal oscillatory behavior on all three axes, with each axis having constant amplitude over time. The three axes must not be in phase, and there is no expectation on the relative magnitude of the oscillation from axis to axis.
5. Near-Geosynchronous Orbit.
   - The Cartesian coordinate output from this data should show minimal variation on all three axes; the vehicle should be stationary above a constant point on Earth.
   - The inertial position should show one complete oscillation (simulation ran for 24 hours) on all three axes.

- The altitude, latitude, and longitude should all show minimal variation through the simulation.

Results:

1. Low Earth Orbit, equatorial.
   - The Cartesian coordinate gradually diverge from 0 on the z-axis, but by less than 10 meters over a day. The period of oscillation on the other axes matches that from prediction. See Figure 35.1
   - The inertial position oscillates on all three axes with a period equal to that predicted. See Figure 35.2
   - The altitude remains approximately constant (the variation seen in Figure 35.3 is less than 1 meter). The longitude exhibits the expected saw-tooth appearance with the correct period. The latitude diverges slowly from zero, but only to less than a micro-radian. See Figure 35.3.

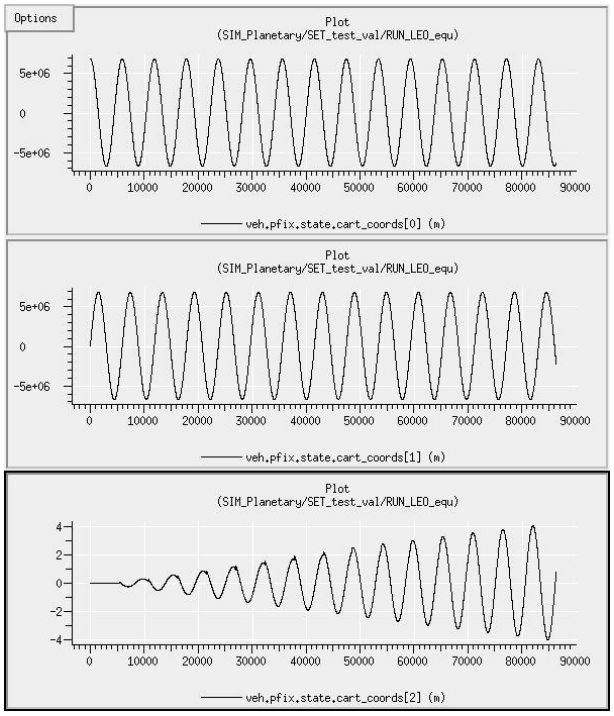


Figure 35.1: The variation of position with time, expressed in Cartesian coordinates.

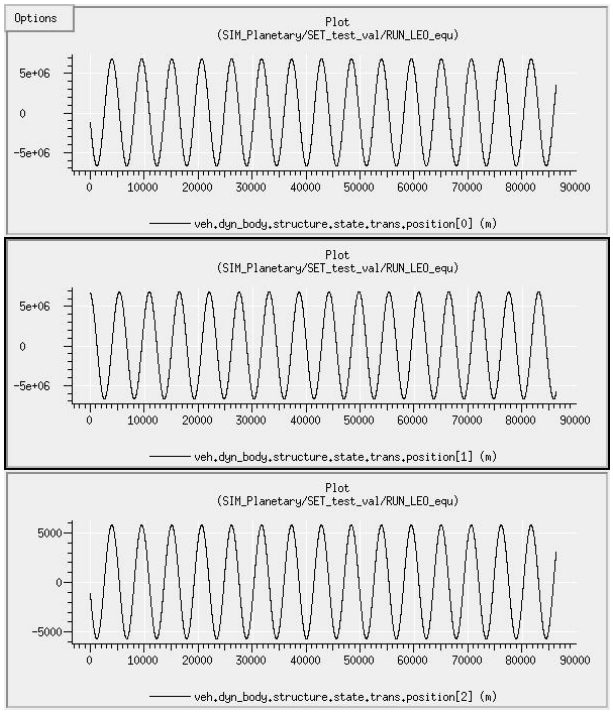


Figure 35.2: The variation of position with time, expressed in inertial coordinates.

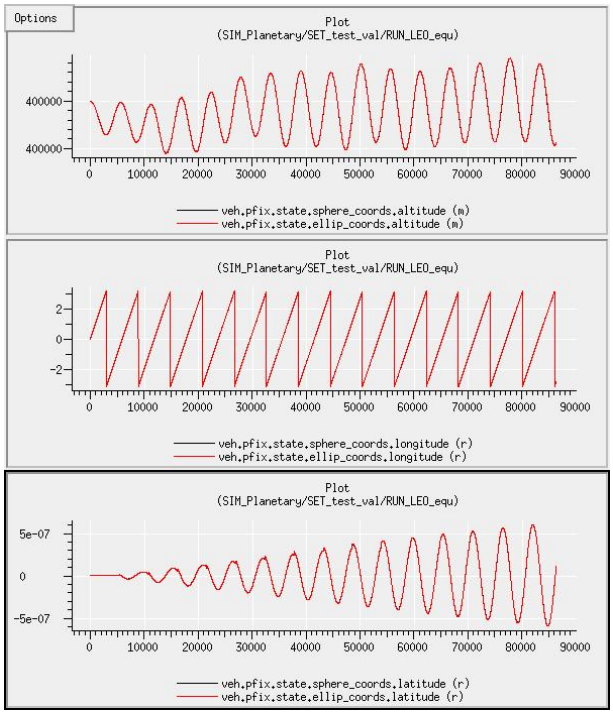


Figure 35.3: The variation of position with time, expressed in Altitude-Longitude-Latitude coordinates, both spherical and elliptical (geodetic).

2. Low Earth Orbit, inclined.
   All data is as expected. See Figures 35.4, 35.5, and 35.6. Figure 35.5 superposes the data from the equatorial low-altitude orbit with that from the inclined low-altitude orbit.

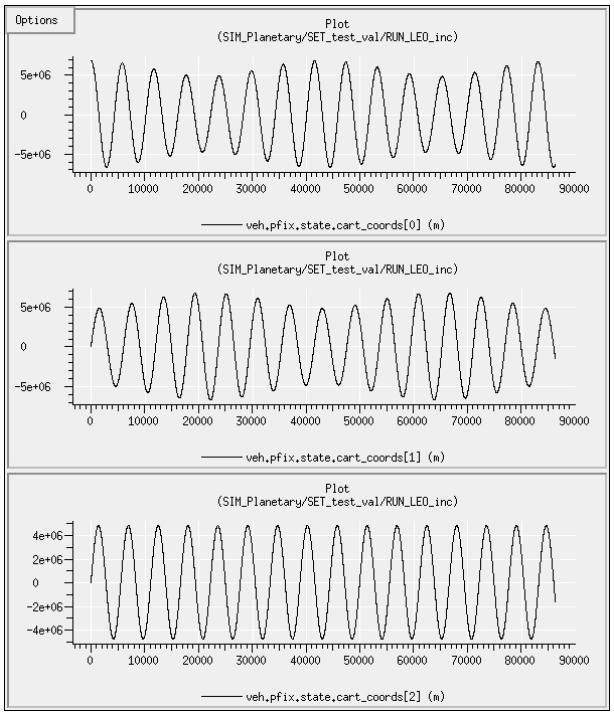


Figure 35.4: The variation of position with time, expressed in Cartesian coordinates.

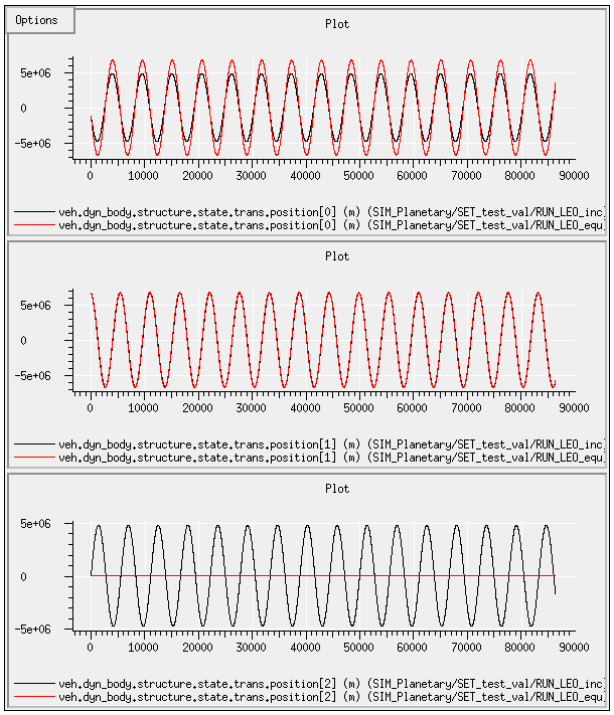


Figure 35.5: The variation of position with time, expressed in inertial coordinates.

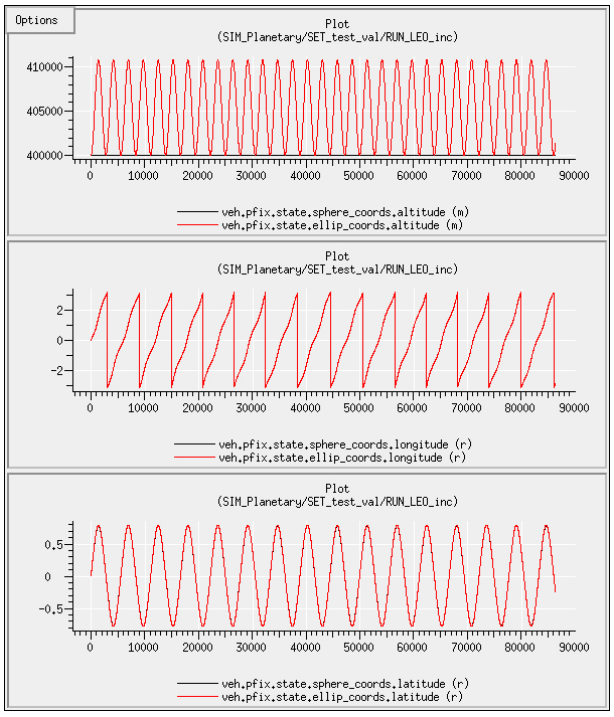


Figure 35.6: The variation of position with time, expressed in Altitude-Longitude-Latitude coordinates, both spherical and elliptical (geodetic).

3. Low Earth Orbit, polar.
   All data is as expected. See Figures 35.7, 35.8, and 35.9. Figure 35.8 superposes the data from the equatorial low-altitude orbit with that from the inclined low-altitude orbit and the low-altitude polar orbit.

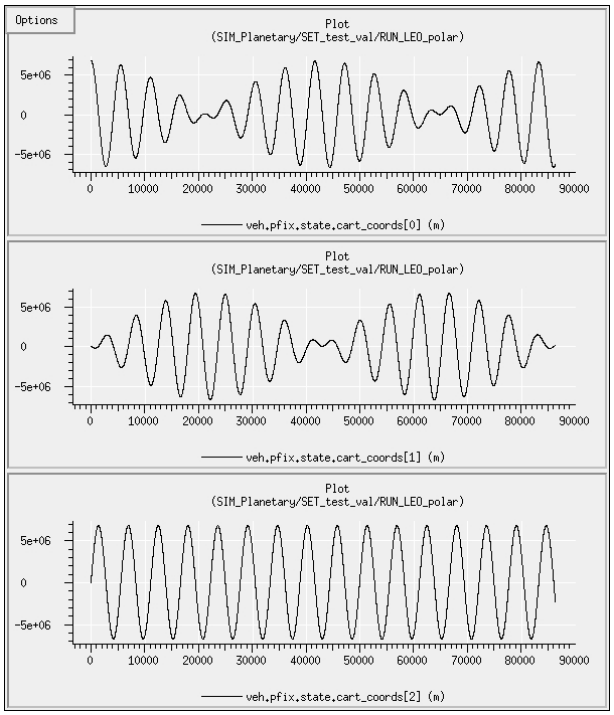


Figure 35.7: The variation of position with time, expressed in Cartesian coordinates.

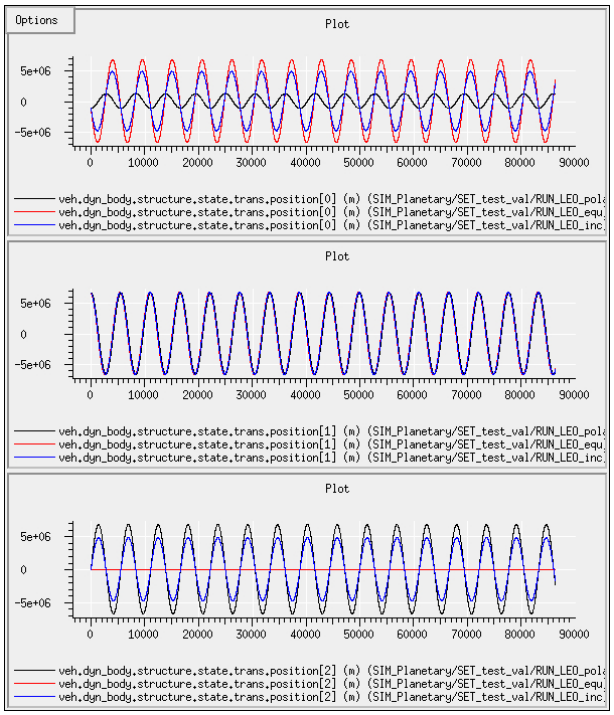


Figure 35.8: The variation of position with time, expressed in inertial coordinates.

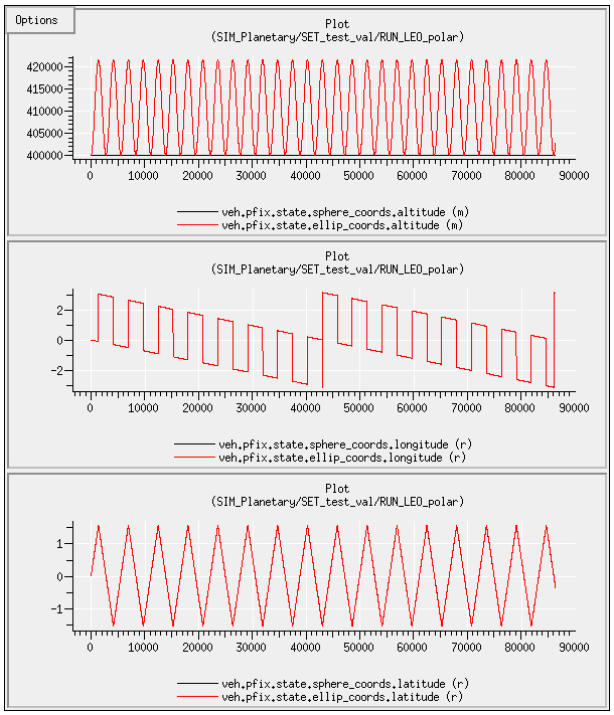


Figure 35.9: The variation of position with time, expressed in Altitude-Longitude-Latitude coordinates, both spherical and elliptical (geodetic).

4. Eccentric Earth Orbit, equatorial.
   All data is as expected. See Figures 35.10, 35.11, and 35.12.

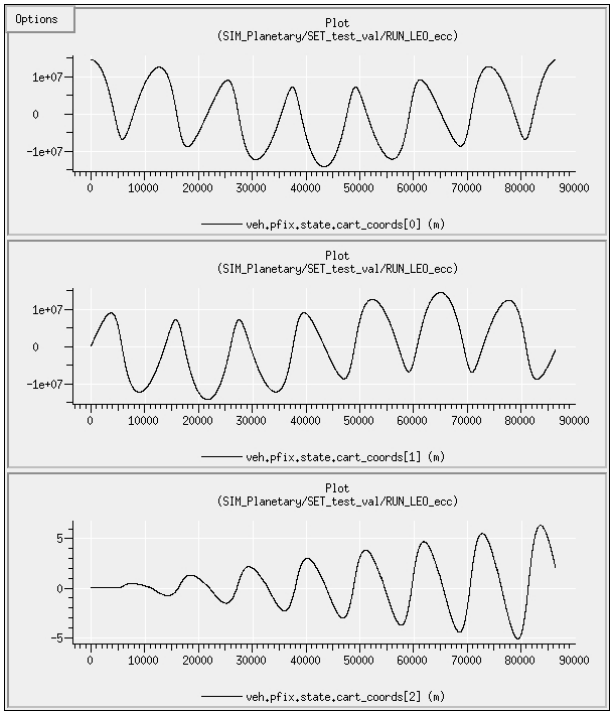


Figure 35.10: The variation of position with time, expressed in Cartesian coordinates.

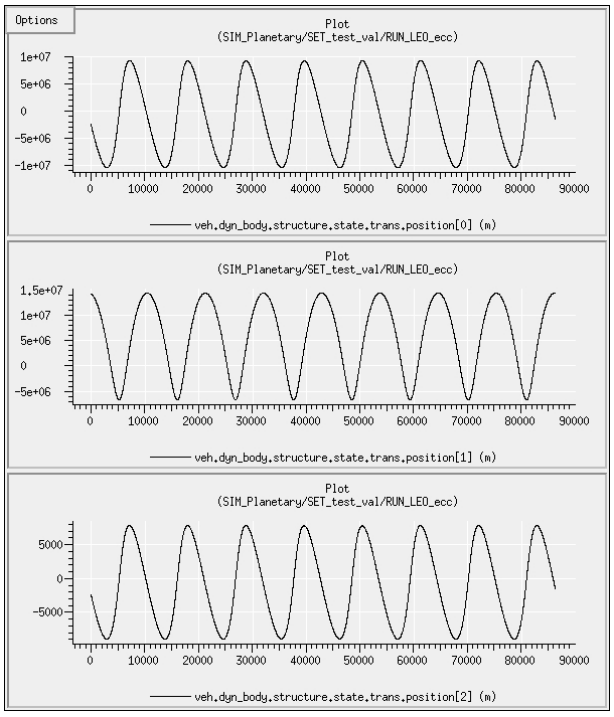


Figure 35.11: The variation of position with time, expressed in inertial coordinates.

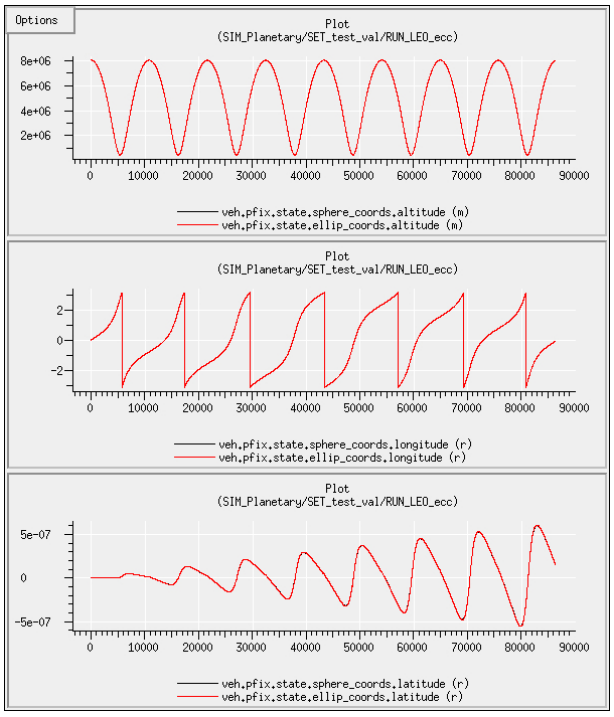


Figure 35.12: The variation of position with time, expressed in Altitude-Longitude-Latitude coordinates, both spherical and elliptical (geodetic).

5. Near-Geosynchronous Orbit.
   All data is as expected. See Figures 35.13, 35.14, and 35.15. Note that the variations on the Cartesian and altitude, longitude, latitude values are generally quite small, with the vehicle advancing very slowly along its orbit; this is most likely a result of an altitude that is not quite appropriate for a geosynchronous orbit.

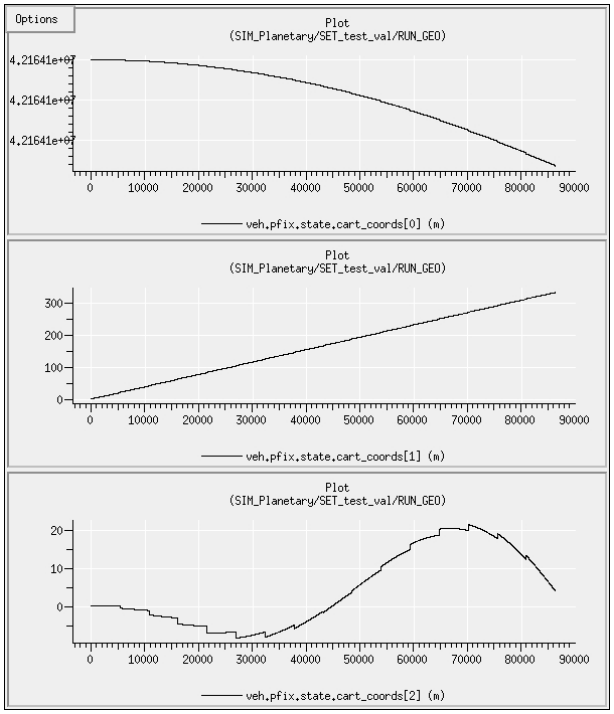


Figure 35.13: The variation of position with time, expressed in Cartesian coordinates.

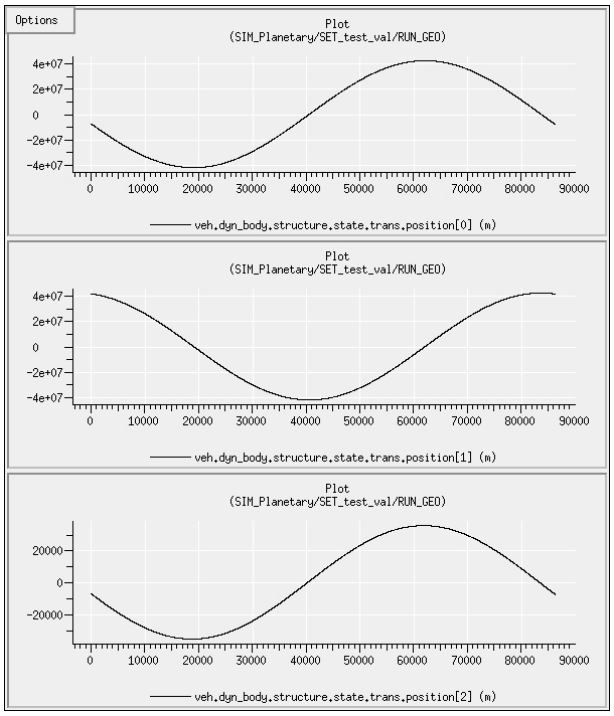


Figure 35.14: The variation of position with time, expressed in inertial coordinates.

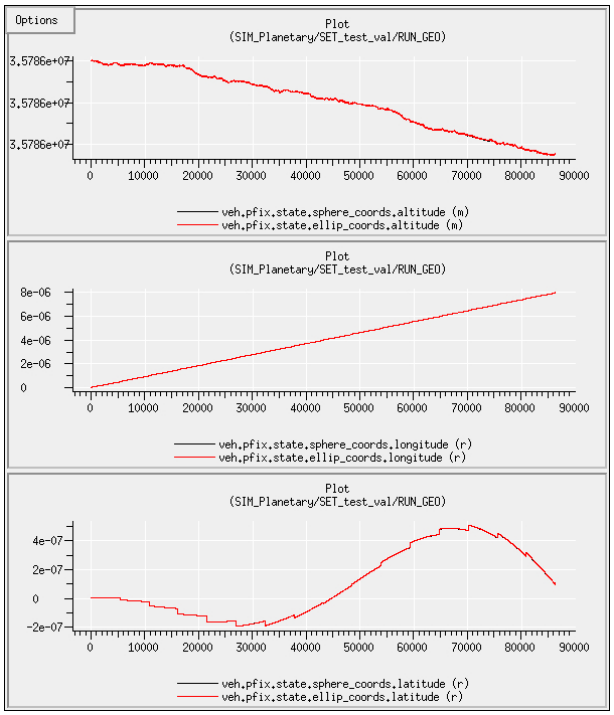


Figure 35.15: The variation of position with time, expressed in Altitude-Longitude-Latitude coordinates, both spherical and elliptical (geodetic).

35.2 Validation

This method has not been validated against external data.

Part VIII

Relative Reference Frame State Representation

Chapter 36

Introduction

36.1 Purpose and Objectives of the Relative Reference Frame State Representation

The Relative Reference Frame state representation allows the state of one reference frame (e.g. the body reference frame of a vehicle) to be expressed relative to any other reference frame (e.g., the LVLH reference frame of another vehicle). As of JEOD version 3.4, it is no longer required that one of the reference frames be associated with a DynBody; the relative state can now be calculated between any two arbitrary frames in the simulation.

Chapter 37

Product Requirements

Requirement DerivedState_11: Relative representation

Requirement:
The Relative Derived State will provide the capability for expressing the state of one reference frame relative to some other reference frame in a specified frame of reference.

Rationale:
Capability from JEOD 1.5.

Verification:
Inspection, unit test, integrated test by proxy.

37.1 Requirements Traceability

Table 37.1: Requirements Traceability

Requirement	Inspection and Testing
DerivedState_11 - Relative State Representation	Insp. DerivedState_7 Test DerivedState_8

Chapter 38

Product Specification

38.1 Conceptual Design

The Relative Reference Frame state representation provides the state of one vehicle with respect to some other reference frame (often associated with another vehicle, but not necessarily so). The two reference frames (the vehicle's reference frame and the other reference frame) are labeled *subject* and *target*. This model is sufficiently flexible to represent the state of either reference frame with respect to the other. As of version 3.4, the *subject* frame no longer must be associated with a Dynamic Body, but can be any arbitrary frame in the simulation. However, the model is fully backwards compatible to the previous usage when it was required to be associated with a Dynamic Body. The *target* never had the Dynamic Body restriction. The original intent was that the subject of the relative state calculation be a vehicle, while the target could be a planet, or some defined point unassociated with any mass, though in actuality the calculation is the same regardless of the nature of the reference frame's owning point or model. The updated capability reflects this.

When expressing the state of some frame, A with respect to some other frame, B, the tranlational position and velocity are expressed in frame B, while the angular velocity is expressed in frame A; the angular position (expressed as a quaternion or transformation matrix) is frame-independent.

As a simple example, consider two reference frames, S and S'. Suppose that S' is moving with some velocity u along the x-axis of reference frame S, and has its origin momentarily located at (x_0, y_0) Consider further that S' is rotated by ninety degrees about the z-axis from S, so that $x' = y, y' = -x, z' = z$. An illustration is shown in Figure 38.1.

There are four ways of expressing this relation: the state of S' can be expressed with respect to S, or S with respect to S'; for each case, the result could be expressed in S or in S'. In this example, the state of S' with respect to S, can be represented as

$$\begin{aligned}
\vec{x}_{S'|S:S} &= (x_0, y_0) \\
\vec{v}_{S'|S:S} &= (u, 0) \\
\vec{x}_{S'|S:S'} &= (y_0, -x_0) \\
\vec{v}_{S'|S:S'} &= (0, -u)
\end{aligned}$$

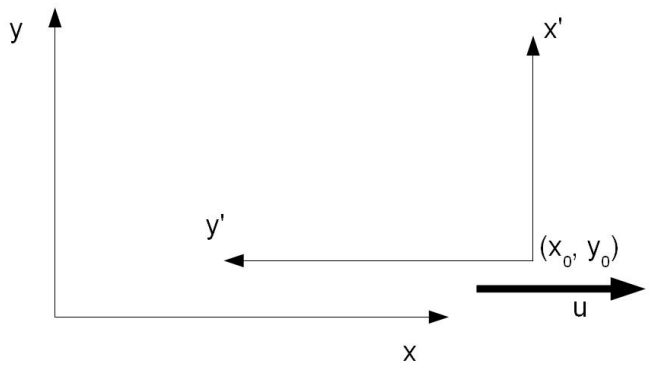


Figure 38.1: The momentary positions, velocity, and orientation of frames S and S'

where $\vec{x}_{A|B:C}$ is the position of A with respect to B, expressed in C.

Similarly, the state of S with respect to S', can be represented as

$$\begin{aligned}\vec{x}_{S_S':S} &= (-x_0, -y_0)\\ \vec{v}_{S_S':S} &= (-u, 0)\\ \vec{x}_{S_S':S'} &= (-y_0, x_0)\\ \vec{v}_{S_S':S'} &= (0, u)\end{aligned}$$

Thus, the relative position (or velocity) could be described in four ways, using just these two frames. Of these four representations, this model can provide $\vec{x}_{S'|S:S}$ and $\vec{x}_{S_S':S'}$.

To make the system completely generic, the usage of *subject* and *target* are, for the most part, just names. The Relative Reference Frame state representation will calculate the state of either with respect to the other. Hence, in the example above, S and S' can be interchangeably identified as *subject* and *target*. Again, the only restriction on assigning the *subject* and *target* is that the *subject* must be associated with a Dynamic Body.

The user must specify three quantities:

- Subject reference frame (by name)
- Target reference frame (by name)
- Sense in which to interpret the request

the latter item in this list can be one of the two values described above:

- Subject with respect to Target, expressed in Target (ComputeSubjectStateinTarget)
- Target with respect to Subject, expressed in Subject (ComputeTargetStateinSubject)

Note that this description is intended to convey information on the translational state only. The angular velocity of the Subject frame will be expressed in the Subject frame, and the angular velocity of the Target frame will be expressed in the target frame. The expression of the relative orientations of the two frames is frame independent.

38.2 Mathematical Formulations

The mathematical formulation for calculating the Relative Reference Frame state representation is entirely incorporated into the Reference Frames model (see *Reference Frames* [10]).

38.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

38.3.1 Process Architecture

The architecture for the Relative Reference Frame state representation is trivial, comprising methods that are operationally independent.

38.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The Relative Reference Frame state representation contains only one class:

- **Class: RelativeDerivedState** **Inheritance:** *DerivedState* (on page 8)

  This contains only the methods *initialize* and *update*:

  1. **initialize**

     The initialization process comprises the following steps:

     (a) The generic DerivedState initialization routine is called to assign the *subject* value, and establish the naming convention for the state identifier. Note - this is based on the *subject* name, not on the *target* name. The user may wish to consider this in determining which object to associate as target, and which to associate as subject.

     (b) Ensures that the *subject_frame* and *target_frame* are well-defined.

     (c) The registry of active bodies, maintained by the Dynamics Manager (see *Dynamics Manager Documentation* [2]) is updated to ensure that the *target_frame* is being updated, if necessary.

  2. **update**

     The variable *direction_sense* can be set to one of two values:

     (a) ComputeSubjectStateinTarget

     (b) ComputeTargetStateinSubject

The appropriate call is made to the appropriate reference frame's *calculate_relative_state* method, using the other reference frame as an argument (e.g., for ComputeSubject-StateinTarget, we are looking for the state of *subject* with respect to *target*, so the subject-frame's method would be called, using the target frame as an argument).

Chapter 39

User's Guide

The Analysis section of the User's Guide is intended primarily for users of pre-existing simulations. It contains:

- A description of how to modify Relative Reference Frame state representation variables after the simulation has compiled, including an in-depth discussion of the input file,
- An overview of how to interpret (but not edit) the S_define file,
- A sample of some of the typical variables that may be logged.

The Integration section of the User's Guide is intended for simulation developers. It describes the necessary configuration of the Relative Reference Frame state representation within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section of the user guide. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 38).

The Extension section of the User's Guide is intended primarily for developers needing to extend the capability of the Relative Reference Frame state representation. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 38).

39.1 Analysis

39.1.1 Identifying the Relative Reference Frame State Representation

If Relative States have been included in the simulation, there will be an instance of *RelativeDerivedState* located in the S_define file. This would typically be found in either the vehicle object, or a separate relative-state object. There should be an accompanying call to both an initialization routine and an update function. The *target_frame* and *subject_frame* are defined elsewhere, possibly in the input file.

Example:

```
sim_object{
dynamics/derived_state:    RelativeDerivedState example_of_rel_der_state;

(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_rel_der_state.initialize (
    Inout DynBody &      subject_body = vehicle_1.dyn_body,
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);

{environment} dynamics/derived_state:
example_of_rel_state_object.example_of_rel_der_state.update ( )

} example_of_rel_state_object;
```

Then the input file may have entries comparable to:

```
example_of_rel_state_object.example_of_rel_der_state.subject_frame_name =
                                                    "vehicle_1.composite_body";
example_of_rel_state_object.example_of_rel_der_state.target_frame_name =
                                                    "vehicle_2.Earth.lvlh";
example_of_rel_state_object.example_of_rel_der_state.direction_sense =
                              RelativeDerivedState::ComputeSubjectStateinTarget;
```

(In this case, the relative state provides the state of *vehicle_1* in the Earth-based LVLH frame associated with *vehicle_2*. Note that there are many variations on this concept, so the analyst should not be surprised if entries are significantly different from this example).

As of JEOD version 3.4, it is no longer required that a relative state be associated with a vehicle object. Thus, it can be used to calculate the state between any two arbitrary frames in the simulation. To use it in this manor, one needs only to invoke the initialization routine that does not take a *subject_body* as input:

```
(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_rel_der_state.initialize (
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);
```

All other usage remains the same.

39.1.2 Editing the Relative Reference Frame State Representation

There is very little to edit in the Relative Reference Frame state representation. Changing the *direction_sense* is trivial (see section *Detailed Design* (on page 128) for options). Changing the *subject_frame* and *target_frame* may produce unintended consequences, and should not be attempted at this level; see the *Integration* (on the following page) for details on setting up a relative state calculation.

39.1.3 Output Data

The output available from this model is a complete 12-vector state description.

The translational component of the state is accessible as

```
example_of_rel_state_object.example_of_rel_der_state.rel_state.trans.position
example_of_rel_state_object.example_of_rel_der_state.rel_state.trans.velocity
```

The rotational state can be output as either quaternion or transformation matrix, with velocity as either an angular velocity vector, or an angular velocity magnitude and unit vector. See the documentation on *RefFrameState* in the RefFrame document [10] for details.

39.2 Integration

Including the Relative Reference Frame state representation into the simulation requires some knowledge of other reference frames and derived states.

39.2.1 Generating the S_define

Conventional practice would add the Relative Reference Frame state representation to a separate relative-state object at the end of the S_define file. It is also possible to add the relative state to a specific vehicle, but care must be taken to ensure that the target frame has been fully updated before the relative state is called. For example, if the relative state is to be between two vehicles, A and B (suppose A is the *subject*, and B the *target*), then the relative state must be generated after object B, and the particular reference frame of interest associated with B, are both updated. A simple solution to this is to either place the relative state calculation at the end of the simulation, or to prioritize the call to update the relative state such that is is processed after that of B. There is no internal check that this ordering is achieved, the responsibility for ensuring this lies entirely on the Integrator.

The instance of *RelativeDerivedState* needs to be defined, the model initialized, and a routine update scheduled. A simple example of how this may look is found in the *Analysis* (on page 130) section. Simple example simulations are found in the verification simulations for the *North-East-Down Reference Frame* (on page 82) and the *Local-Vertical-Local-Horizontal (LVLH) Reference Frame* (on page 53) released with JEOD v5.4. For more complicated setups, where there are multiple relative states defined, it is strongly recommended that all relative states be put together into one relative state object, and grouped under the Relative Kinematics manager. This manager handles all of the relative states together after all of the individual frames have been evaluated, and avoids the need to make a separate call from the S_define for each individual state. See the Relative Kinematics [6] document for details.

39.2.2 Generating the Input File

The input file (or Modified Data file) must contain the names of the *subject_frame* (*subject_frame_name*) and *target_frame* (*target_frame_name*), as well as the *direction_sense* for each instance of a Rela-

tiveDerivedState. The *subject_body* is defined elsewhere, as it is passed into the initialization routine from the S_define. The *subject_body* so passed into the initialization routine must include a reference frame with a name that matches the *subject_frame_name*, or the initialization routine will fail. See the Reference Frames [10] documentation for more details on the convention for naming the reference frames.

See the *Analysis* (on page 130) section for an example of how the input data may appear.

39.2.3 Logging the Data

There are multiple variables available for output, to represent the 12-vector state description. The most commonly used are the relative position and velocity, which are logged by adding the following lines to the list of logging data:

```
"example_of_rel_state_object.example_of_rel_der_state.rel_state.trans.position[0-2]",
"example_of_rel_state_object.example_of_rel_der_state.rel_state.trans.velocity[0-2]",
```

The rotational state has some flexibility in its representation. The angular position can be accessed through a quaternion or transformation matrix representation, e.g.

```
"example_of_rel_state_object.example_of_rel_der_state.rel_state.rot.T_parent_this[0][0-2]",
"example_of_rel_state_object.example_of_rel_der_state.rel_state.rot.T_parent_this[1][0-2]",
"example_of_rel_state_object.example_of_rel_der_state.rel_state.rot.T_parent_this[2][0-2]",
```

and the angular rates can be accessed through the angular velocity vector, or with a magnitude and unit vector representation, e.g.

```
"example_of_rel_state_object.example_of_rel_der_state.rel_state.rot.ang_vel_this[0-2]",
```

39.3 Extension

The Relative Reference Frame state representation is not intended to be extensible at this time. An obvious conceptual extension would be the ability to state the frame used to express the state of frame A with respect to frame B, as opposed to the current capability which dictates the expression frame (e.g. one could wish to express the relative position of frame A with respect to frame B in frame A, or in a third frame, frame C, but neither capability is currently available). However, great care should be taken if this is attempted, due to the nature of the inherent paradigm of the RefFrameState.

The Relative Reference Frame state representation takes advantage of the RefFrameState object to describe the state. It is assumed that the RefFrameState (of frame A relative to frame B) has the following interpretations:

- Position: expresssed in frame B.
- Translational velocity: expresssed in frame B.

- Rotational velocity: expresssed in frame A.

The RefFrameState contains not only those state elements (if it did, the extension would be a trivial transformation), but also provides the methods to manipulate them. Changing the paradigm by transforming the output data values will make the output inconsistent with the class that represents it, and will invalidate those methods. However, the output will still be a RefFrameState, so those invalid methods will still be applicable, however inappropriately; peculiar and unexpected behavior will likely result. So, while the Relative Reference Frame state representation can be easily extended conceptually, the implementation of that extension is far beyond the scope of this document. The capability to provide these extensions is currently under review for future releases.

Chapter 40

Verification and Validation

40.1 Verification

40.1.1 Inspection of Behavior in Integrated Testing

Inspection DerivedState_7: Inspection of LVLH and NED Verification Simulations

The verification of the integrated behavior of the Relative Reference Frame state representation is performed by proxy in the verification of the *North-East-Down Reference Frame* (on page 82) and *Local-Vertical-Local-Horizontal (LVLH) Reference Frame* (on page 53). The performance of the simulations in these verification tests indicates that the Relative Reference Frame state representation is performing as expected at a crude, conceptual level (comparison against analytical results was not performed in this case). This result partially satisfies requirement DerivedState_11. Requirement DerivedState_11 is fully satisfied with the demonstration of performance against analytical results; this demonstration is provided in Test DerivedState_8.

40.1.2 Analytical Verification of Comprehensive Behavior in Unit Testing

Test DerivedState_8: Analytical Verification of Relative Reference Frame State Representation Output Data for Multiple Scenarios

Purpose: To demonstrate that the data output by the Relative Reference Frame state representation provides accurate relative state information in simple, analytical situations.

Requirements: Satisfactory conclusion of this test partially satisfies requirement DerivedState_11. Requirement DerivedState_11 is fully satisfied with the additional demonstration of satisfactory performance within integrated testing; this demonstration is conducted at Inspection DerivedState_7.

Procedure: A simulation was developed containing two vehicles in empty space. Each vehicle contained one vehicle-point. Tests were run with the vehicles initialized in various relative

states, and the relative states between the points were examined. The state outputs were provided as the relative state of:

- A with respect to B, expressed in reference frame B
- B with respect to A, expressed in reference frame A

Notation:

- Vectors are expressed numerically as a set of three values enclosed in square braces ($[x, y, z]$). Subscripts on a vector denote the reference frame in which the values are expressed (i.e. which x,y,z axes are being used). The subscripts used in this section are:
  - I - inertial
  - SA - structural of vehicle A
  - SB - structural of vehicle B
  - PA - point frame of vehicle A
  - PB - point frame of vehicle B
- Position, velocity and angular velocity are symbolically represented by:
  - $\vec{x}$ Position.
  - $\vec{v}$ Translational velocity.
  - $\vec{\omega}$ Rotational velocity.
- Subscripts on the symbolic vector values are to be interpreted as follows (P, Q, and R are used here as representatives of the frames used in this section):
  - $\vec{x}_P$ Position of frame P (reference taken from context).
  - $\vec{x}_{P|Q}$ Position of frame P with respect to frame Q (expression frame taken from context).
  - $\vec{x}_{P|Q:R}$ Position of frame P with respect to frame Q, expressed in frame R.
- Conventionally,
  - $\vec{x}_{P|Q}$ is expressed in frame Q.
  - $\vec{v}_{P|Q}$ is expressed in frame Q.
  - $\vec{\omega}_{P|Q}$ is expressed in frame P.

The vehicles were configured as shown in Figure 40.1. For both vehicles, the respective structure and body frames are equivalent; the body frame will therefore not be mentioned again in this section.

Vehicle B is initially located such that $\vec{x}_{SB|I} = [0, 0, 0]_I$, with frame SB aligned with frame I.

Point B (the vehicle point on Vehicle B) is located at
$\vec{x}_{PB|SB} = [1, 0, 0]_{SB}$ (initial conditions, $\vec{x}_{PB|I} = [1, 0, 0]_I$). Frame PB is initially rotated by -90 degrees about the z-axis with respect to frame SB.

Vehicle A is initially located such that $\vec{x}_{SA|I} = [10, 0, 0]_I$. Its structural reference frame (SA) is oriented with a +90 degree rotation about the z-axis from frame I.

Frame PA, associated with Point A, is located such that $\vec{x}_{PA|SA} = [1, 0, 0]_{SA}$ (initial conditions, $\vec{x}_{PA|I} = [10, 1, 0]_I$). It has a +90 degree rotation about the z-axis with respect to frame SA.

The dynamic states, where applicable, of the two vehicles are initialized such that:

- $\vec{v}_{SA|I} = [2, 1, 0]_I = [-2, -1, 0]_{PA}$
- $\vec{\omega}_{SA|I} = [0, 9, 0]_{SA}\ deg\ s^{-1} = [-9, 0, 0]_I\ deg\ s^{-1} = [9, 0, 0]_P A\ deg\ s^{-1}$
- $\vec{v}_{SB|I} = [0, 0, 1]_I$
- $\vec{\omega}_{SB|I} = [0, 0, 4.5]_I\ deg\ s^{-1}$ (same value in frames I, SB, PB).

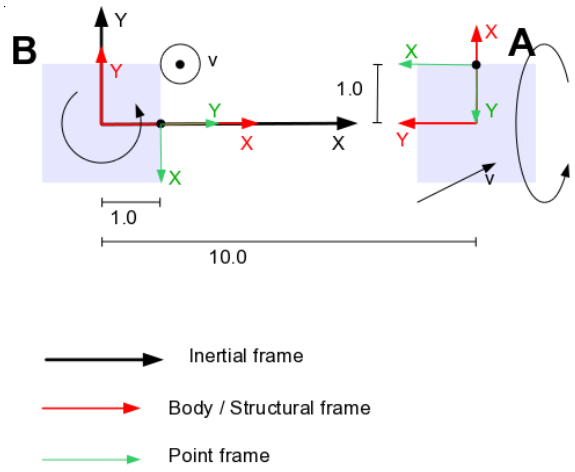


Figure 40.1: The configuration of the two vehicles and their respective reference frames.

Several simulation setups were considered, gradually increasing in complexity.

First, the rotational states were turned off, and only the effects of relative translational motion studied – first one at a time, then both moving together. Then, the translational motion was zeroed, and only the effect of relative rotational motion considered – first one at a time, then both together. Finally, the situation in which both vehicles had translational and rotational motion was considered.

1. RUN_no_rot_A_trans
2. RUN_no_rot_B_trans
3. RUN_no_rot_AB_trans
4. RUN_A_rot_no_trans
5. RUN_B_rot_no_trans
6. RUN_AB_rot_no_trans
7. RUN_AB_rot_AB_trans

Predictions: 1. RUN_no_rot_A_trans

With no rotation, the relative states are straightforward. In this case, only vehicle A is moving.

- $\vec{x}_{PA|PB} = [(-1 - t), (9 + 2t), 0]_{PB}$
- $\vec{v}_{PA|PB} = [-1, 2, 0]_{PB}$
- $\vec{\omega}_{PA|PB} = [0, 0, 0]_{PA}$
- $\vec{x}_{PB|PA} = [(9 + 2t), (1 + t), 0]_{PA}$

- $\vec{v}_{PB|PA} = [2, 1, 0]_{PA}$
- $\vec{\omega}_{PB|PA} = [0, 0, 0]_{PB}$

2. RUN_no_rot_B_trans
   In this run, vehicle A is static, and vehicle B is moving.
   - $\vec{x}_{PA|PB} = [-1, 9, -t]_{PB}$
   - $\vec{v}_{PA|PB} = [0, 0, -1]_{PB}$
   - $\vec{\omega}_{PA|PB} = [0, 0, 0]_{PA}$
   - $\vec{x}_{PB|PA} = [9, 1, t]_{PA}$
   - $\vec{v}_{PB|PA} = [0, 0, 1]_{PA}$
   - $\vec{\omega}_{PB|PA} = [0, 0, 0]_{PB}$
3. RUN_no_rot_AB_trans
   In this run, both vehicles have translational motion.
   - $\vec{x}_{PA|PB} = [(-1 - t), (9 + 2t), -t]_{PB}$
   - $\vec{v}_{PA|PB} = [-1, 2, -1]_{PB}$
   - $\vec{\omega}_{PA|PB} = [0, 0, 0]_{PA}$
   - $\vec{x}_{PB|PA} = [(9 + 2t), (1 + t), t]_{PA}$
   - $\vec{v}_{PB|PA} = [2, 1, 1]_{PA}$
   - $\vec{\omega}_{PB|PA} = [0, 0, 0]_{PB}$
4. RUN_A_rot_no_trans
   In this run, vehicle A is rotating (bit has no translational motion). Vehicle B is static.

   The motion of A with respect to B, expressed in B, is fairly straightforward. On the y-axis, there is no motion. On the x-axis, the motion starts at zero, and increases as A rotates around towards +x, and should follow a sine curve. On the z-axis, the motion starts at a (negative) maximum, and slows, as in a negative cosine curve.

   In frame A, it is quite apparent that the x-value of the position of Point B (i.e. frame B) remains constant, at 9.0. However, the motion of Point B on the y- and z- axes is less obvious. Point B will clearly be moving relative to Point A in the inertial frame, but frame A is rotating at the same time.

   At any given time, the location of Point A with respect to the center of mass of vehicle A is $\vec{x}_{PA|SA} = [0, -1, 0]_{PA}$, since the y- and z-axes rotate with the vehicle. Point B does not move relative to the vehicle center of mass, so must be at $\vec{x}_{PB|PA} = (9, 1, 0)_{PA}$ relative to frame A. In other words, Point B DOES NOT MOVE in frame *PA*.

   This can be illustrated by considering the relative position of *PB* with respect to *PA* in the inertial frame:

$$\begin{aligned} \vec{x}_{PB|PA} &= [1\ ,\ 0\ ,\ 0]_I - [10\ ,\ \cos k_A t\ ,\ -\sin k_A t]_I \\ &= [-9\ ,\ -\cos k_A t\ ,\ \sin k_A t]_I \end{aligned}$$

   where $k_A = \frac{2\pi}{40}\ s^{-1} = 0.157\ s^{-1}$.

   With a rotation rate of 9 degrees per second, the rotation period is 40 seconds; there should be 2.5 rotations during the 100 second simulation).

The transformation matrix from I to PA is a combination of a 180-degree z=rotation to begin, followed by a dynamic rotation on the x-axis:

$$\begin{aligned} T_{I\to PA} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos k_A t & \sin k_A t \\ 0 & -\sin k_A t & \cos k_A t \end{bmatrix} \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} -1 & 0 & 0 \\ 0 & -\cos k_A t & \sin k_A t \\ 0 & \sin k_A t & \cos k_A t \end{bmatrix} \end{aligned} \tag{40.1}$$

Hence, applying the transformation matrix to the inertial expression yields:

$$\begin{aligned} \vec{x}_{PB|PA} &= [9\ ,\ -\cos^2 k_A t - \sin^2 k_A t\ ,\ 0]_{PA} \\ &= [9\ ,\ 1\ ,\ 0]_{PA} \end{aligned}$$

- $\vec{x}_{PA|PB} = [-\cos k_A t\ ,\ 9\ ,\ -\sin k_A t]_{PB}$
- $\vec{v}_{PA|PB} = [k_A \sin k_A t\ ,\ 0\ ,\ -k_A \cos k_A t]_{PB}$
- $\vec{\omega}_{PA|PB} = [9\ ,\ 0\ ,\ 0]_{PA}\ deg\ s^{-1}$
- $\vec{x}_{PB|PA} = [9\ ,\ 1\ ,\ 0]_{PA}$
- $\vec{v}_{PB|PA} = [0\ ,\ 0\ ,\ 0]_{PA}$
- $\vec{\omega}_{PB|PA} = [0\ ,\ 9\ ,\ 0]_{PB}\ deg\ s^{-1}$

5. RUN_B_rot_no_trans

In this scenario, vehicle B is rotating, and vehicle A is static. The rotation axis of the rotating vehicle does not pass through the other vehicle's reference point, so the results are a little different to those from the previous test.

The position and velocity of B with respect to A is easier, since the A frame is static. For position, the x-value will oscillate between 9 and 11, starting at 9; the y-value will oscillate between 0 and 2, starting at 1 and decreasing; the z-value remains at 0 throughout. For velocity: the x-value starts at 0 and increases initially; the y-value starts at a minimum; z remains 0 throughout. The angular velocity is a simple rotation on the z-axis.

The position and velocity of A with respect to B is more challenging. We start by considering the relative position in the inertial frame:

$$\vec{x}_{PA|PB} = [10 - \cos k_B t\ ,\ 1 - \sin k_B t\ ,\ 0]_I$$

where $k_B = \frac{2\pi}{80}\ s^{-1} = 0.0785\ s^{-1}$

The transformation matrix from I to PB is a simple z-axis rotation matrix, since both the initial rotation and the dynamic rotation are on the z-axis:

$$T_{I\to PB} = \begin{bmatrix} \sin k_B t & -\cos k_B t & 0 \\ \cos k_B t & \sin k_B t & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{40.2}$$

Hence,

$$\vec{x}_{PA|PB} = [10 sin k_B t - cos k_B t, 10 cos k_B t + sin k_B t - 1, 0]_{PB}$$

The velocity is the derivative of this expression.

- $\vec{x}_{PA|PB} = [10\sin k_B t - \cos k_B t\ ,\ 10\cos k_B t + \sin k_B t - 1\ ,\ 0]_{PB}$
- $\vec{v}_{PA|PB} = [10k_B\cos k_B t + k_B\sin k_B t\ ,\ k_B\cos k_B t - 10k_B\sin k_B t\ ,\ 0]_{PB}$
- $\vec{\omega}_{PA|PB} = [0\ ,\ 0\ ,\ -4.5]_{PA}\ deg\ s^{-1}$
- $\vec{x}_{PB|PA} = [10 - \cos k_B t\ ,\ 1 - \sin k_B t\ ,\ 0]_{PA}$
- $\vec{v}_{PB|PA} = [k_B\sin k_B t\ ,\ -k_B\cos k_B t\ ,\ 0]_{PA}$
- $\vec{\omega}_{PB|PA} = [0\ ,\ 0\ ,\ 4.5]_{PB}\ deg\ s^{-1}$

6. RUN_AB_rot_no_trans
   Combining the two rotations produces:

$$\begin{aligned}\vec{x}_{PA|PB} &= [10\ ,\ \cos k_A t\ ,\ -\sin k_A t]_I - [\cos k_B t\ ,\ \sin k_B t\ ,\ 0]_I \\ &= [10 - \cos k_B t\ ,\ \cos k_A t - \sin k_B t\ , -\sin k_A t]_I\end{aligned}$$

   and, by symmetry,

$$\vec{x}_{PB|PA} = -\vec{x}_{PA|PB}$$

   Applying the transformation matrices (equations 4 and 5) provides the relative positions in the appropriate frames.
   The translational velocity expressions are derived from the derivatives of the relative positions. The rotational velocity expressions are derived in a similar way to the translational positions.

$$\begin{aligned}\vec{\omega}_{PA|PB} &= [-9\ ,\ 0\ ,\ 0]_I\ deg\ s^{-1} - [0\ ,\ 0\ ,\ 4.5]_I\ deg\ s^{-1} \\ &= [-9\ ,\ 0\ , -4.5]_I\ deg\ s^{-1}\end{aligned}$$

- $\vec{x}_{PA|PB} = \begin{bmatrix} 10\sin k_B t - \cos k_A t\cdot\cos k_B t \\ 10\cos k_B t + \cos k_A t\cdot\sin k_B t - 1 \\ -\sin k_A t \end{bmatrix}_{PB}$
- $\vec{v}_{PA|PB} = \begin{bmatrix} 10k_B\cdot\cos k_B t + k_B\cdot\cos k_A t\cdot\sin k_B t + k_A\cdot\sin k_A t\cdot\cos k_B t \\ -10k_B\cdot\sin k_B t + k_B\cdot\cos k_A t\cdot\cos k_B t - k_A\cdot\sin k_A t\cdot\sin k_B t \\ -k_A\cos k_A t \end{bmatrix}_{PB}$
- $\vec{\omega}_{PA|PB} = \begin{bmatrix} 9 \\ -4.5 sin k_A t \\ -4.5 cos k_A t \end{bmatrix}_{PA}\ deg\ s^{-1}$
- $\vec{x}_{PB|PA} = \begin{bmatrix} 10 - \cos k_B t \\ 1 - \cos k_A t\cdot\sin k_B t \\ sin k_A t\cdot sin k_B t \end{bmatrix}_{PA}$
- $\vec{v}_{PB|PA} = \begin{bmatrix} k_B\sin k_B t \\ k_A\cdot\sin k_A t\cdot\sin k_B t - k_B\cdot\cos k_A t\cdot\cos k_B t \\ k_A\cdot\cos k_A t\cdot\sin k_B t + k_B\cdot\sin k_A t\cdot\cos k_B t \end{bmatrix}_{PA}$
- $\vec{\omega}_{PB|PA} = \begin{bmatrix} 9\sin k_B t \\ 9\cos k_B t \\ 4.5 \end{bmatrix}_{PB}\ deg\ s^{-1}$

7. RUN_AB_rot_AB_trans
   With this run, the previously generated translational motion is added to the rotational motion; the same methods are applied (i.e. generate inertial expressions, transform to appropriate frame) to obtain the following expressions:

- $\vec{x}_{PA|PB} = \begin{bmatrix} 10\sin k_B t - \cos k_A t \cdot \cos k_B t + t(2\sin k_B t - \cos k_B t) \\ 10\cos k_B t + \cos k_A t \cdot \sin k_B t - 1 + t(2\cos k_B t + \sin k_B t \\ -\sin k_A t - t \end{bmatrix}_{PB}$
- $\vec{v}_{PA|PB} = \begin{bmatrix} \cos k_B t(k_B(10+2t) + k_A(\sin k_A t) - 1) + \sin k_B t(k_B(t+\cos k_A t) + 2) \\ \cos k_B t(k_B(t+\cos k_A t) + 2) - \sin k_B t(k_B(10+2t) + k_A(\sin k_A t) - 1) \\ -k_A \cos k_A t - 1 \end{bmatrix}_{PB}$
- $\vec{\omega}_{PA|PB} = \begin{bmatrix} 9 \\ -4.5 sin k_A t \\ -4.5 cos k_A t \end{bmatrix}_{PA} \quad deg\ s^{-1}$
- $\vec{x}_{PB|PA} = \begin{bmatrix} 10 - \cos k_B t + 2t \\ 1 - \cos k_A t \cdot \sin k_B t + t(\cos k_A t + \sin k_A t) \\ sin k_A t \cdot sin k_B t + t(\cos k_A t - \sin k_A t) \end{bmatrix}_{PA}$
- $\vec{v}_{PB|PA} = \begin{bmatrix} k_B \sin k_B t + 2 \\ \sin k_A t(k_A(\sin k_B t - t) + 1) + \cos k_A t(1 + k_A t - k_B cos k_B t) \\ \cos k_A t(k_A(\sin k_B t - t) + 1) - \sin k_A t(1 + k_A t - k_B cos k_B t) \end{bmatrix}_{PA}$
- $\vec{\omega}_{PB|PA} = \begin{bmatrix} 9\sin k_B t \\ 9\cos k_B t \\ 4.5 \end{bmatrix}_{PB} \quad deg\ s^{-1}$

Results: For test cases 1-4, the analytical solution is so simple that a simple inspection demonstrated that the results matched the predictions. Test cases 5-7 required a little more analysis. The results for test cases 6 and 7 are included below. The results for test 5 were very similar.

- Test Case 6
  Figures 40.2-40.5 show the differences between the analytical results and the numerical results for the two relative positions and the relative velocities.
  The results show very close agreement between the two sets of data.
- Test Case 7
  Figures 40.6-40.11 show the differences between the analytical results and the numerical results for the two relative positions, velocities, and angular velocities.
  The results show very close agreement between the two sets of data.

Conclusion: In all cases, the numerical values very closely approximated the analytical values. This test has demonstrated that the Relative Reference Frame state representation is correctly calculating the relative states when used in isolation.

In order to demonstrate that the model works identically regardless of whether or not it is associated with a Dynamic Body, an additional relative state was added to each of the test cases described above. It was identical in every way, in every scenario, to one of the already existing relative states, with the exception that it was initialized independently of any Dynamic Body. The generalized results proved to be a bitwise match to those for the pre-existing analog, and thus the results are not separately presented here.

40.2 Validation

There is no independent validation of the Relative Reference Frame state representation.

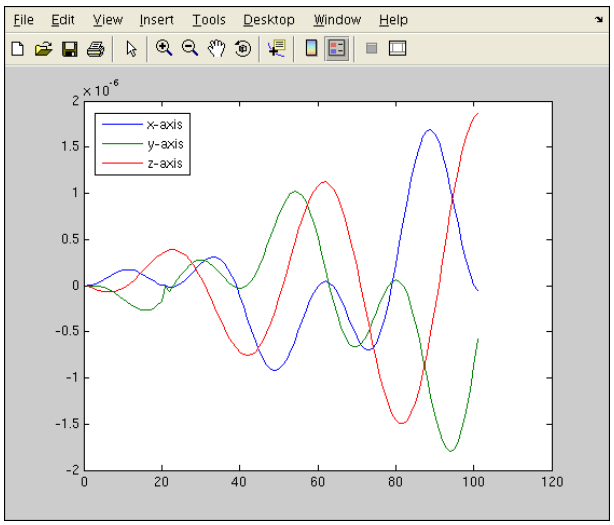


Figure 40.2: The variation with time of the difference between the analytical and numerical solutions for the position of A with respect to B.

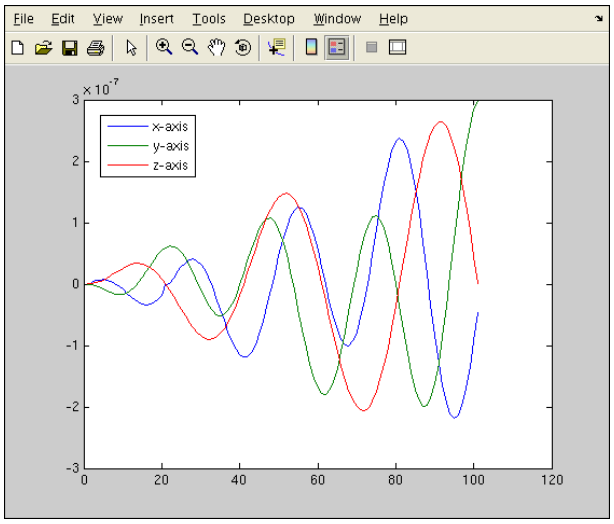


Figure 40.3: The variation with time of the difference between the analytical and numerical solutions for the velocity of A with respect to B.

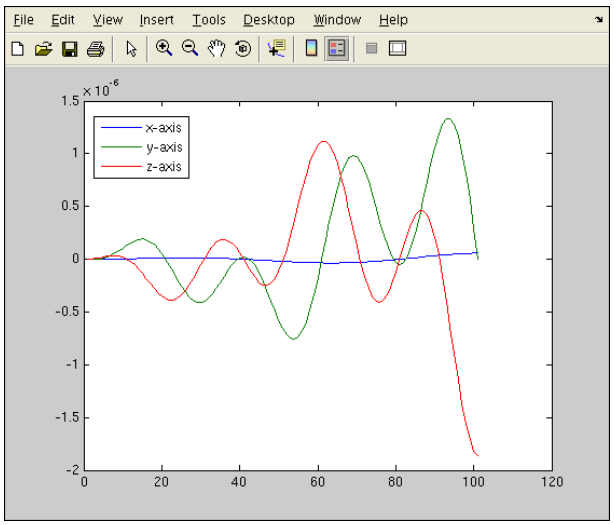


Figure 40.4: The variation with time of the difference between the analytical and numerical solutions for the position of B with respect to A.

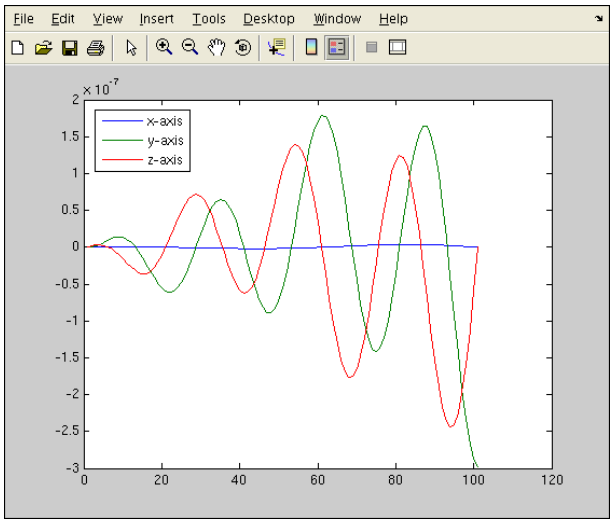


Figure 40.5: The variation with time of the difference between the analytical and numerical solutions for the velocity of B with respect to A.

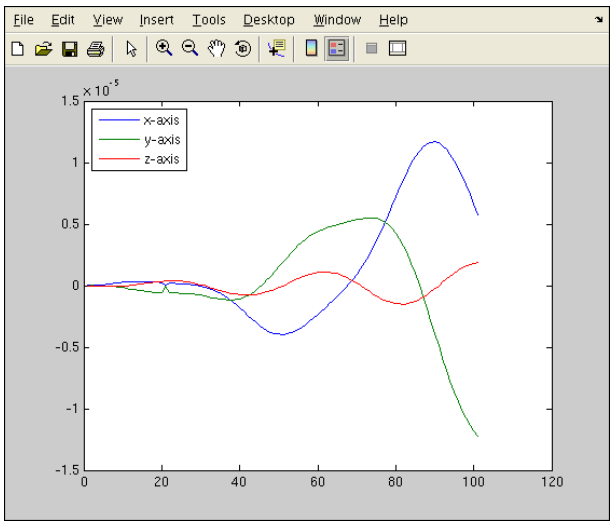


Figure 40.6: The variation with time of the difference between the analytical and numerical solutions for the position of A with respect to B.

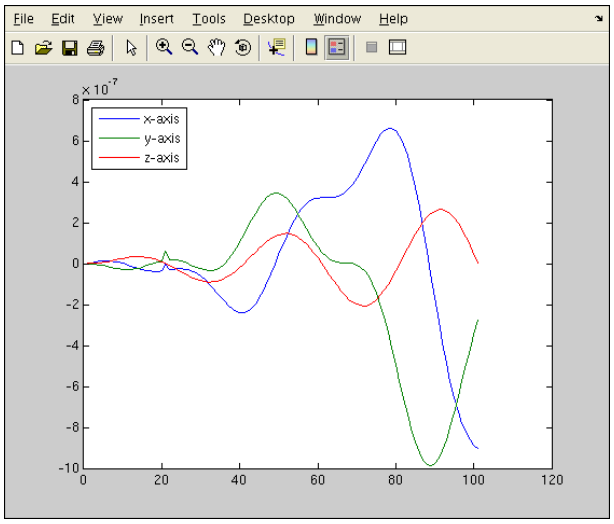


Figure 40.7: The variation with time of the difference between the analytical and numerical solutions for the velocity of A with respect to B.

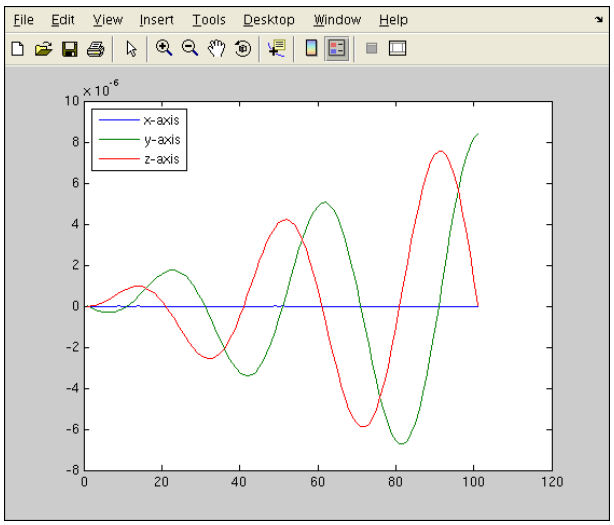


Figure 40.8: The variation with time of the difference between the analytical and numerical solutions for the angular velocity of A with respect to B.

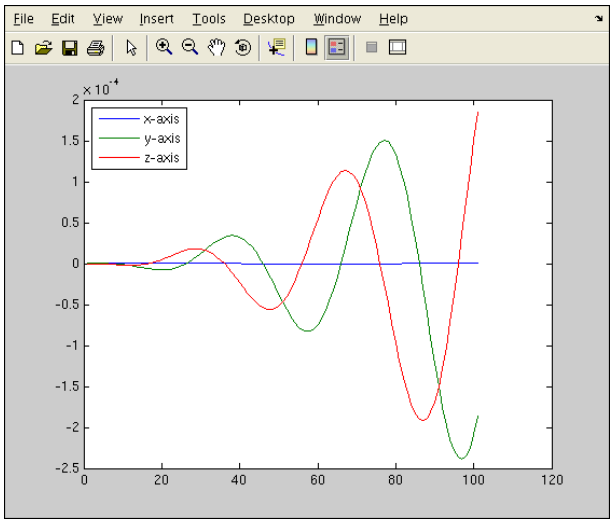


Figure 40.9: The variation with time of the difference between the analytical and numerical solutions for the position of B with respect to A.

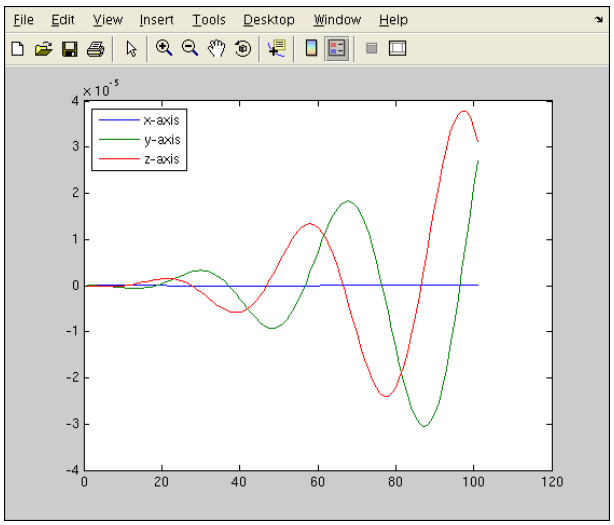


Figure 40.10: The variation with time of the difference between the analytical and numerical solutions for the velocity of B with respect to A.

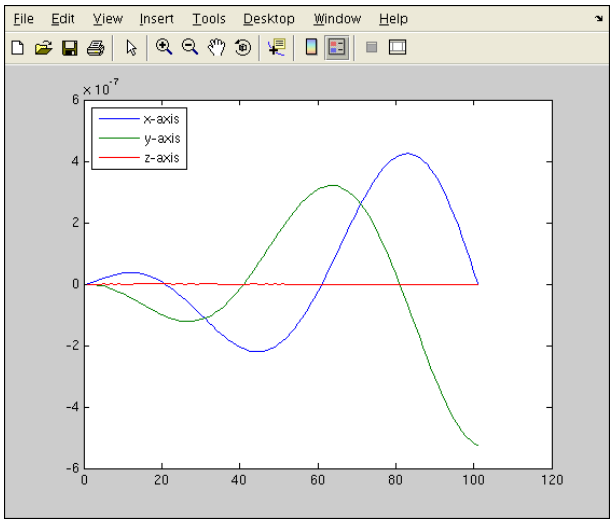


Figure 40.11: The variation with time of the difference between the analytical and numerical solutions for the angular velocity of B with respect to A.

Part IX

Solar Beta Angle Representation

Chapter 41

Introduction

41.1 Purpose and Objectives of the Solar Beta Angle Representation

Spacecraft with movable solar panels and thermal radiators continuously rotate the solar panels to catch as much sunlight as possible and rotate the thermal radiators to catch as little sunlight as possible. The varying orientations of these devices present ever changing cross sections to atmospheric drag. Properly modeling the orientation of these devices improves the fidelity of simulation of such spacecraft.

One of the principal drivers of these panel orientations is the angle between the vector toward the sun and the spacecraft's orbital plane, which is called the *solar beta angle*. The Solar Beta Angle representation represents this angle.

This part of the document describes the Solar Beta Angle representation developed for use with JEOD v5.4.

Chapter 42

Product Requirements

Requirement DerivedState_12: Solar Beta representation

Requirement:
The Solar Beta Derived State will provide the capability for calculating the Solar Beta angle associated with the vehicle's current state.

Rationale:
Capability from JEOD 1.5.

Verification:
Test

42.1 Requirements Traceability

Table 42.1: Requirements Traceability

Requirement	Inspection and Testing
DerivedState_12 - Solar Beta representation	Test DerivedState_9 Test DerivedState_10

Chapter 43

Product Specification

43.1 Conceptual Design

The Solar Beta angle, β, is the angle between the Sun-vector (the vector from the center of Earth to Sun), and the projection of the Sun-vector onto the orbital plane. An alternative view is to consider β as $\pi/2 - \alpha$, where α is the angle between the orbital angular momentum vector and the Sun vector. See Figure 43.1 for a graphical representation.

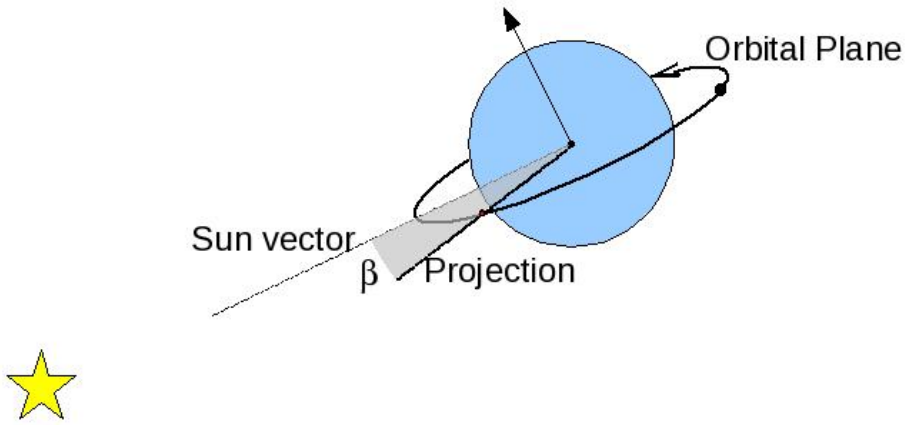


Figure 43.1: The definition of the Solar Beta angle. The arrow represents the orbital angular momentum vector. Note that the position of the vehicle (upper right) on the orbit is of no consequence to the determination of the Solar Beta angle.

β is positive when the Sun-vector is above the orbital plane (i.e., when it is aligned in the same general sense as the orbital angular momentum vector, also interpreted as when the dot product of these two vectors is positive), zero when the Sun-vector lies in the orbital plane, and negative when the Sun-vector lies below the orbital plane. $\beta \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$.

β is affected by orbital precession, and by the orbital motion of Earth with respect to Sun. It is not directly affected by the vehicle's position on its orbit.

As an illustration, consider a vehicle in an equatorial orbit. At the equinoxes, the Sun-vector lies in the orbital plane, and $\beta = 0$. In the Northern summer (April - September), the sun lies above the equator, hence above the orbit (assuming that the orbit is in the same sense as Earth's rotation), and $\beta > 0$, reaching a peak at the solstice. In the Southern summer (October - March), the sun is below the equator, hence below the orbit, and $\beta < 0$, reaching a minimum at the solstice.

The effect of precession and time of year can be seen in Figure 43.2, which shows the typical variation with time of the Solar beta angle for a vehicle in an orbit comparable to that of the International Space Station.

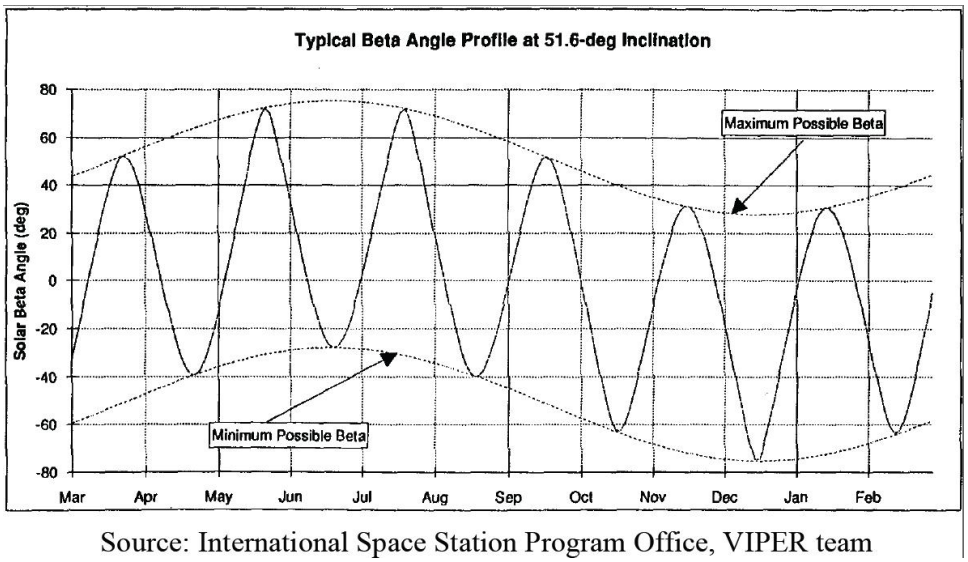


Figure 43.2: The variation with time of the Solar Beta angle for an orbit with inclination of 51.6 degrees. The high frequency oscillation is due to precession, and the long period variation due to Earth's motion around Sun.

43.2 Mathematical Formulations

The mathematical algorithm for computing the Solar Beta angle is straightforward, and builds on tools developed in other models.

While Solar Beta is typically defined (and used) for Earth-orbiting situations, the same concepts can easily be extended to any other planet. Therefore, the code is written in such a way as to make the "host" planet a user-defined quantity. Throughout this description, we will refer to *planet*; for most applications, this can be read as *Earth*.

First, the position of *sun* with respect to *planet* is determined using the *compute_position_from* method from the *Planet* model (see the *Planet model documentation* [7] for details), and the state of the vehicle with respect to *planet* is determined using the *compute_relative_state* method from the *DynBody* model (see the *Dynamics Body documentation* [3] for details). Both relative positions are represented in the inertial reference frame (origin is not relevant since these are relative positions).

Denote $\vec{r}_{A,B}$ as the position of object A with respect to B.

The specific angular momentum of the vehicle due to its orbital motion about the specified planetary body is calculated from a straightforward vector (cross) product of position with velocity. Since the state is recorded in the inertial reference frame centered on the planet, the angular momentum will also be expressed in the inertial reference frame (origin is not relevant). Note, however, that while the definition of Solar Beta breaks down when the vehicle is not in orbit about its specified planet, this calculation continues to be completely valid. Verifying that the vehicle is in orbit would be computationally expensive, and this verification IS NOT carried out. It is left to the user to ensure that the vehicle is, in fact, in orbit about the specified planet

$$\vec{h} = \vec{r}_{veh,planet} \times \vec{v}_{veh,planet}$$

Next, the scalar product of the specific angular momentum and sun-planet vectors is taken, and manipulated to provide β.

$$\vec{h} \cdot \vec{r}_{sun,planet} = \left|\vec{h}\right| |\vec{r}_{sun,planet}| \, cos\alpha = \left|\vec{h}\right| |\vec{r}_{sun,planet}| \, sin\beta$$

$$\beta = sin^{-1}\left(\frac{\vec{h} \cdot \vec{r}_{sun,planet}}{\left|\vec{h}\right| |\vec{r}_{sun,planet}|}\right) \tag{43.1}$$

43.3 Detailed Design

See the Reference Manual[1] for a summary of member data and member methods for all classes.

43.3.1 Process Architecture

The process architecture for the Solar Beta Angle representation is trivial, comprising operationally independent methods.

43.3.2 Functional Design

This section describes the functional operation of the methods in each class.

The Solar Beta Angle representation contains only one class:

- **Class: SolarBetaDerivedState** **Inheritance:** *DerivedState* (on page 8)

  This contains only the methods *initialize* and *update*:

  1. **initialize**

     The initialization process comprises the following steps:

     (a) The generic DerivedState initialization routine is called to establish the naming convention of the reference object (i.e., the planet about which the vehicle is orbiting) and state identifier.

(b) The registry of active bodies, maintained by the Dynamics Manager (see *Dynamics Manager Documentation* [2]) is updated to ensure that the ephemeris of the sun and the reference body are both being updated.

2. **update**

   The process at update is described entirely in the *Mathematical Formulations* (on page 151) section.

Chapter 44

User's Guide

The Analysis section of the User's Guide is intended primarily for users of pre-existing simulations. It contains:

- A description of how to modify Solar Beta Angle representation variables after the simulation has compiled, including an in-depth discussion of the input file,
- An overview of how to interpret (but not edit) the S_define file,
- A sample of some of the typical variables that may be logged.

The Integration section of the User's Guide is intended for simulation developers. It describes the necessary configuration of the Solar Beta Angle representation within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section of the user guide. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 43).

The Extension section of the User's Guide is intended primarily for developers needing to extend the capability of the Solar Beta Angle representation. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 43).

44.1 Analysis

44.1.1 Identifying the Solar Beta Angle Representation

If Solar Beta has been included in the simulation, there will be an instance of *SolarBetaDerivedState* located in the S_define file. This would typically be found in either the vehicle object, or a separate relative-state object. There should be an accompanying call to an initialization routine, which takes a reference to the vehicle as one if its inputs. The reference body is defined elsewhere, possibly in the input file.

Example:

```
sim_object{
dynamics/derived_state:    SolarBetaDerivedState example_of_solar_beta;

(initialization) dynamics/derived_state:
example_of_rel_state_object.example_of_solar_beta.initialize (
    Inout DynBody &      subject_body = vehicle_object.dyn_body,
    Inout DynManager &   dyn_manager  = manager_object.dyn_manager);

{environment}
example_of_rel_state_object.example_of_solar_beta.update ( )

} example_of_rel_state_object;
```

Then the input file may have a line comparable to:

```
example_of_rel_state_object.example_of_solar_beta.reference_name = "Earth";
```

44.1.2 Editing the Solar Beta Angle Representation

There is very little to edit in the Solar Beta Angle representation. The *reference_name* should match the planet around which the vehicle is orbiting; if it does not, that may be changed.

There is also an *active* flag that can be set to turn on or off the calculation of the Solar Beta angle. If the Solar Beta Angle representation appears in the simulation, but is not outputting time-varying data (and remember, this data will vary very slowly because it depends on the precession of the orbit, and on the position of the reference planet with respect to Sun), then it may be that the model is inactive.

44.1.3 Output Data

The only data intended for usable output is the variable *solar_beta*, the Solar Beta angle.

44.2 Integration

Including the Solar Beta Angle representation into the simulation is very straightforward.

44.2.1 Generating the S_define

Conventional practice would add the Solar Beta Angle representation to a specific vehicle, or, if there are multiple relative states between different vehicles such that a relative-state object becomes desirable, it could be added into that relative-state object.

The instance of *SolarBetaDerivedState* needs to be defined, the model initialized, and a routine update scheduled. An example of how this may look is found in the *Analysis* section (44.1).

Examples of how to include the Solar Beta within the vehicle object are found in the Solar Beta Verification simulations, released with JEOD v5.4.

44.2.2 Generating the Input File

The only value necessary to define in the input file (or in Modified Data) is the name of the reference object. The reference object MUST BE the planet about which the vehicle is orbiting. The calculations determine the orbit based on the angular momentum of the vehicle about the reference object. If the vehicle is orbiting a different object, the angular momentum will still be calculated based on the reference object, which will produce results not at all relevant to the problem.

The example given in the *Analysis* (on page 154) section is

```
example_of_rel_state_object.example_of_solar_beta.reference_name = "Earth";
```

44.2.3 Logging the Data

There is only one variable for output, that is the Solar Beta angle itself. Continuing with the example in the *Analysis* (on page 154) section, log

```
example_of_rel_state_object.example_of_solar_beta.solar_beta.
```

44.3 Extension

It is not anticipated that the Solar Beta Angle representation be extended; the angle is independent of the vehicle or the position of the vehicle on its orbit. Extension beyond Earth orbit is already included; it is as easy to compute Solar Beta angles with respect to any other planetary body (for example, for Lunar orbiting vehicles) as it is for Earth.

The other feasible extension would be to replace Sun with some other planetary body. This could be desirable for obtaining orientation data when pointing toward other planetary objects is important, but it would no longer be a "Solar" Beta angle. To implement this extension, copy the Solar Beta Angle representation to generate a new model, rename the variable *sun* to something more suitable, and change the value *"Sun"* in the *initialize* method to the name of the target object.

Chapter 45

Verification and Validation

45.1 Verification

Given the simplicity of the model, a single simulation can be used for all of the verification and validation tests. The initialization of some aspects, such as solar and lunary gravity, is redundant for some tests, but having one simulation simplifies that validation greatly. The simulation uses the DE405 ephemeris model to compute planetary ephemeris and simulates an orbital body as a "falling rock" orbiting the Earth for up to a year.

Test DerivedState_9: Long-Term Behavior

Purpose:
The purpose of this test is to verify the solar beta angle characteristics and accuracy over a period of time.

Requirements
The succesful outcome of this test, together with the validation Test DerivedState_10 satisfy requirement DerivedState_12.

Procedure:
The simulation is run for one year with different orbital inclinations and gravity modeled as spherical or 8x8, with solar and lunar gravitational perturbations on and off.

Predictions:

1. Equatorial orbit, spherical gravity
   The Solar Beta will show a single cycle of variation associated with the orbit (which has a fixed-orientation in an inertial frame) passing once around the sun. The magnitude of that oscillation should be equal to the tilt of Earth's axis with respect to the ecliptic, 23.44°
2. Inclined orbit, 23.4°, non-spherical gravity
   The Solar Beta will oscillate more rapidly as the orbital plane precesses in the non-spherical gravitational field. This should cause an additional oscillation of 23.4° due

to the orbital inclination *on top of* the annual variation due to the orientation of the equator as expected from the previous case.

3. Inclined orbit, 51.6°, non-spherical gravity
   This case should be comparable to the previous case, with a longer-term precession, and larger oscillation about the equatorial oscillation seen in the first case.
4. Inclined orbit, 23.4°, spherical gravity, lunar and solar gravitational perturbation on
   Several cases were run for comparison
   (a) Inclined orbit, 23.4°, with spherical gravity.
   (b) Inclined orbit, 23.4°, with spherical gravity, and the addition of lunar gravitational perturbations.
   (c) Inclined orbit, 23.4°, with spherical gravity, and the addition of lunar and solar gravitational perturbations.

   All of these should show an annual variation, with small oscillations induced by the gravitational perturbations to the orbit.

Results:

1. Equatorial orbit, spherical gravity
   Data is as expected, see Figure 45.1 for comparison of cases 1, 2, and 3.
2. Inclined orbit, 23.4°, non-spherical gravity
   Data is as expected, see Figure 45.1 for comparison of cases 1, 2, and 3. The figure shows an appropriately scaled short-period oscillation superposed on the long-period (annual) oscillation.
3. Inclined orbit, 51.6°, non-spherical gravity
   Data is as expected, see Figure 45.1 for comparison of cases 1, 2, and 3. The figure shows an appropriately scaled short-period oscillation superposed on the long-period (annual) oscillation. The short-period oscillation has larger period than for the less inclined orbit. Similar runs for orbits inclined at 70, 85, and 90 degrees continued the trend, with more pronounced variation in the Solar beta angle, and longer duration precession, gradually slowing until no orbital precession was evident at 90 degrees.
4. Gravitational Perturbations
   The effect of graviational perturbations on the orientation of the orbit are small; the addition of lunar perturbations has more effect than the addition of solar perturbations.

45.2 Validation

Test DerivedState_10: Comparison with External Data

Purpose:
The purpose of this test is to compare solar beta angles calculated by this model with externally-provided values.

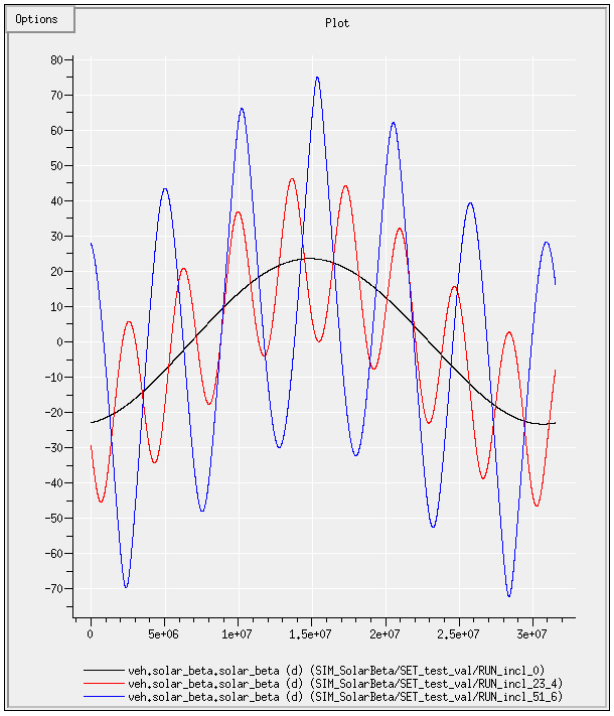


Figure 45.1: The variation of Solar Beta with time for the first three test cases. These are all circular orbits with orbital inclinations of 0, 23.4, and 51.6 degrees respectively

Requirements:

The succesful outcome of this test, together with the validation Test DerivedState_9 satisfy requirement DerivedState_12.

Procedure:

Several tests are carried out, including:

1. RUN_comp_ISS
   This run compares against the Solar Beta for ISS, using a simple ephemeris model. The reference data from The Flight Operation Directorate at NASA JSC provides a solar beta value for a given ISS state (inertial position/velocity) at a given date. The underlying program uses a simple ephemeris model, resulting in solar beta angles accurate to within 1 degree. The vehicle state and time as portrayed in Figure 45.2 were used to initialize the test simulation and to validate the results.
2. RUN_comp_STS121a and RUN_comp_STS121b] STS-121
   These runs compare against the Solar Beta for ISS, generated using the DE405 ephemeris The Mission Operations Directorate uses the DE405 ephemeris to compute solar beta angles in flight-certified applications. Mission STS-121 data, provided by William H. Tracy/NASA/JSC/DM32, provided solar beta values for a pair of given STS-121 states. The vehicle state and time as portrayed in figures 45.3 and 45.4 were used to initialize the test simulation and to validate the results.

Predictions:

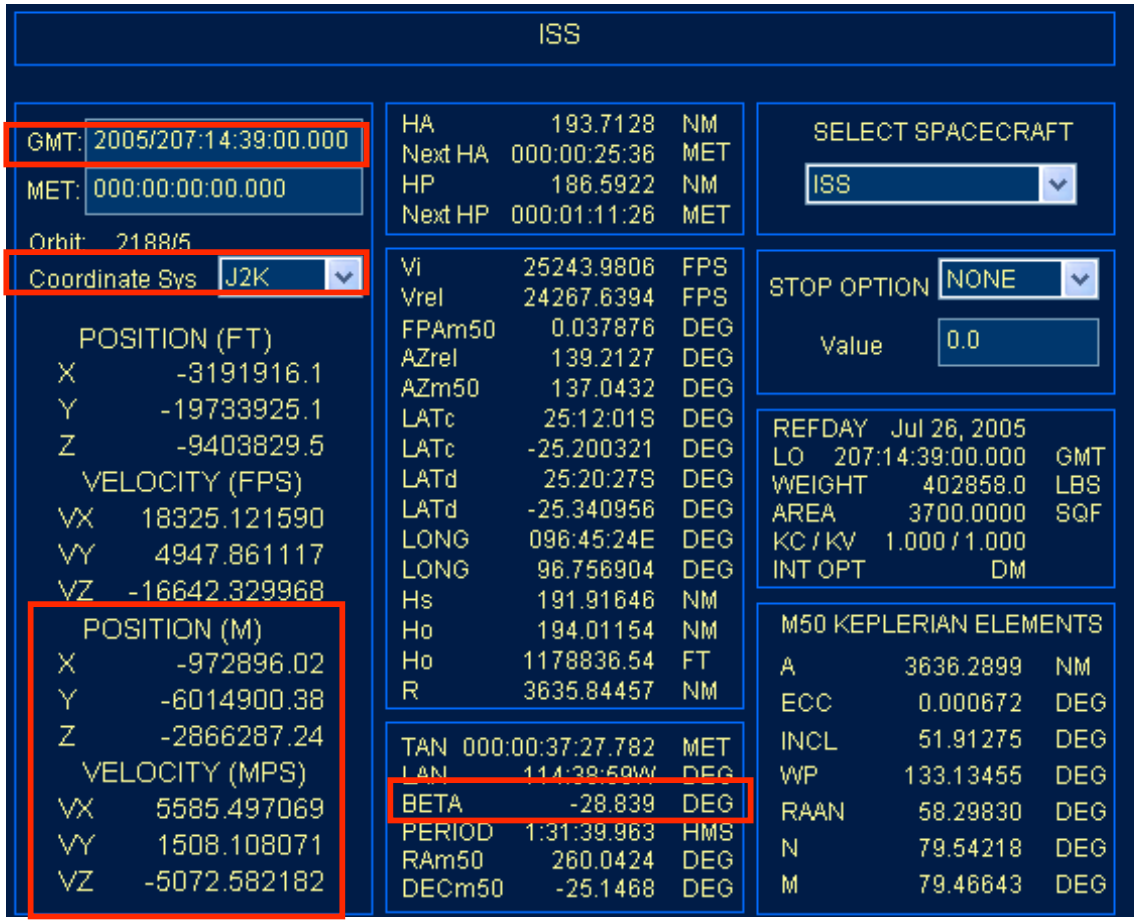


Figure 45.2: Mission Operations ISS State and Solar Beta

1. RUN_comp_ISS
   The Solar Beta angles generated by the simple ephemeris model are accurate to within one degree compared to angles computed with the aid of the highly accurate DE405 ephemeris model.

   Therefore, the solar beta angle computed using the Solar Beta Angle representation should differ with the externally-provided value by no more than one degree for this test case.

2. RUN_comp_STS121a and RUN_comp_STS121b
   Solar beta angles generated externally using the DE405 ephemeris model should agree with angles computed by the Solar Beta Angle representation to a very high degree. The validation data for these tests provide initial conditions in the M50 reference frame, whereas JEOD v5.4 uses the J2000 frame. A conversion of the specification state from M50 to J2000 serves as the initialization data for these tests. This incorporates a potential source of error. Other sources of error are in the state vector and the epoch time. The externally-provided data give solar beta angles with three decimal places of precision (that is, $1/1000^{th}$ degree). The initial state and Solar Beta angle for these two tests are shown in Figures 45.3 and 45.4.

   Even accounting for the potential errors introduced by the precision in the specification of the epoch time, and of the M50 state, and further, its conversion to J2000 conversion, the solar beta angle computed using the Solar Beta Angle representation should differ with the externally-provided value by no more than $0.001°$ for these test cases.

Results:

1. RUN_comp_ISS
   The solar beta angle calculated by the simulation at initialization time was $-28.41°$, which, to within $1°$, agrees with the externally- supplied value of $-28.839°$.

2. RUN_comp_STS121a and RUN_comp_STS121b
   The solar beta angles calculated by the simulation at initialization time are $5.29269°$ (RUN_comp_STS121a) and $3.85718°$ (RUN_comp_STS121b), which, to $0.001°$, compare exactly the externally-supplied values of $5.293°$ and $3.857°$

All test cases pass their respective test criterion.

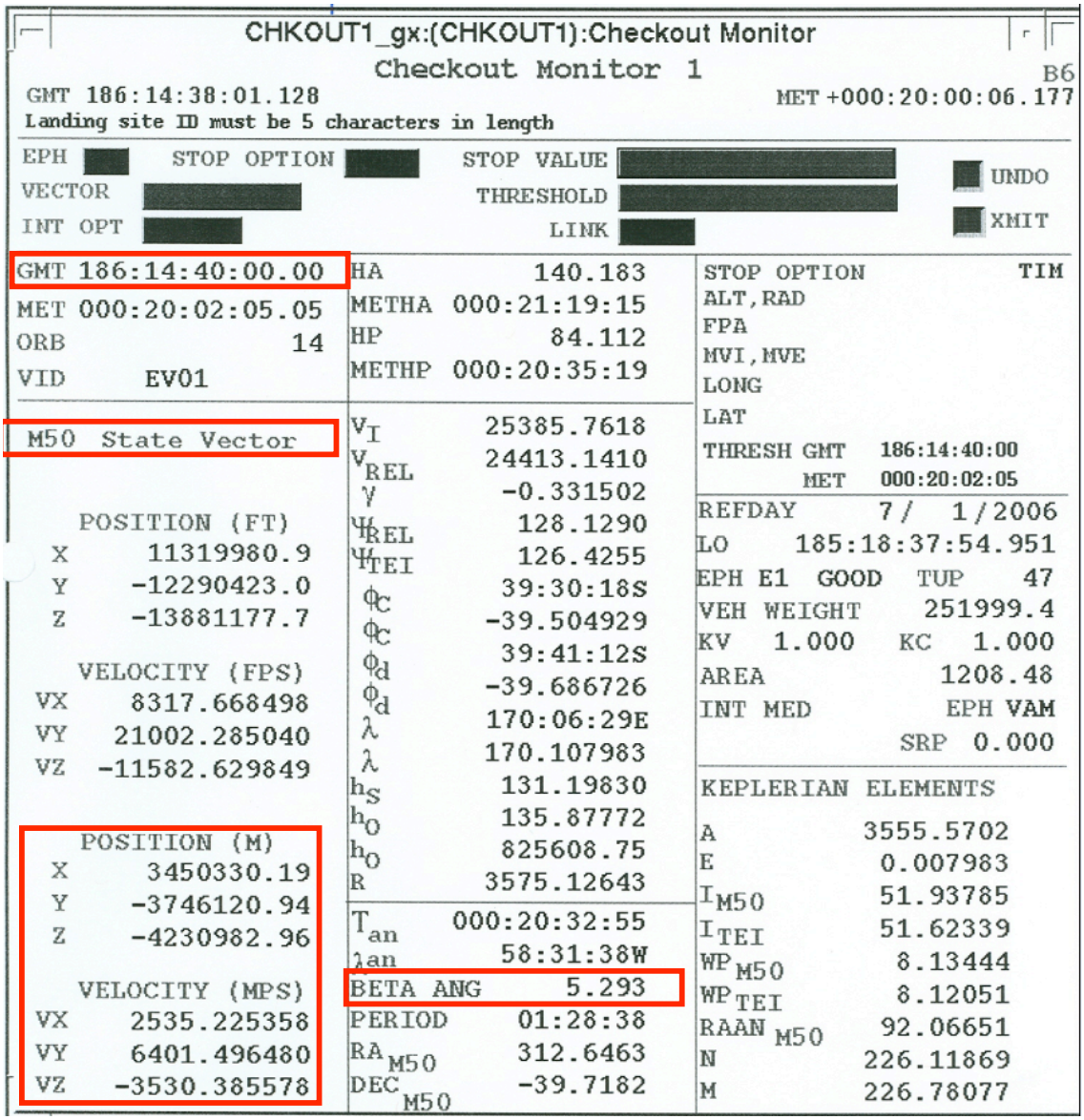


Figure 45.3: Mission Operations STS-121 Checkout Monitor Data

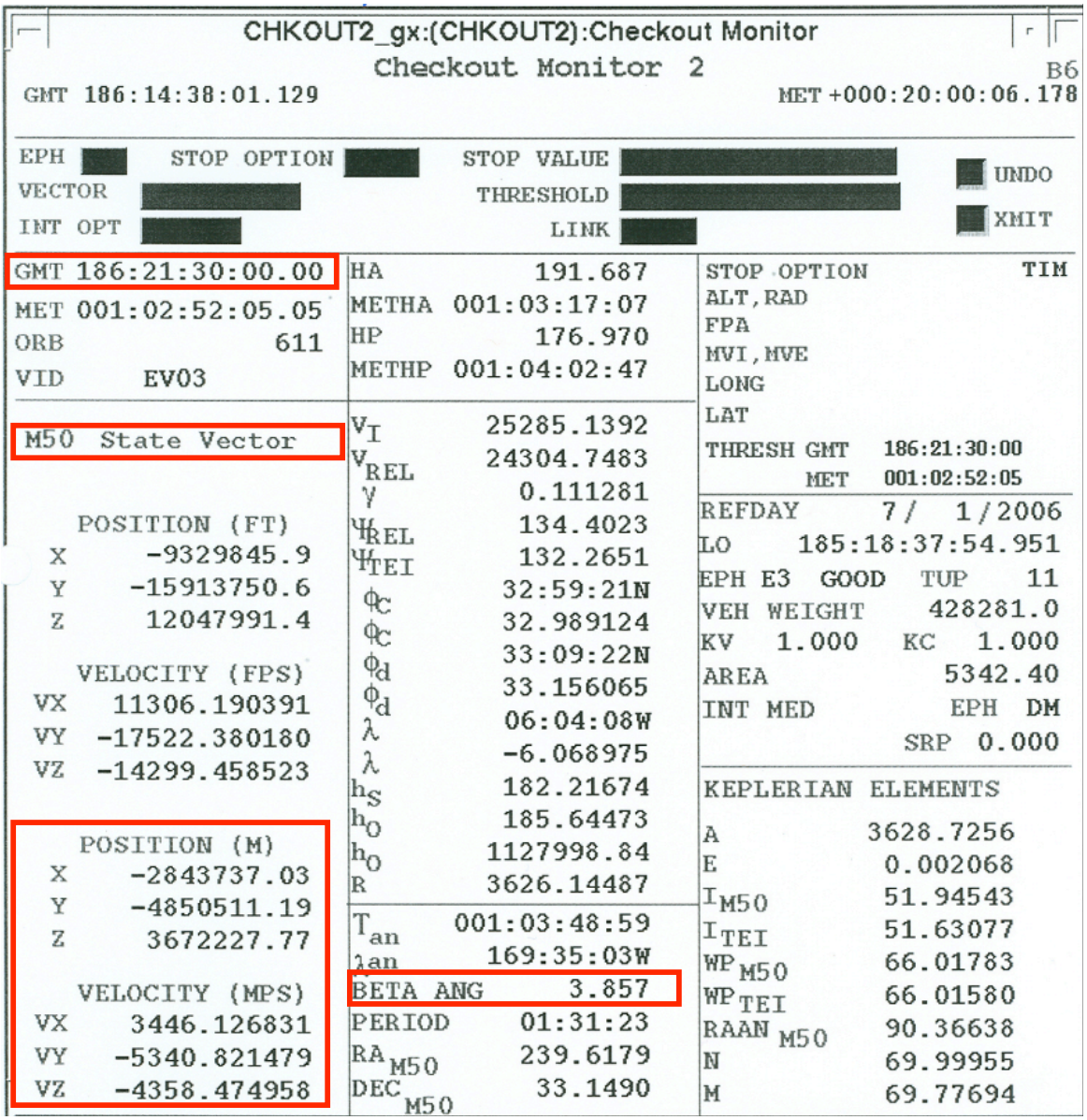


Figure 45.4: Mission Operations STS-121 Checkout Monitor Data

Bibliography

[1] Generated by doxygen. *Derived State Reference Manual*. National Aeronautics and Space Administration, Johnson Space Center, Software, Robotics & Simulation Division, Simulation and Graphics Branch, 2101 NASA Parkway, Houston, Texas, 77058, January 2026.

[2] Hammen, D. Dynamics Manager Model. Technical Report JSC-61777-dynamics/dyn_manager, NASA, Johnson Space Center, Houston, Texas, January 2026.

[3] Hammen, D. Dynamic Body Model. Technical Report JSC-61777-dynamics/dyn_body, NASA, Johnson Space Center, Houston, Texas, January 2026.

[4] Jackson, A. Planet Fixed Model. Technical Report JSC-61777-utils/planet_fixed, NASA, Johnson Space Center, Houston, Texas, January 2026.

[5] Jackson, A., Thebeau, C. JSC Engineering Orbital Dynamics. Technical Report JSC-61777-docs, NASA, Johnson Space Center, Houston, Texas, January 2026.

[6] Morris, J. Relative Kinematics Computations Model. Technical Report JSC-61777-dynamics/rel_kin, NASA, Johnson Space Center, Houston, Texas, January 2026.

[7] Morris, J. Planet Model. Technical Report JSC-61777-environment/planet, NASA, Johnson Space Center, Houston, Texas, January 2026.

[8] NASA. NASA Software Engineering Requirements. Technical Report NPR-7150.2, NASA, NASA Headquarters, Washington, D.C., September 2004.

[9] Shelton, R. Orbital Elements Model. Technical Report JSC-61777-utils/orbital_elements, NASA, Johnson Space Center, Houston, Texas, January 2026.

[10] Spencer, A. Reference Frame Model. Technical Report JSC-61777-utils/ref_frames, NASA, Johnson Space Center, Houston, Texas, January 2026.